

Software Architecture

Architecture Means – Microservices-based Architecture

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

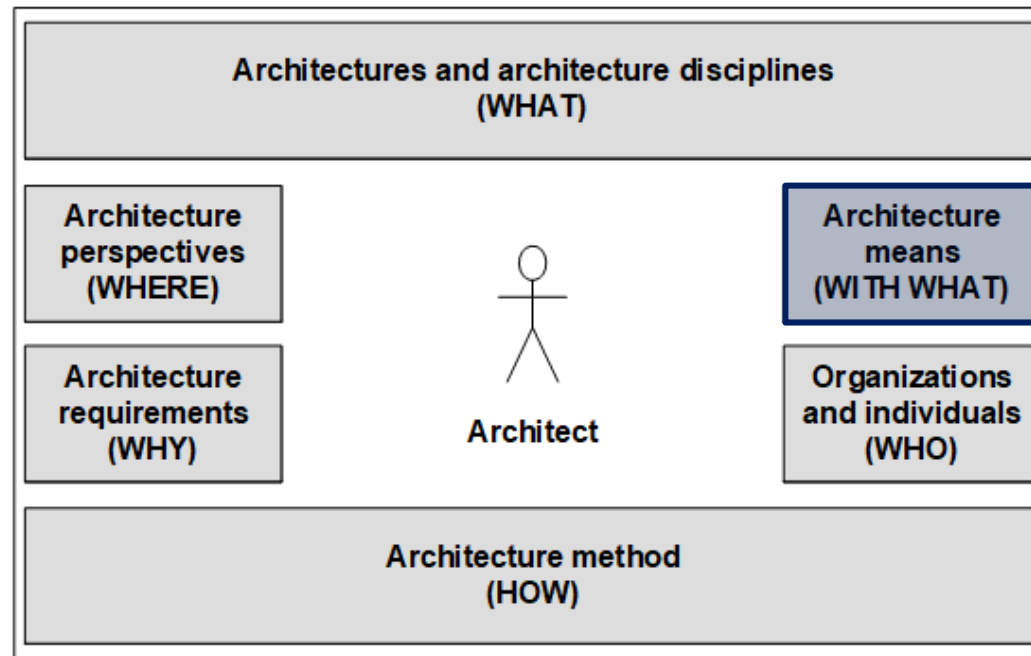
These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Lecture Opening

Architecture Orientation Framework



Architecture **WITH WHAT** discusses the broad range of architectural means architects use to plan, develop, and operate architectures. Thus what we consider here is the architect's toolbox, if you wish. [Vogel, Arnold et al 2011].



Lecture Opening

Motivation

Microservices are not a trend, but a response to real organizational and technical scaling problems.

Traditional monoliths provide centralized simplicity, but suffer from (a) tight coupling of UI/logic/data, (b) limited modifiability and deployment inflexibility, as well as (c) slow scaling.

Microservices enable independent deployment, team ownership, autonomous pipelines, fine-grained scalability, improved modularization & decoupling, and reuse across applications, even unknown future contexts.

The principles remain the same (modularity, abstraction, SoC, explicit interfaces), but microservices push them further.

Lecture Opening

Learning Objectives

You ...

- Define Microservice Architectures and explain how they differ from monoliths in all 4+1 Views.
- Model Microservices in Logical, Development, Process, and Physical Views.
- Explain key Microservice Patterns, including ...
 - API Gateway
 - Service Registry
 - Messaging
 - Hexagonal Architecture
- Apply Microservice Persistence Patterns:
 - Database per Service
 - Event Sourcing and Change Data Capture (CDC)
 - Saga Pattern
- Describe Security Mechanisms in Microservices (Firewall Proxy, RBAC, XACML).
- Discuss Organizational Impact of adopting the Microservices-oriented Architecture Style.

Lecture Agenda



- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Lifecycle Management
- Organizational Impact

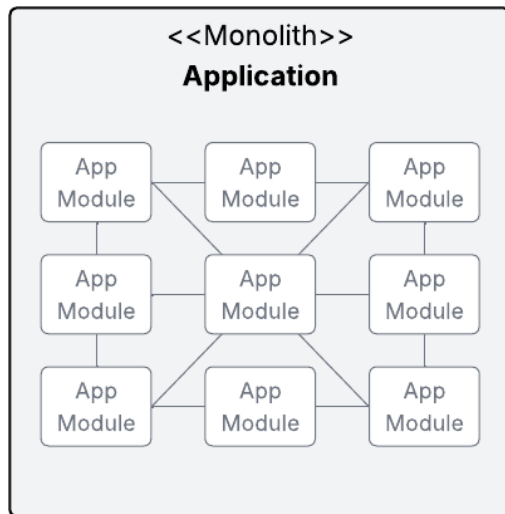
Motivation

Microservices are not a passing trend

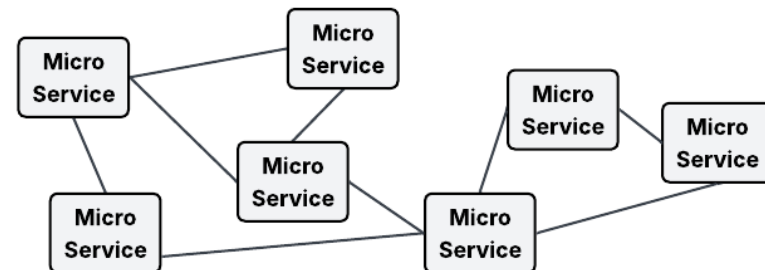
Microservices in the **context** of modern **Software Architecture** are not a **fashion**, but a **response** to **real organizational** and **technical scaling problems**.

Modularity has always been a goal of good **Software Architecture**.

However, a **monolithic Application** is defined by the fact that its **modules** are **all part** of the **same deployment unit**.



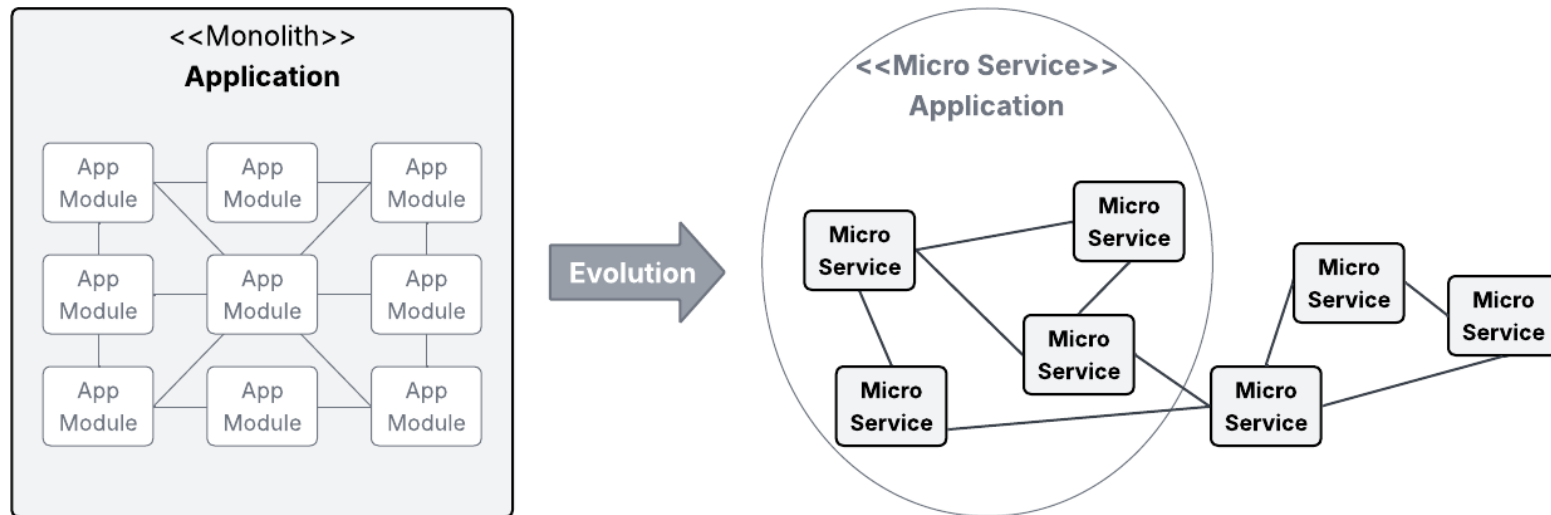
Microservices go further regarding **Modularization**, with each **Microservice** representing its own **deployment unit** and also being **largely self-contained**.



Motivation

Microservices are not a passing trend

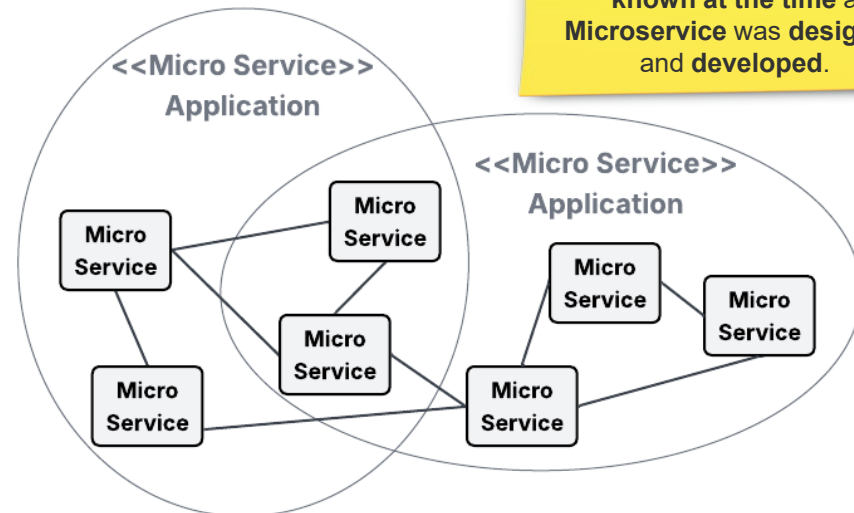
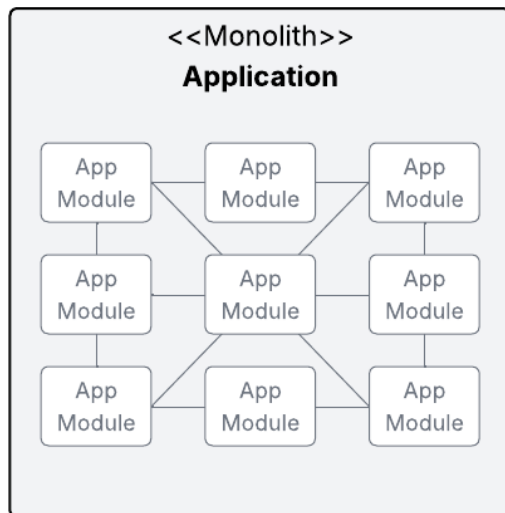
Microservices in the **context** of **modern Software Architecture** are not a **fashion**, but a **response** to **real organizational** and **technical scaling problems**.



Motivation

Microservices are not a passing trend

Microservices in the **context** of **modern Software Architecture** are not a **fashion**, but a **response** to **real organizational** and **technical scaling problems**.



... even in **those** that were **not known at the time** a **Microservice** was **designed** and **developed**.

Motivation

Microservices are not a passing trend

The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.

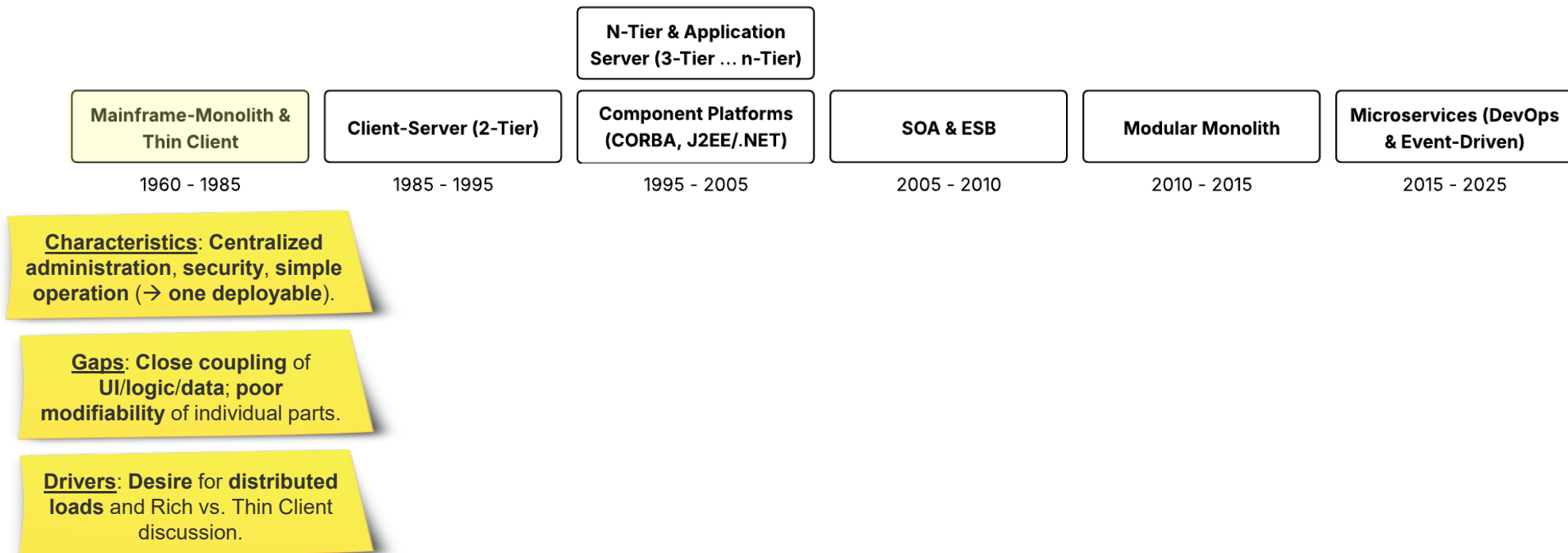
- **Modularity & Abstraction.** Clear Building Blocks with explicit interfaces promote loose coupling and high cohesion.
- **Information Hiding & Separation of Concerns.** Hide internal details and clearly separate responsibilities as a basis for interchangeability.
- **Design for Change & Open/Closed.** Design with change in mind – keep Building Blocks open for extension as well as closed against uncontrolled access.
- **Explicit Interfaces & Interface Segregation.** Design stable contracts with low commitment to respective implementations.
- **Traceability & Self-Documentation.** Decisions and structures are traceable, and anchored in each respective artifact.

Basically **all Architecture Principles** aim at **reducing complexity** and **gaining flexibility** – especially through **loose coupling** and **high cohesion**.

Motivation

Microservices are not a passing trend

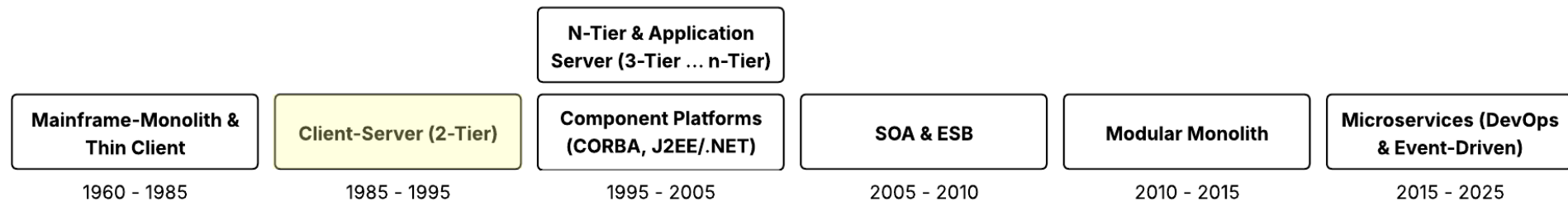
The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Motivation

Microservices are not a passing trend

The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Progress: decoupling of data storage and better separation of Client/UI versus Server/DB.

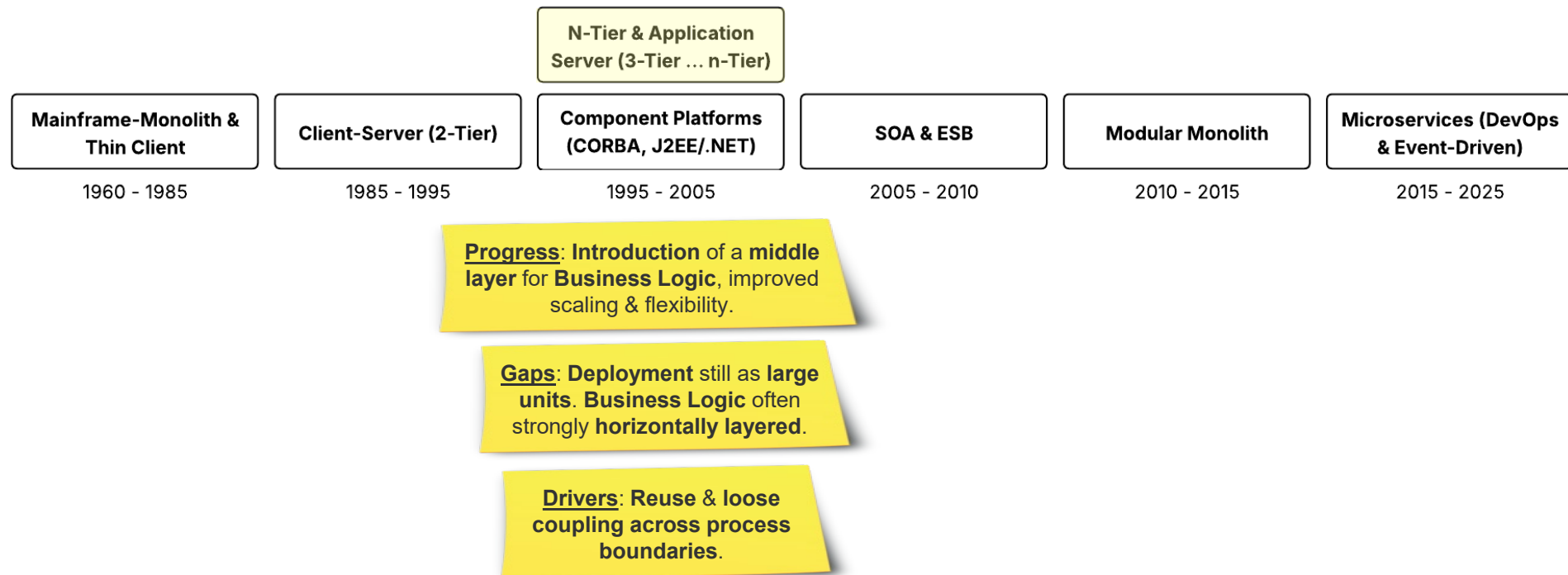
Gaps: Business logic partly in the Client, partly on the Server (e.g., within stored procedures (vendor lock-in)).

Drivers: Scalability, portability, and less distributed (more central) Business Logic.

Motivation

Microservices are not a passing trend

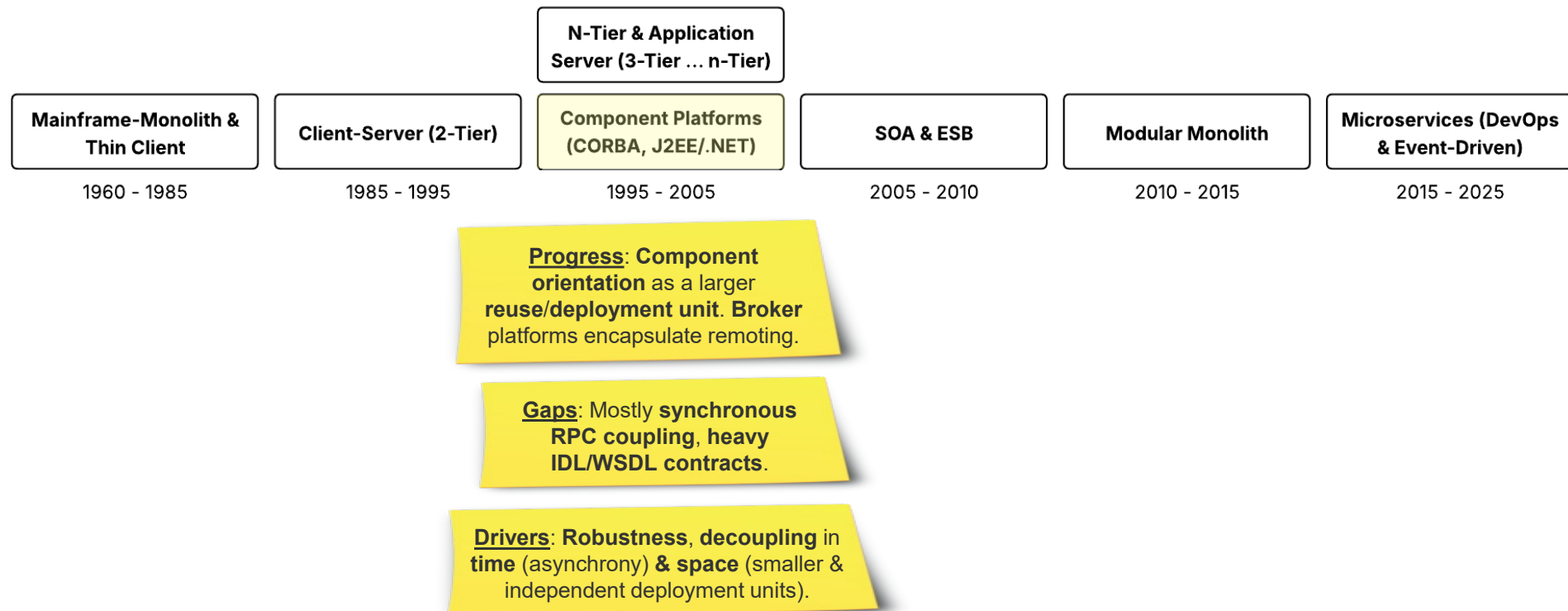
The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Motivation

Microservices are not a passing trend

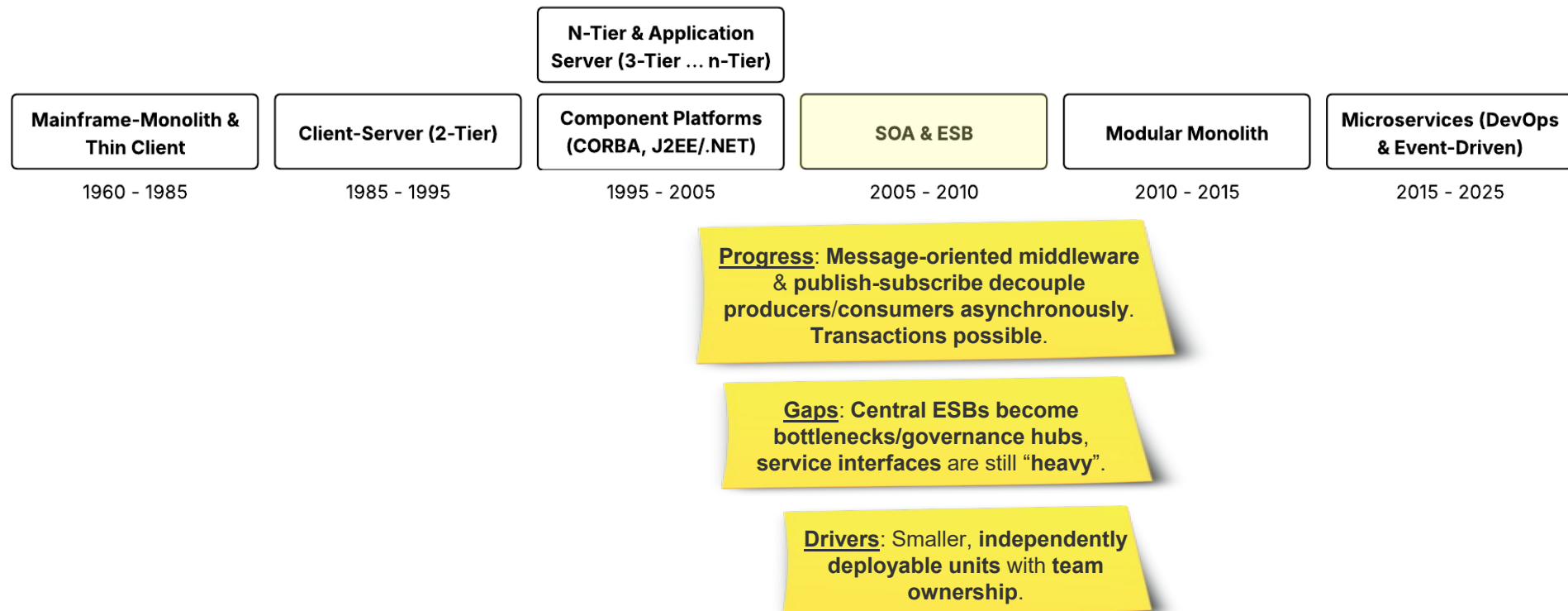
The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Motivation

Microservices are not a passing trend

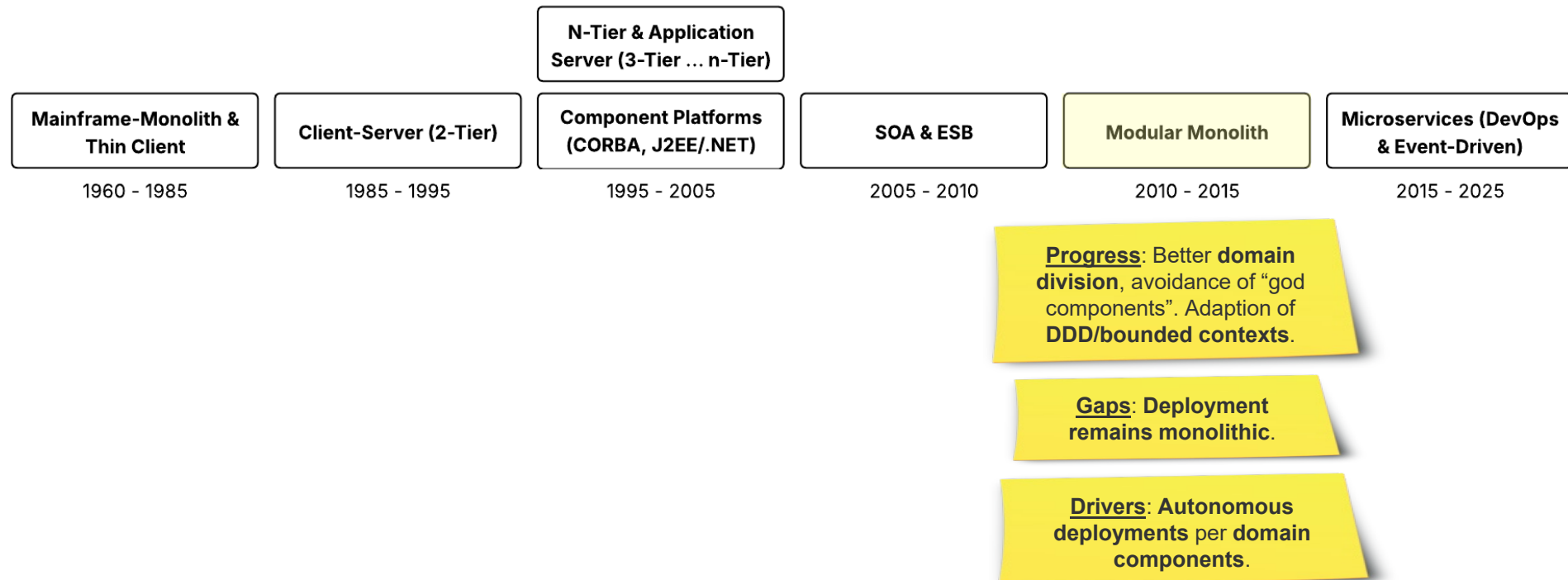
The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Motivation

Microservices are not a passing trend

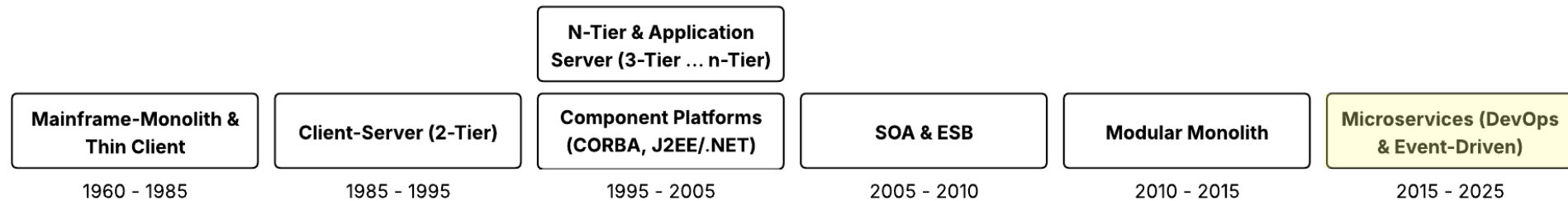
The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Motivation

Microservices are not a passing trend

The **fundamental principles** for **high-quality architectures** have **remained largely unchanged** – both for **monolithic** and for **microservice-based applications**.



Progress: Independently deployable services with own data.

Y-/Z-/X-scaling (function, partition, cloning).

Teams own services end-2-end.

Hexagonal architecture separates business logic from consumers via adapters.

Pub/Sub for loose, anonymous coupling.

Lecture Agenda



- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

Definition

Overview

How **Microservice Architectures** are **defined** in the **Software Industry** ...

A **Microservice Architecture** organizes an **application** into a **collection** of **loosely coupled, fine-grained services** that **communicate via lightweight protocols**.

A **Microservice Architecture** is **characterized** by the **ability** to **develop** and **deploy services independently**, improving modularity, scalability, and adaptability.

However, it **introduces additional complexity**, particularly in **managing distributed systems** and **inter-service communication**, making the **initial implementation more challenging compared to a monolithic architecture**. [Wikipedia]

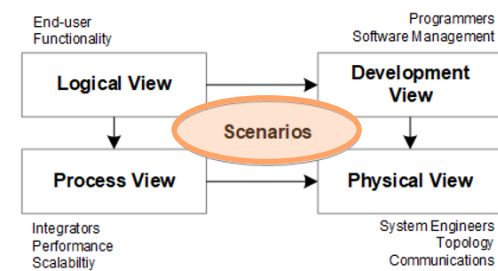
The **distinctive characteristics** of **Microservice Architectures** and hence their **differentiation** from **monolithic Architectures** can be **highlighted particularly well** using the **4+1 View Model**.

Lecture Agenda



- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

4+1 View Model Context Scenarios

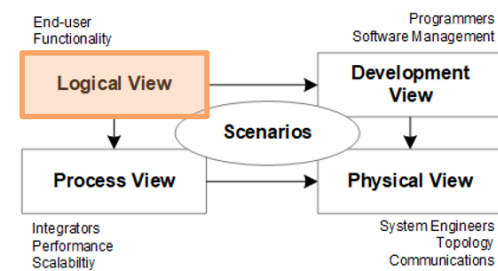


I pick a **Use Case** from Bankolino's **CandidateInsight** application to illustrate the **distinctive characteristics** of **Microservice Architectures**.

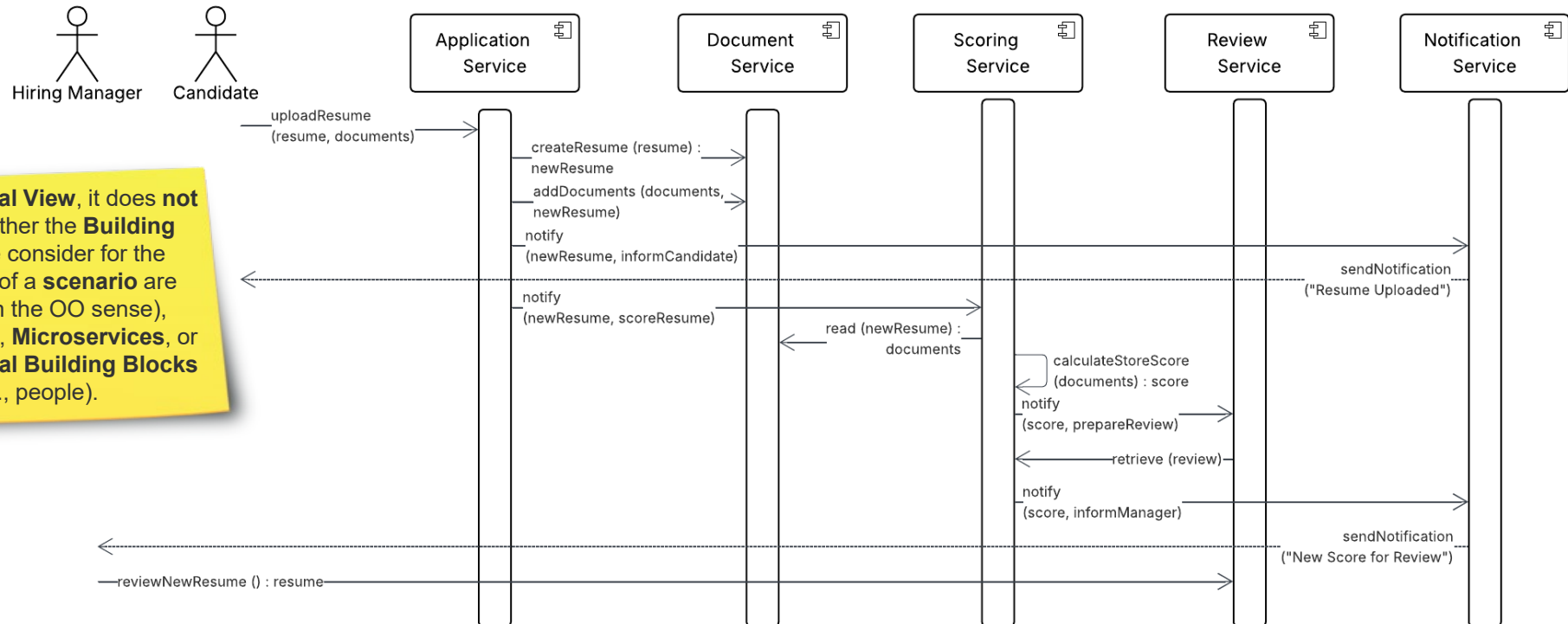
Use Case Name	Candidate submits job application and receives feedback
Description	A candidate applies via the Bankolino portal. Their application is processed, stored, analyzed, and made available to the hiring team via several microservices.
Actor	Candidate (external), Hiring Manager (internal)
Flow of Events	1. Candidate uploads application documents via the Application Portal. 2. The Application Portal informs the Candidate of received documents and analyzes the candidate's resume. 3. The Application Portal notifies the Hiring Manager of the newly analyzed resume. 4. The Hiring Manager sees the analysis result in the Review Dashboard.
Precondition	Candidate and Hiring Manager are registered and logged into platform.
Postcondition	New candidate resume is created.

4+1 View Model Context

Logical View



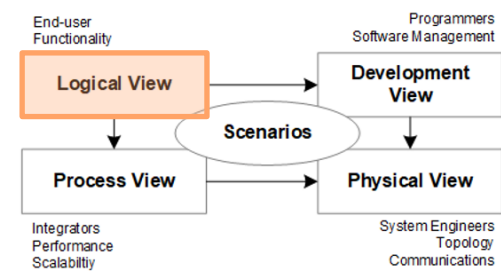
The **focus** of the **Logical View**, as you know, is to **describe** how the **Building Blocks** of a solution (possibly microservices) **cooperate** to **realize** a **given use case**.



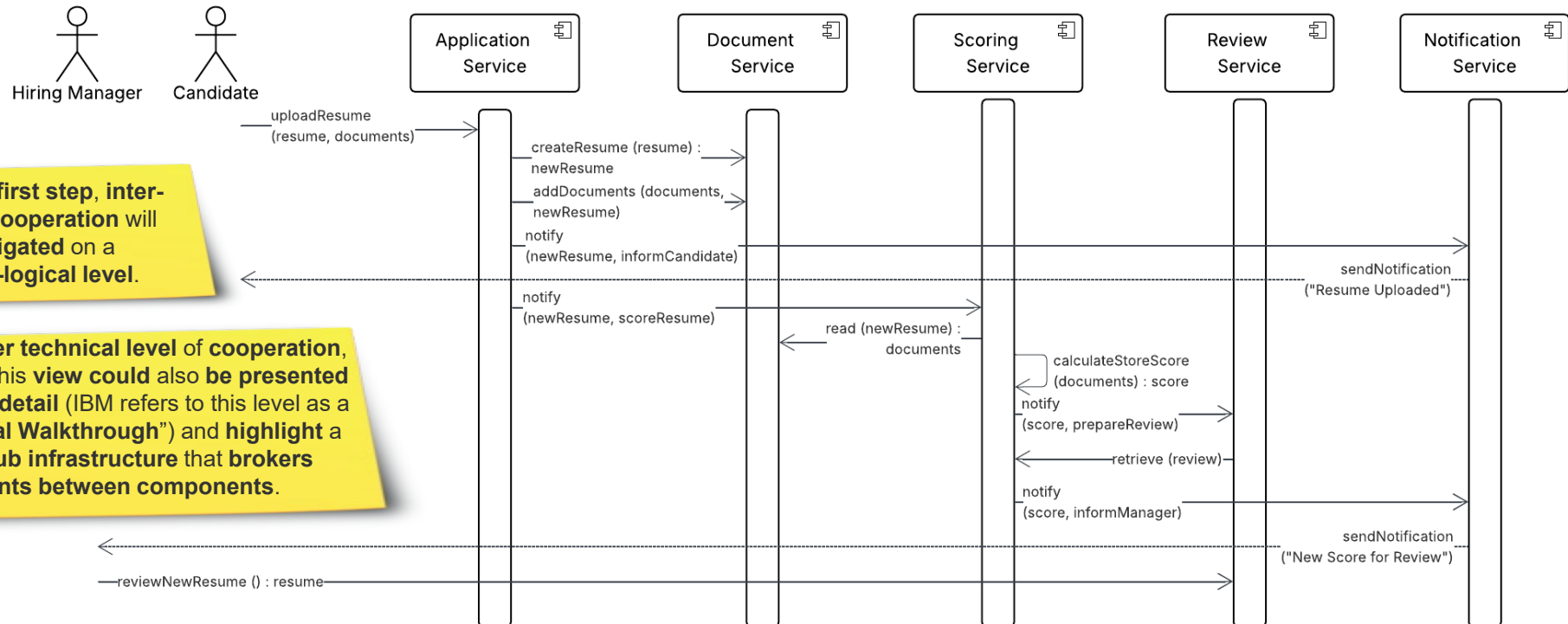
For the **Logical View**, it does **not** matter whether the **Building Blocks** we consider for the realization of a **scenario** are **Objects** (in the OO sense), **Components**, **Microservices**, or **Organizational Building Blocks** (e.g., people).

4+1 View Model Context

Logical View



The **focus** of the **Logical View**, as you know, is to **describe** how the **Building Blocks** of a solution (possibly microservices) **cooperate** to **realize** a **given use case**.

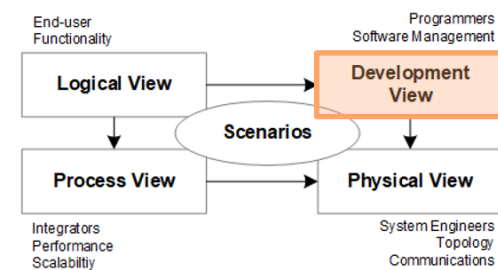


Typically, in a **first step**, **inter-component cooperation** will be **investigated** on a **conceptual-logical level**.

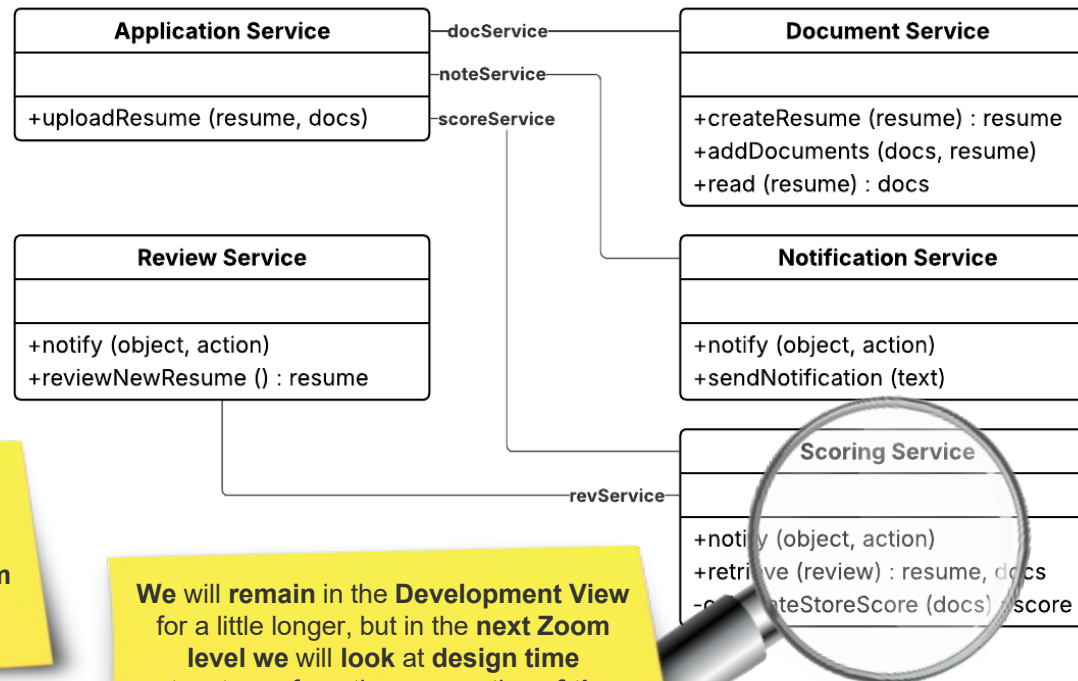
On a **rather technical level** of **cooperation**, however, this **view** could also be **presented** in **greater detail** (IBM refers to this level as a **"Technical Walkthrough"**) and **highlight** a **Pub-Sub infrastructure** that **brokers** events **between components**.

4+1 View Model Context

Development View



The **Development View** provides, as we know, information about the design time structures that can be derived from the **Logical View**.

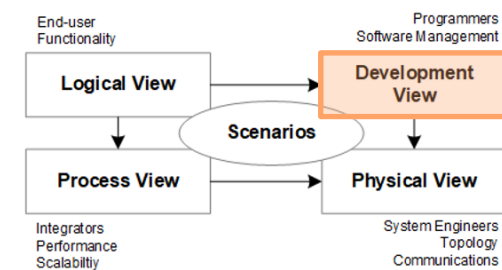


Despite this **example involving coarse-grained structures** (e.g., Components, Microservices), the **UML diagram type Class Diagram** is well suited for capturing a **Development View model**.

We will remain in the **Development View** for a little longer, but in the **next Zoom level** we will look at **design time structures** from the perspective of the associated **Java Deployment Unit**.

4+1 View Model Context Development View

The **Development View** provides, as we know, information about the design time structures that can be derived from the **Logical View**.

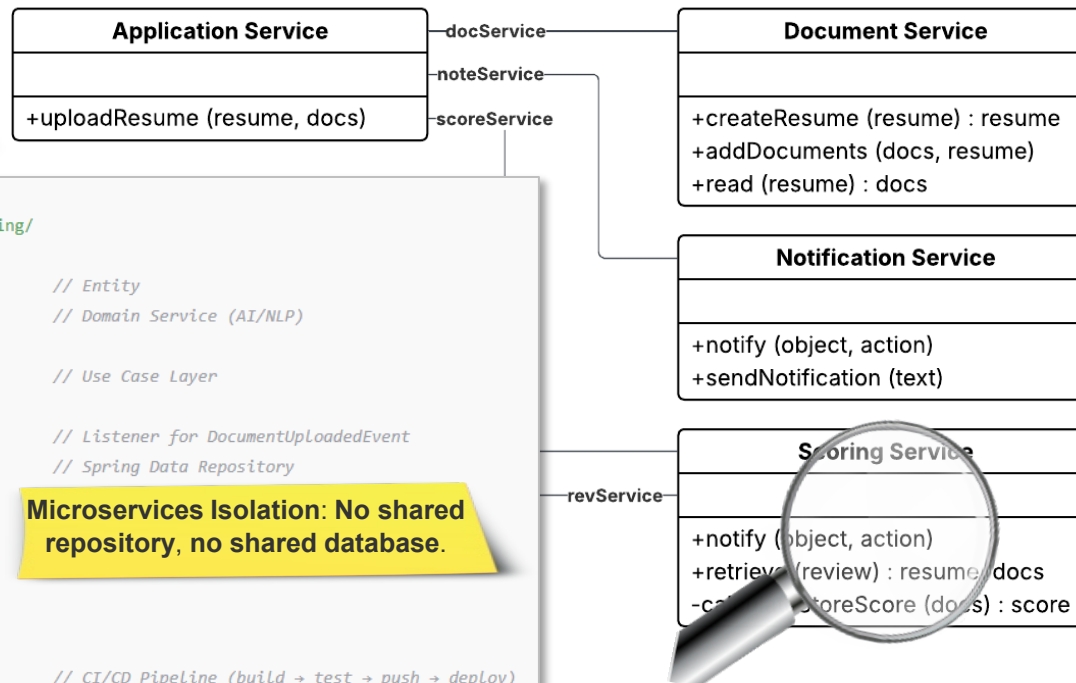


Each Microservice has its own repository with its own build pipeline and deployment automation.

```
/scoring-service
├── src/main/java/com/bankolino/scoring/
│   ├── domain/
│   │   ├── CandidateScore.java    // Entity
│   │   └── ResumeAnalyzer.java    // Domain Service (AI/NLP)
│   ├── application/
│   │   └── ScoringService.java    // Use Case Layer
│   ├── infrastructure/
│   │   ├── KafkaConsumer.java    // Listener for DocumentUploadedEvent
│   │   └── ScoringRepository.java // Spring Data Repository
│   ├── api/
│   │   └── ScoringController.java
│   └── test/
│       └── ScoringServiceTest.java
├── Dockerfile
├── build.gradle
└── .github/workflows/ci.yml    // CI/CD Pipeline (build → test → push → deploy)
```

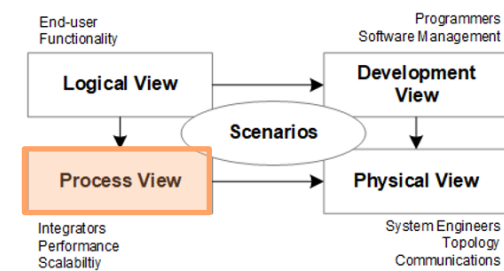
Microservices Isolation: No shared repository, no shared database.

Autonomous pipelines: Each Microservice can be versioned, tested, and delivered independently.



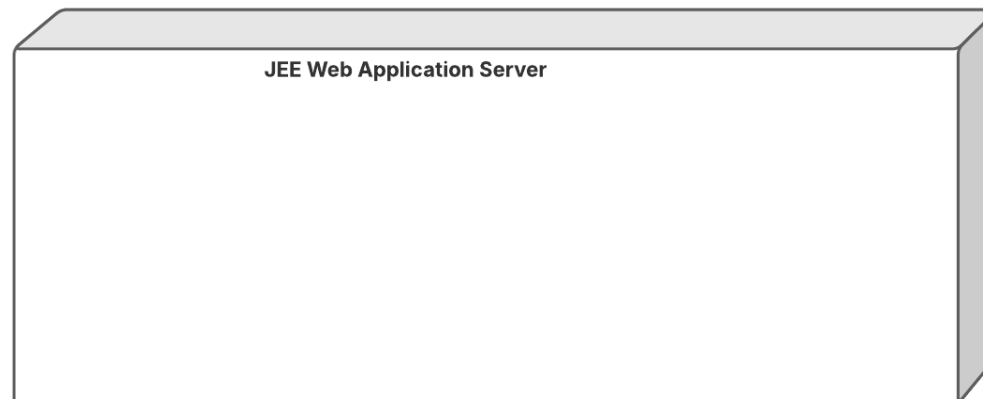
4+1 View Model Context

Process View



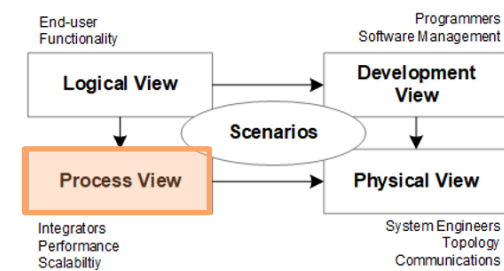
Latest when looking at the **Process View**, **Microservice Architectures** and **Monolithic Architectures** diverge significantly.

While we would see one or a few execution environment(s) for a **Monolithic Application** ...

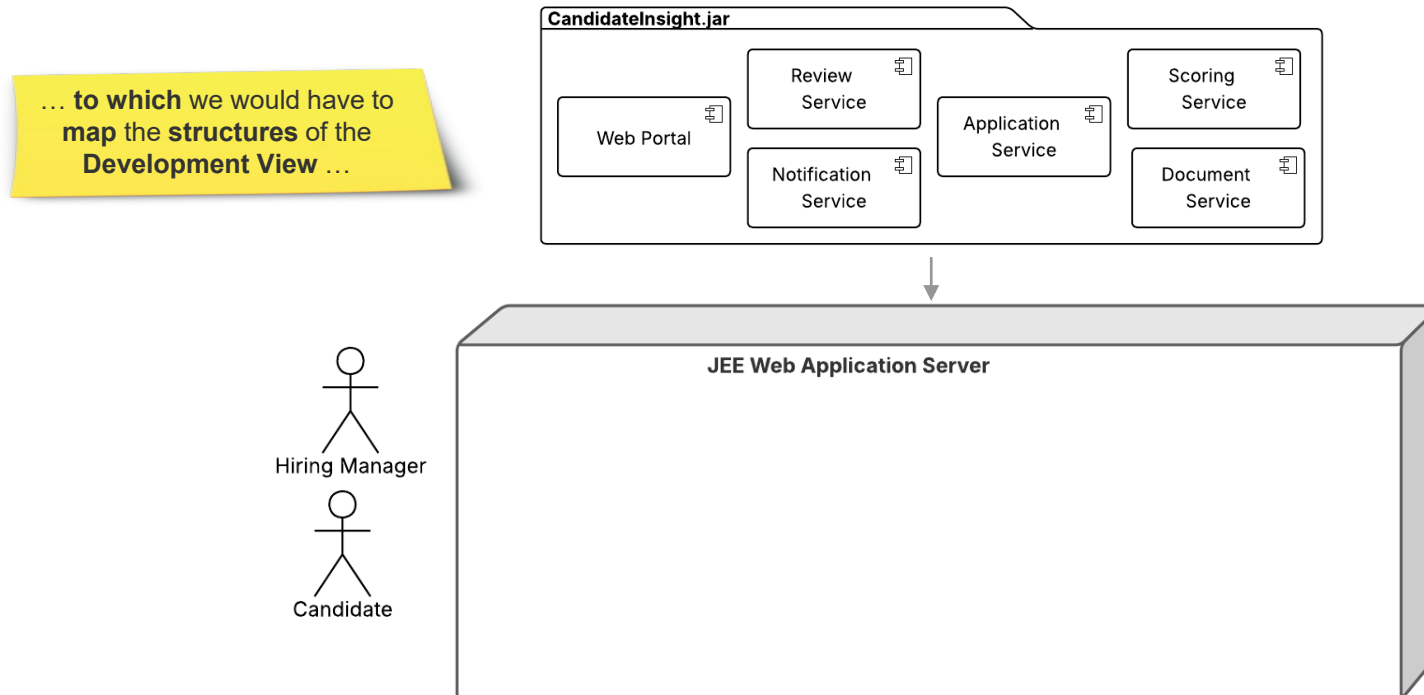


4+1 View Model Context

Process View



Latest when looking at the **Process View**, **Microservice Architectures** and **Monolithic Architectures** diverge significantly.

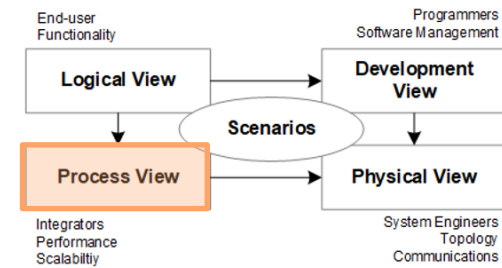
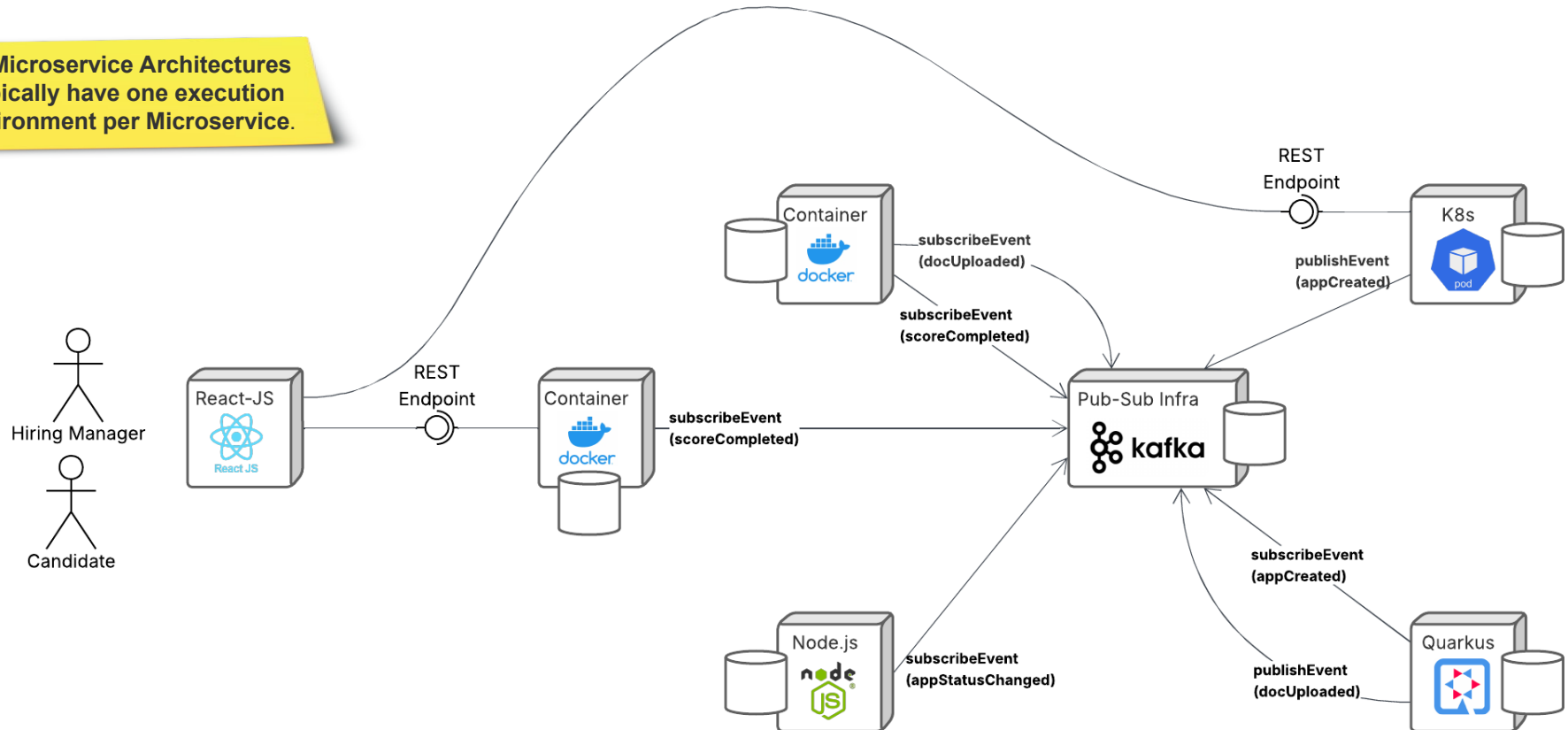


4+1 View Model Context

Process View

Latest when looking at the Process View, Microservice Architectures and Monolithic Architectures diverge significantly.

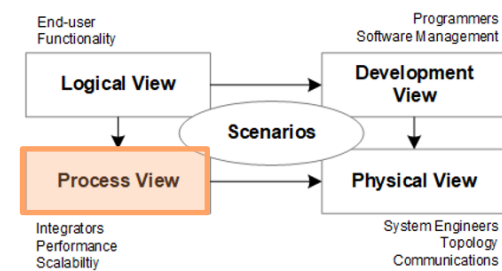
... Microservice Architectures typically have one execution environment per Microservice.



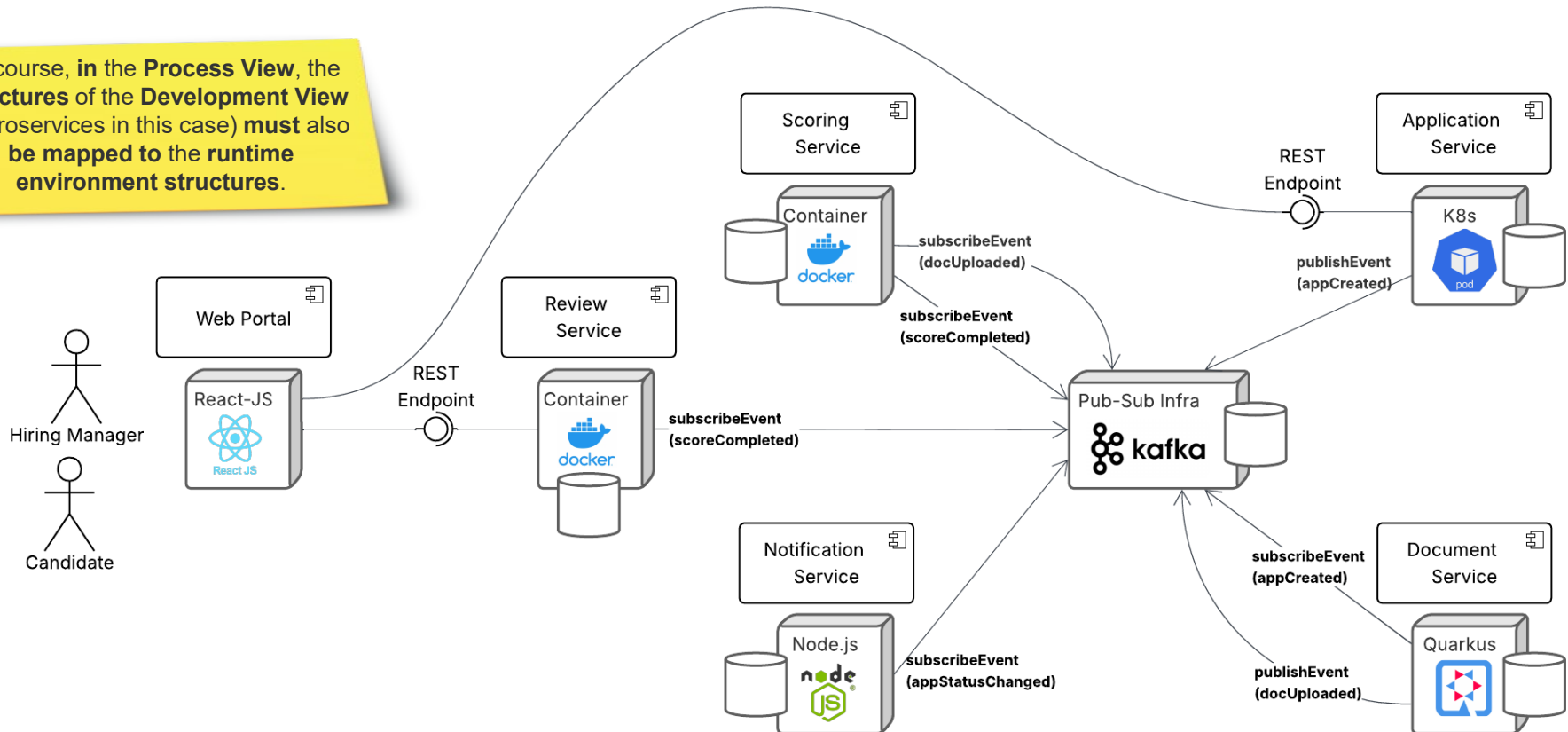
4+1 View Model Context

Process View

Latest when looking at the **Process View**, **Microservice Architectures** and **Monolithic Architectures** diverge significantly.

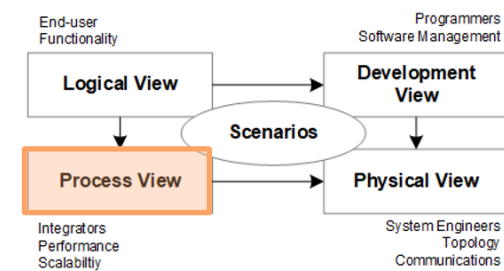


Of course, in the **Process View**, the **structures** of the **Development View** (Microservices in this case) **must** also **be mapped** to the **runtime environment structures**.



4+1 View Model Context

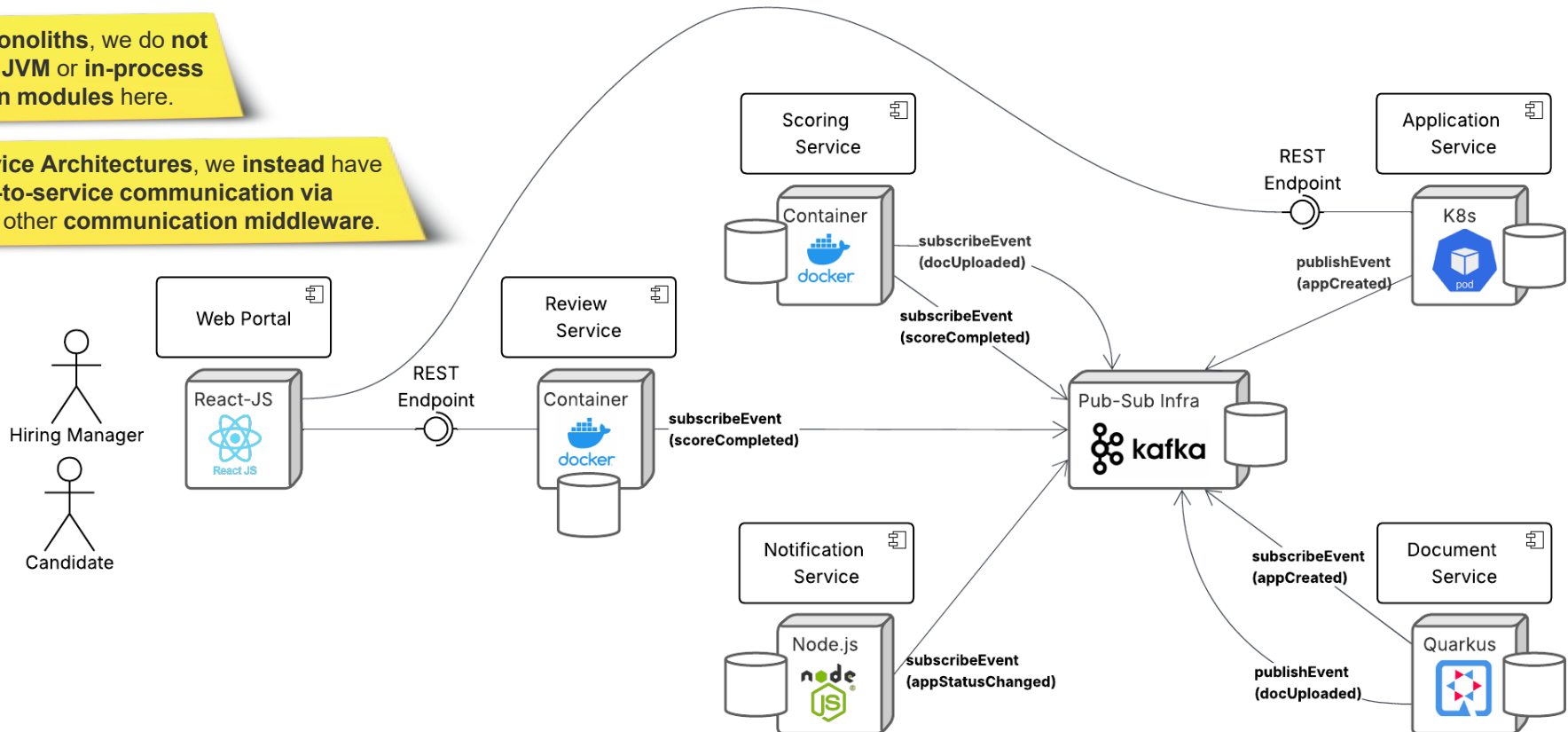
Process View



Latest when looking at the **Process View**, **Microservice Architectures** and **Monolithic Architectures** diverge significantly.

In contrast to **Monoliths**, we do not see any **shared JVM** or **in-process calls** between modules here.

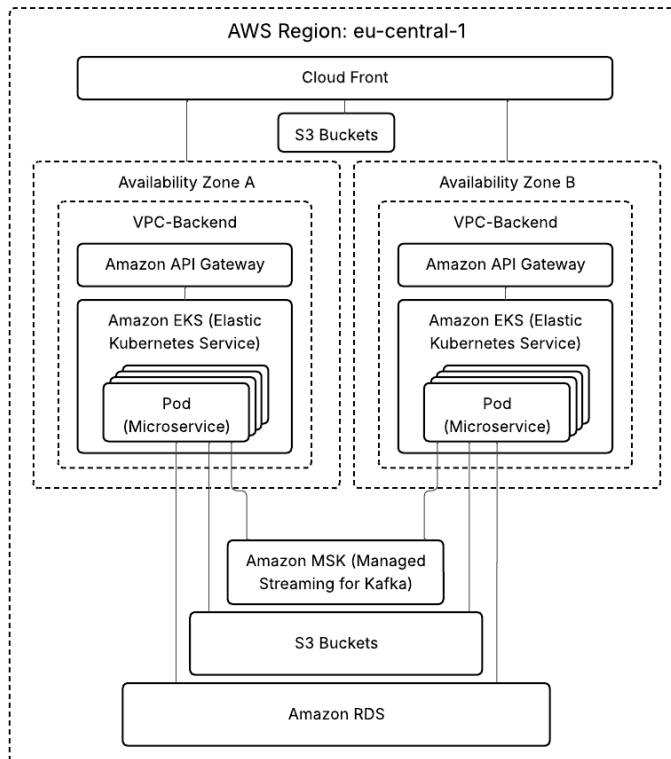
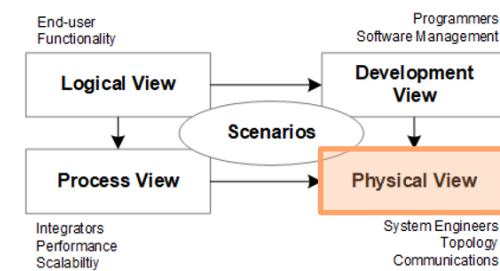
In **Microservice Architectures**, we instead have **service-to-service communication via networks** or other **communication middleware**.



4+1 View Model Context

Physical View

Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e.. **Software**).



The **example** here shows **hardware-related services** from **AWS** – a cloud hyperscaler.

The **physical system design** proposes a **single AWS region** (e.g., eu-central-1), ...

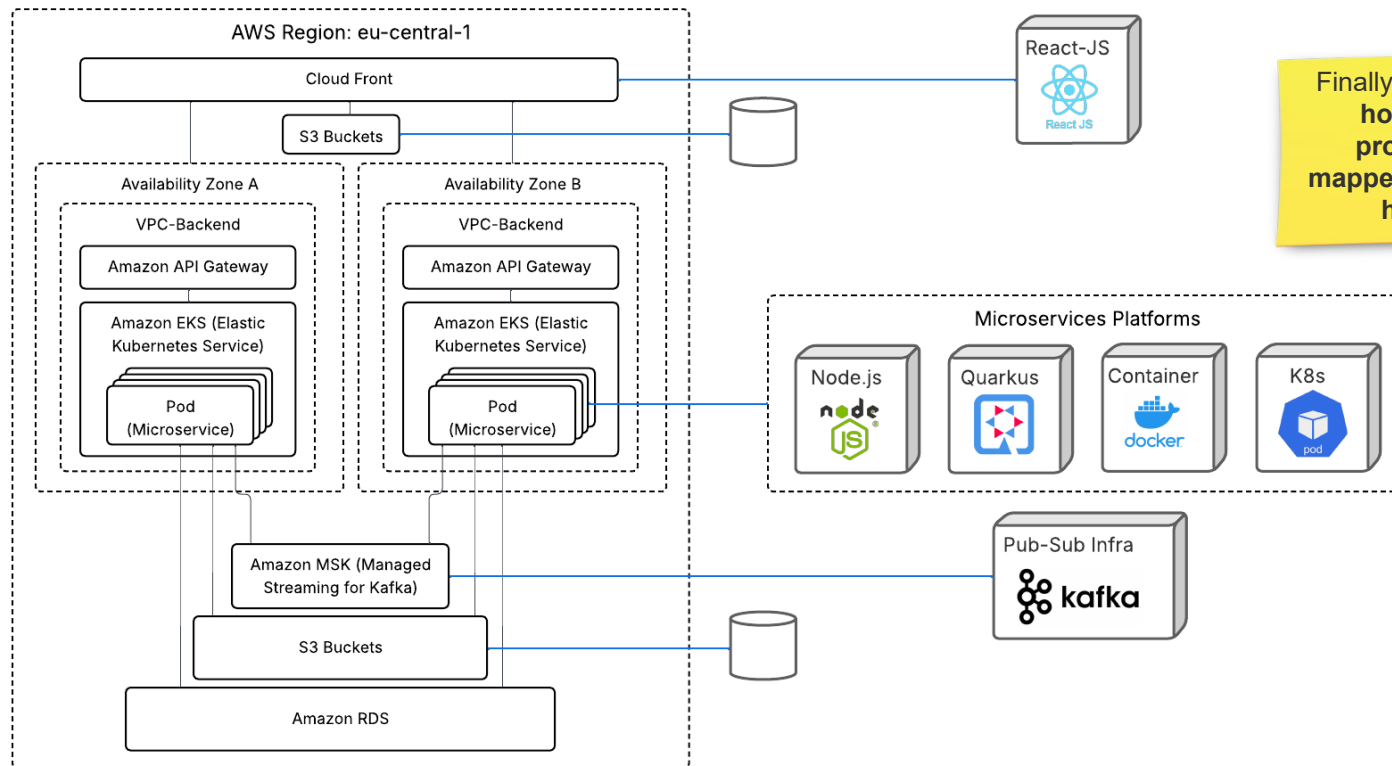
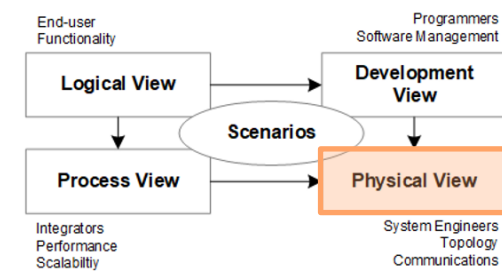
... **distributed across two availability zones (AZ-A / AZ-B)** for high availability.

Virtual Private Cloud (VPC) networks form a logical network boundary.

4+1 View Model Context

Physical View

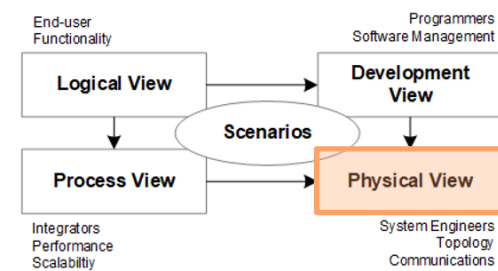
Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).



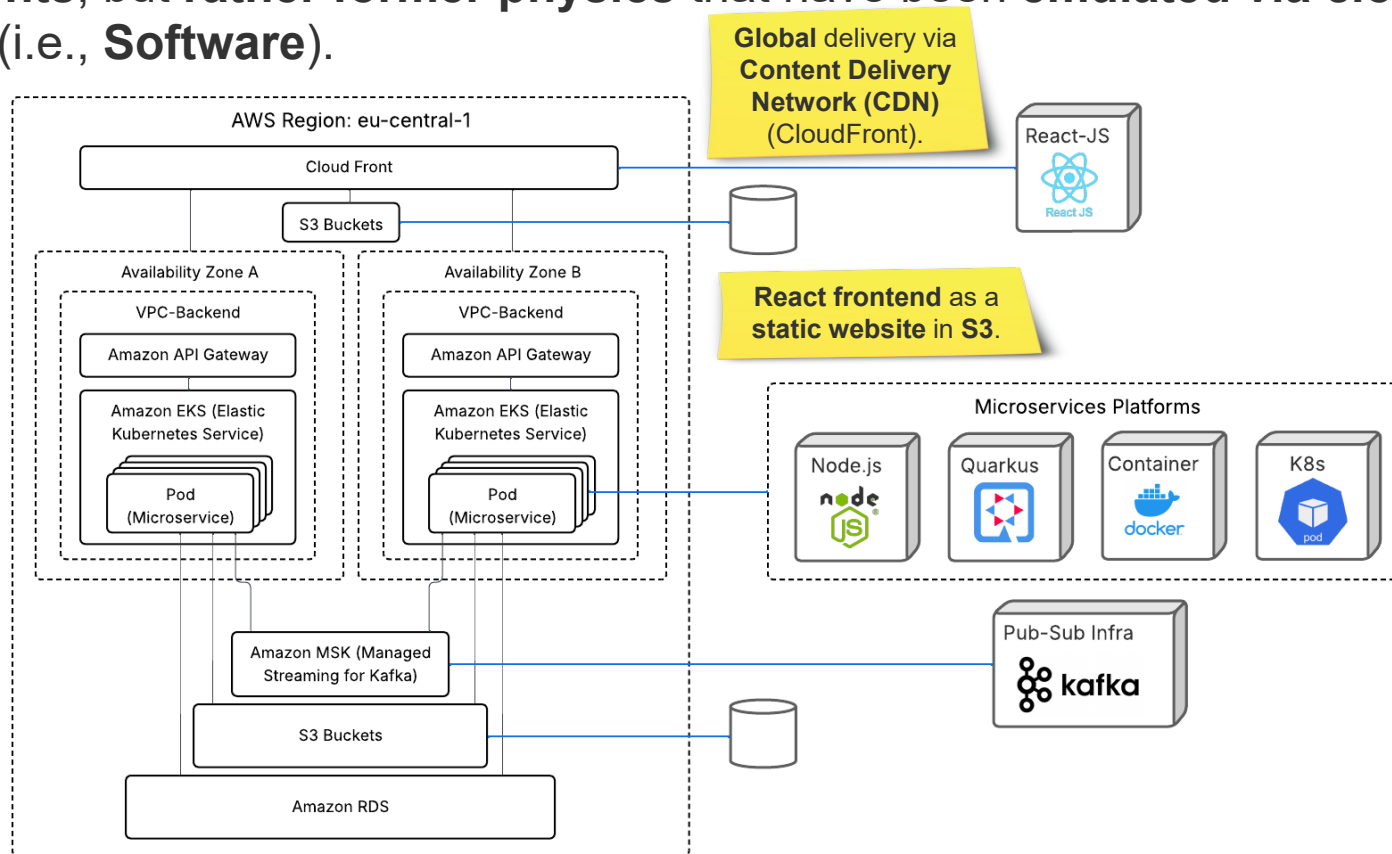
Finally, we will look at how platform processes are mapped to “physical” hardware.

4+1 View Model Context

Physical View



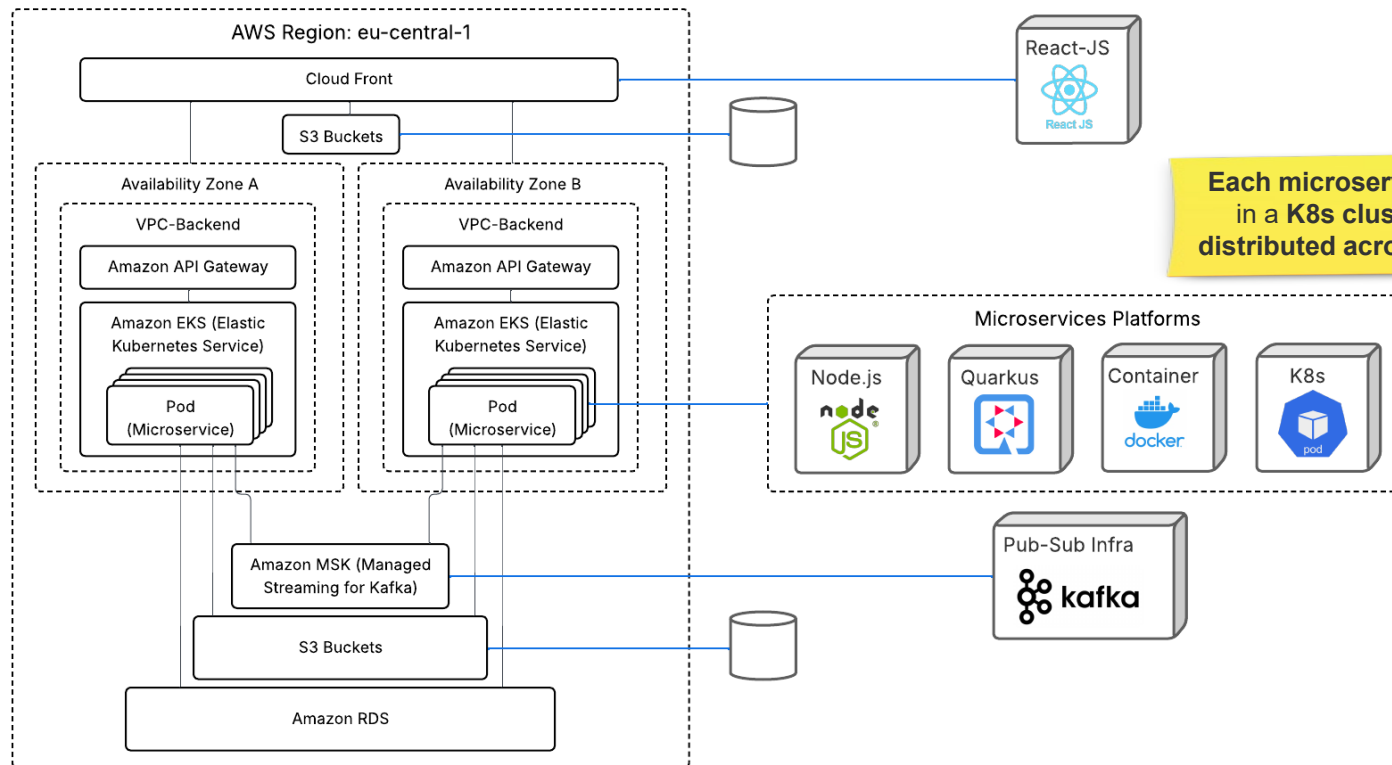
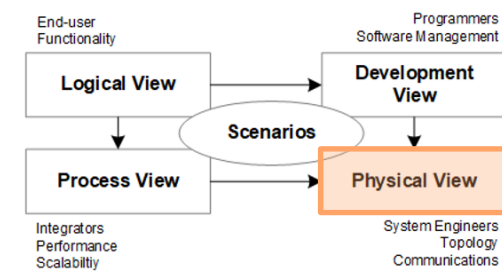
Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).



4+1 View Model Context

Physical View

Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).

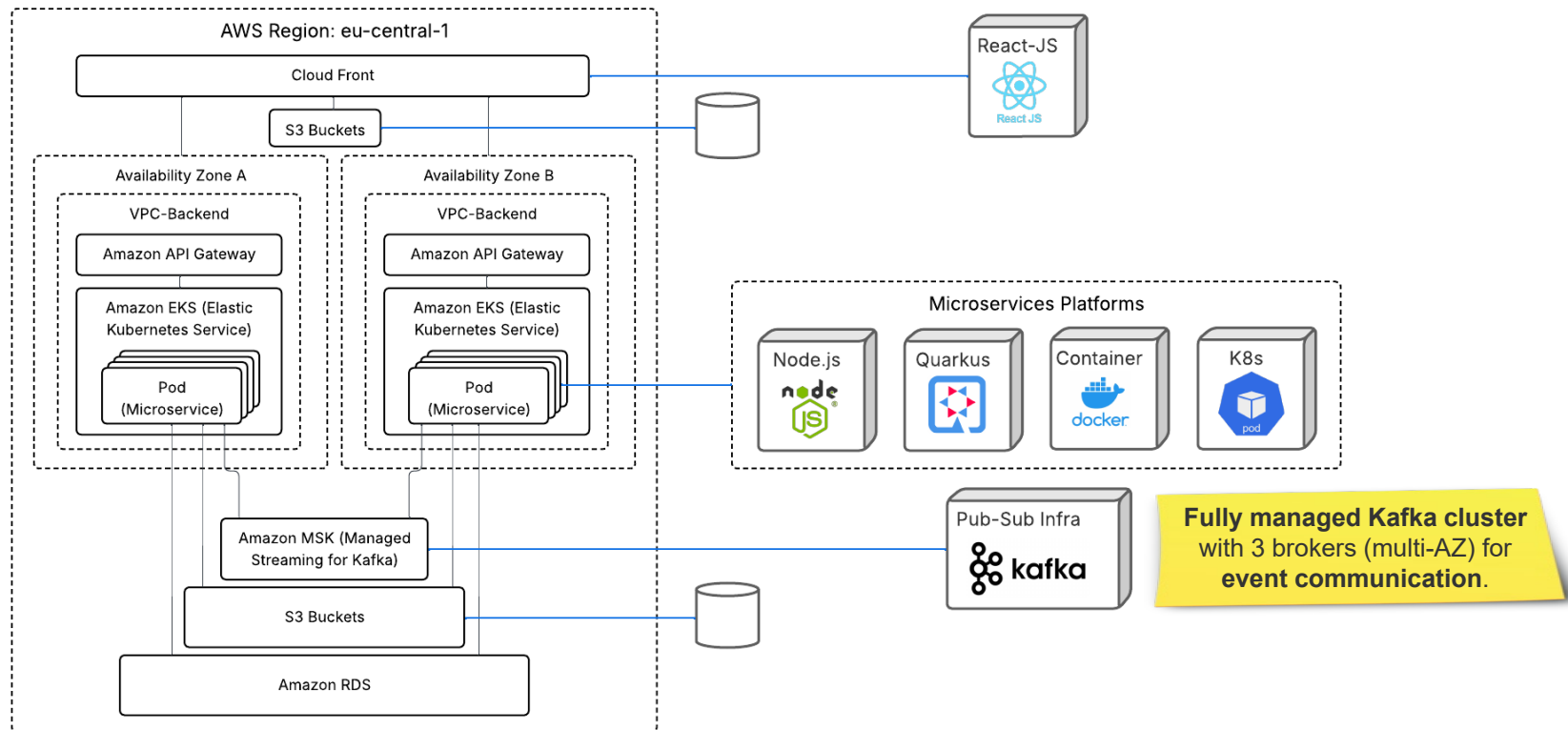
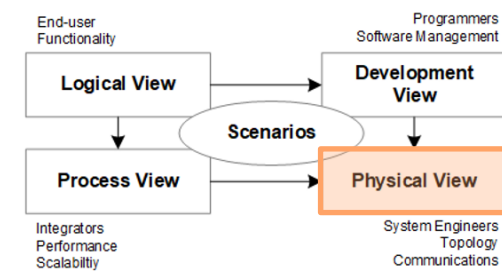


Each microservice runs as a pod in a K8s cluster with replicas distributed across AZ-A and AZ-B.

4+1 View Model Context

Physical View

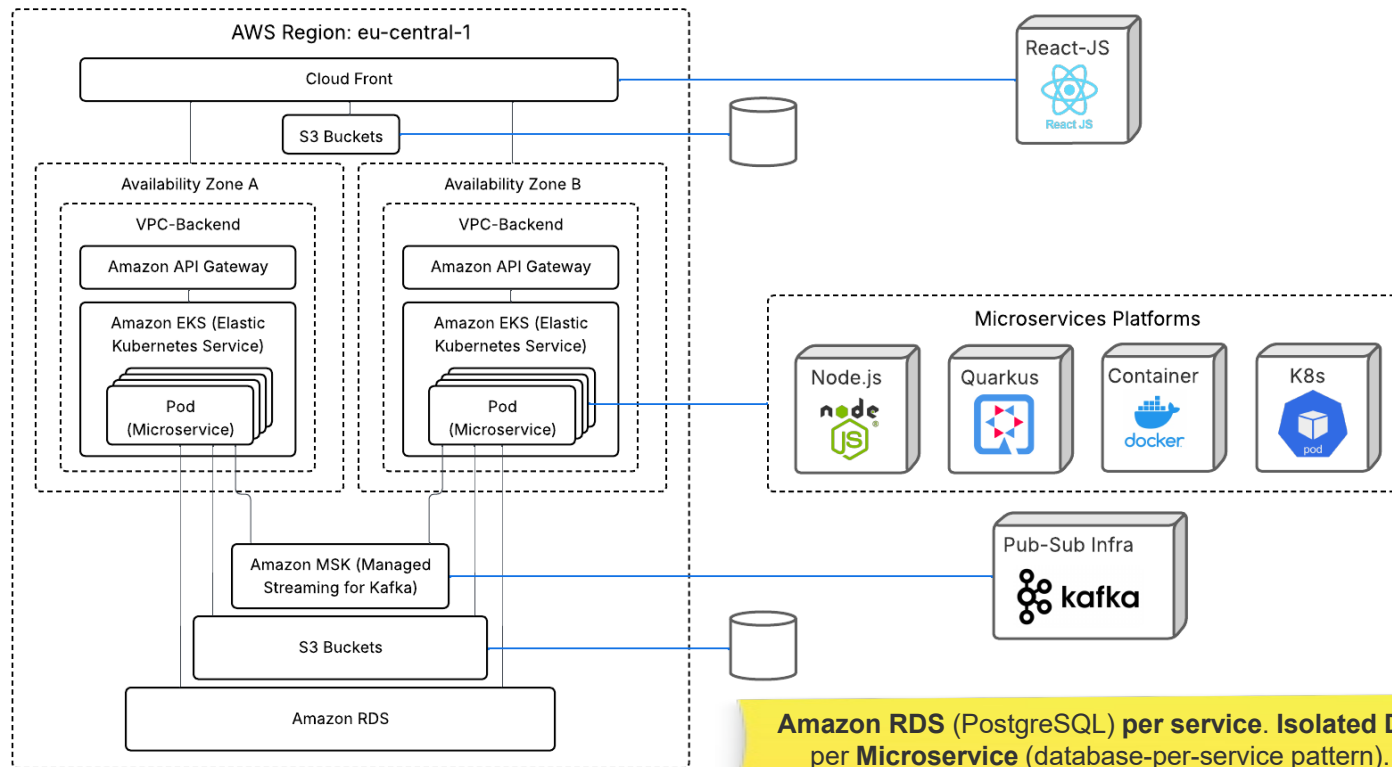
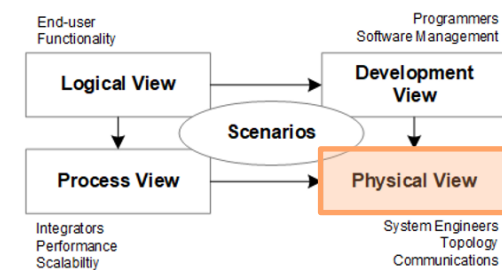
Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).



4+1 View Model Context

Physical View

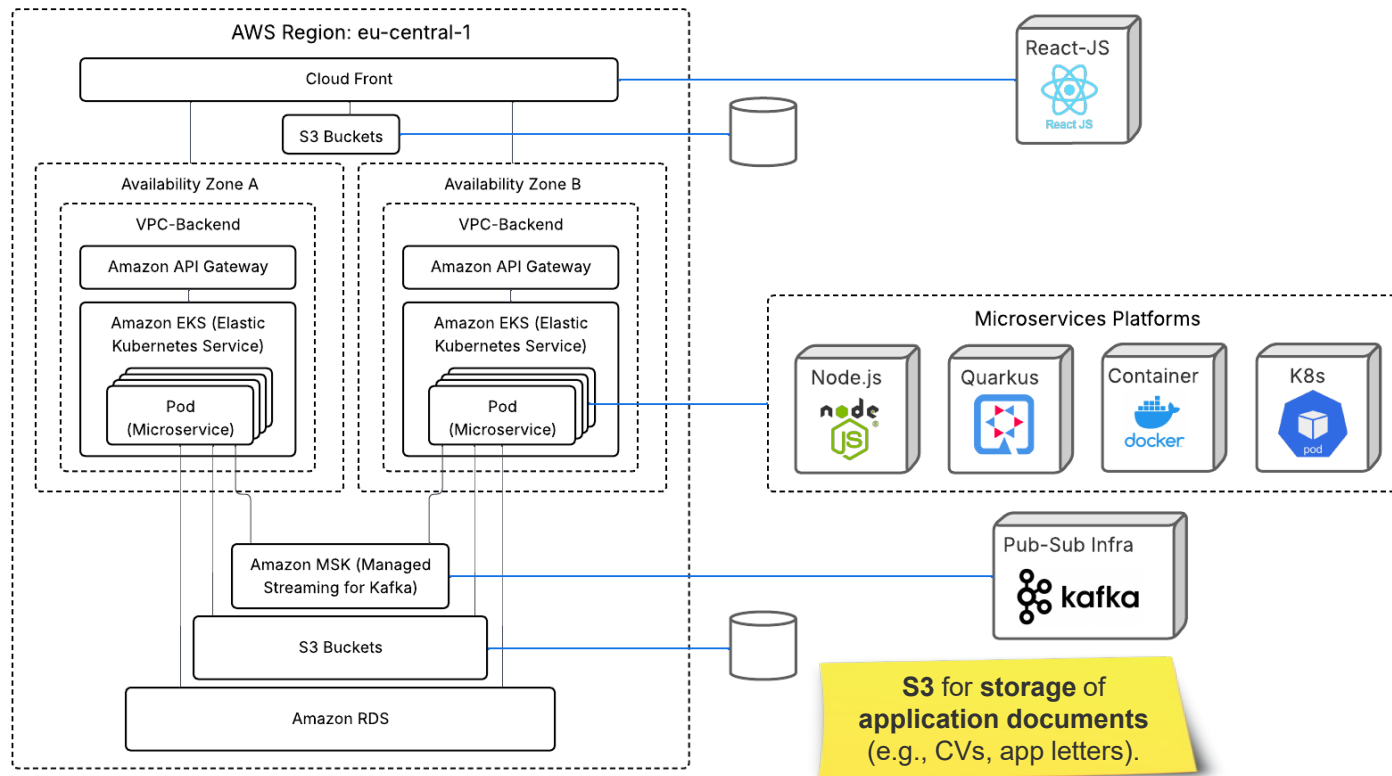
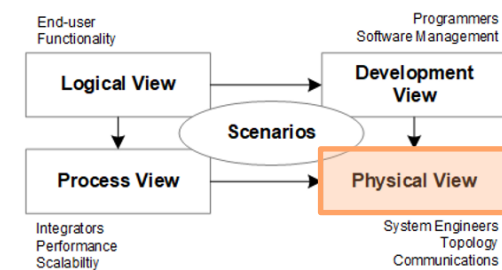
Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).



4+1 View Model Context

Physical View

Finally, the **Physical View** today often no longer considers actual physical components, but rather former physics that have been emulated via cloud services (i.e., **Software**).



Lecture Agenda



- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

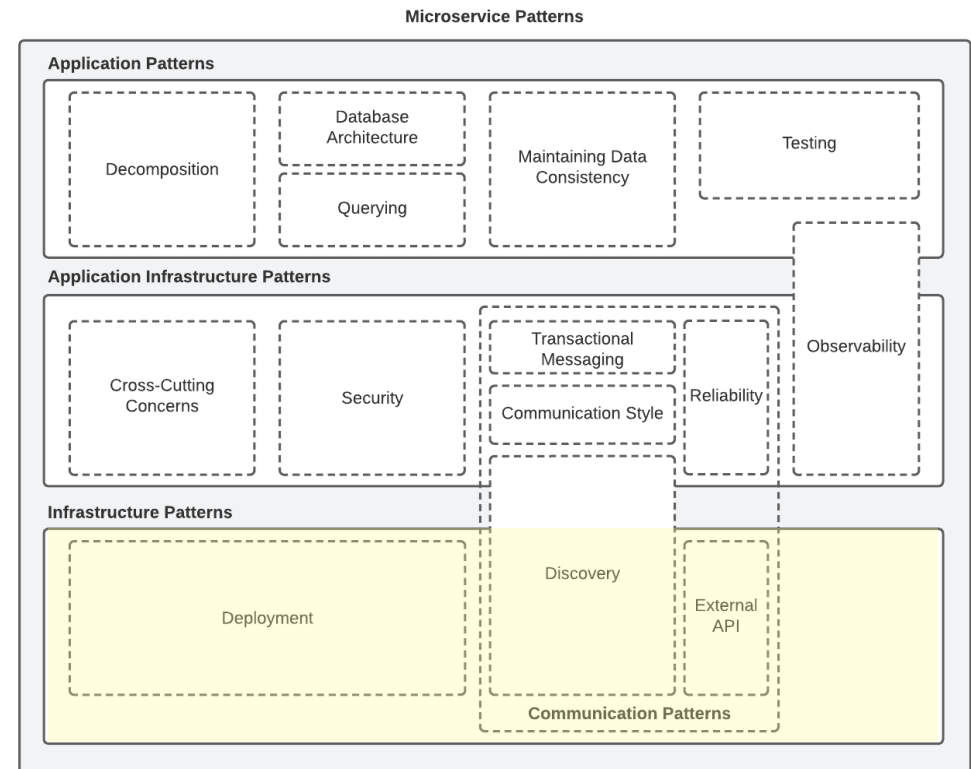
Architecture Patterns

Pattern Language

Microservice Architectures suggest a broad set of patterns as useful means for their realization.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.



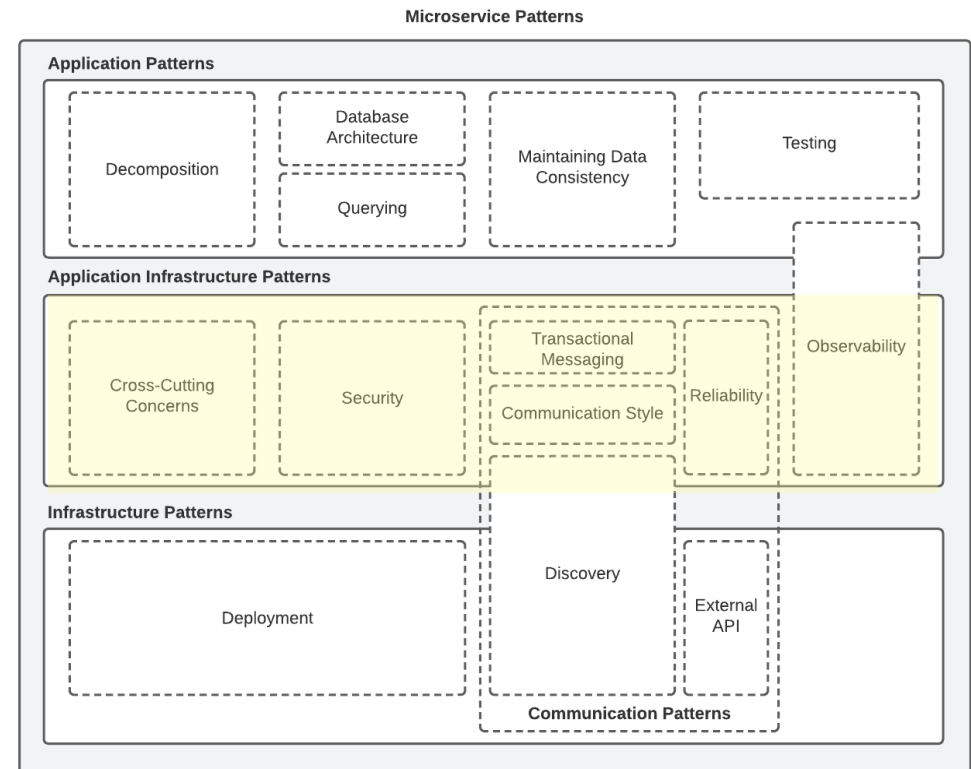
Architecture Patterns

Pattern Language

Microservice Architectures suggest a broad set of patterns as useful means for their realization.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.



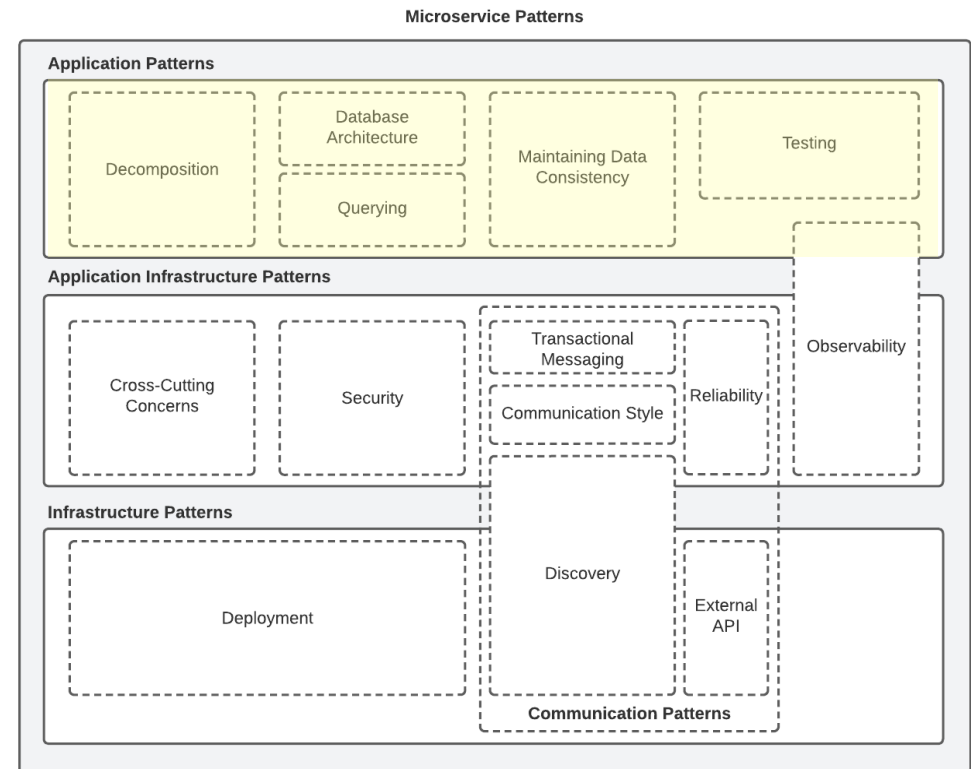
Architecture Patterns

Pattern Language

Microservice Architectures suggest a broad set of patterns as useful means for their realization.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.



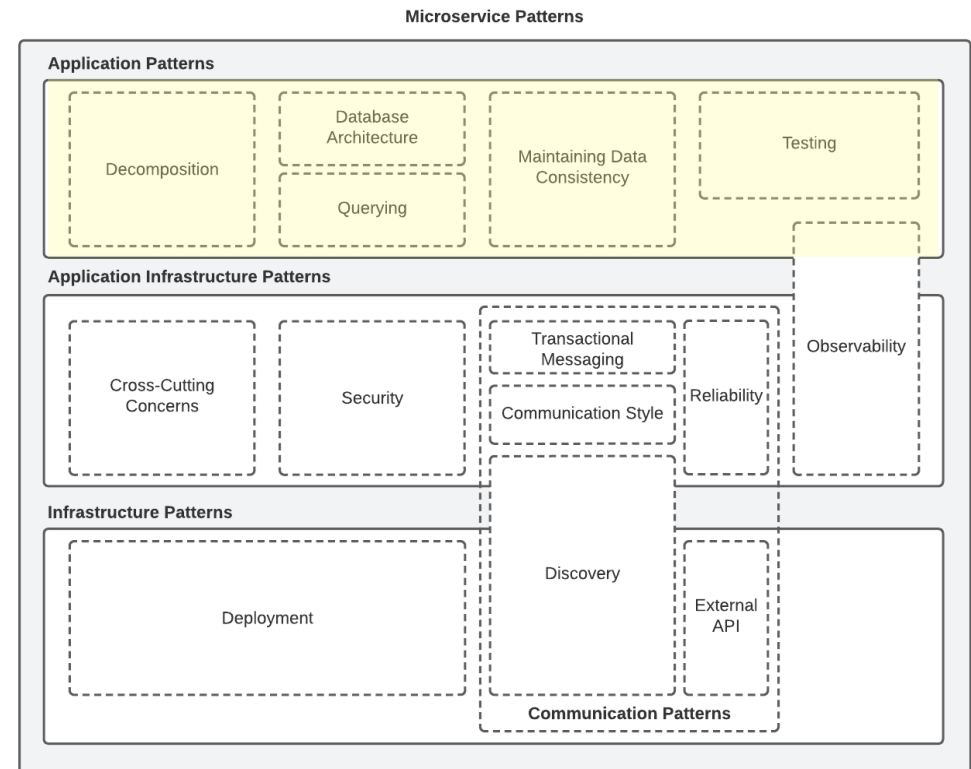
Architecture Patterns

Pattern Language

Microservice Architectures suggest a broad set of patterns as useful means for their realization.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.



Architecture Patterns

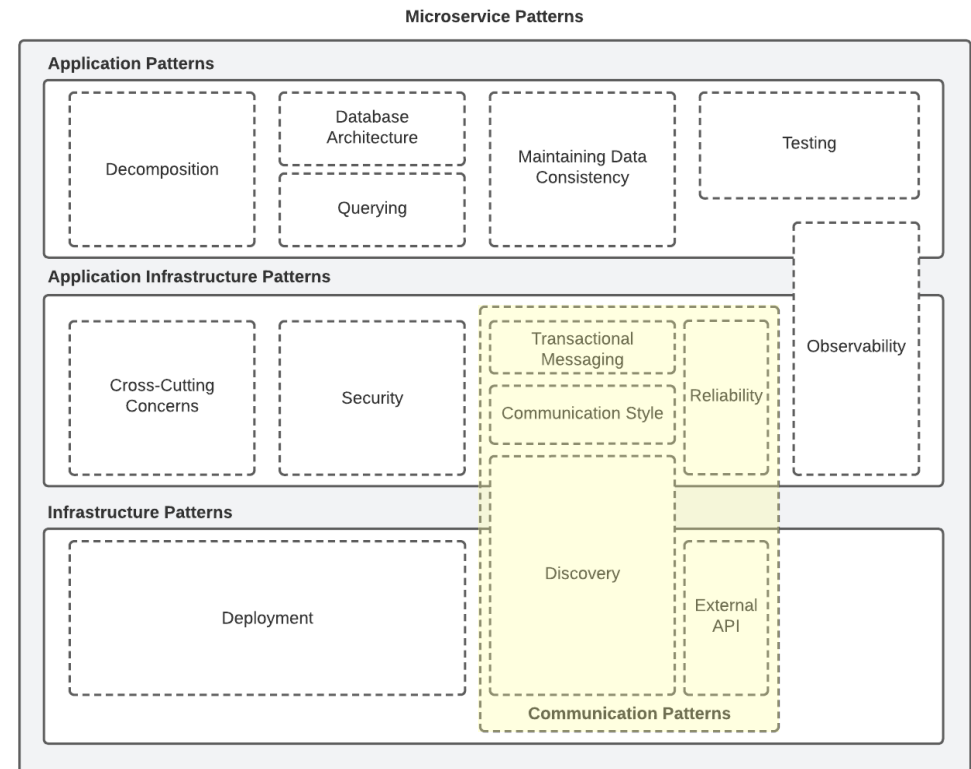
Pattern Language – Communication Patterns

Microservice Architectures suggest a broad set of patterns as useful means for their realization.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.

On the following slides, I **provide an overview** of a **selection of Communication Patterns**.

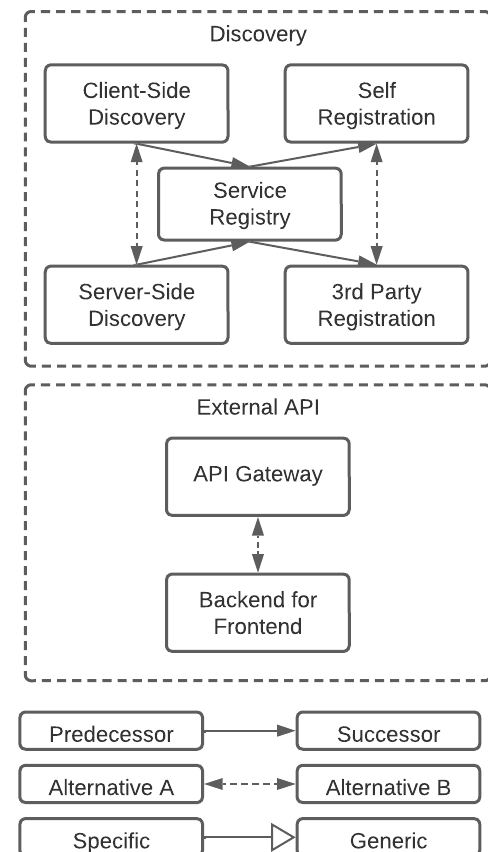


Architecture Patterns

Pattern Language – Communication Patterns

Microservice Architectures suggest a broad set of patterns as useful means for their realization – **Communication Patterns**.

- **Discovery.** How does a client of a service determine the IP address of a service instance so that, for example, it makes an HTTP request?
- **External API.** How do clients of our application communicate with the services?
- **Communication Style.** What kind of IPC mechanisms should be used (e.g., publish-subscribe, messaging, RPC).
- **Reliability.** How can we ensure that communication between services is reliable even though service can be unavailable?
- **Transactional Messaging.** How should we integrate the sending of messages and publishing of events with database transactions that update business data?



Architecture Patterns

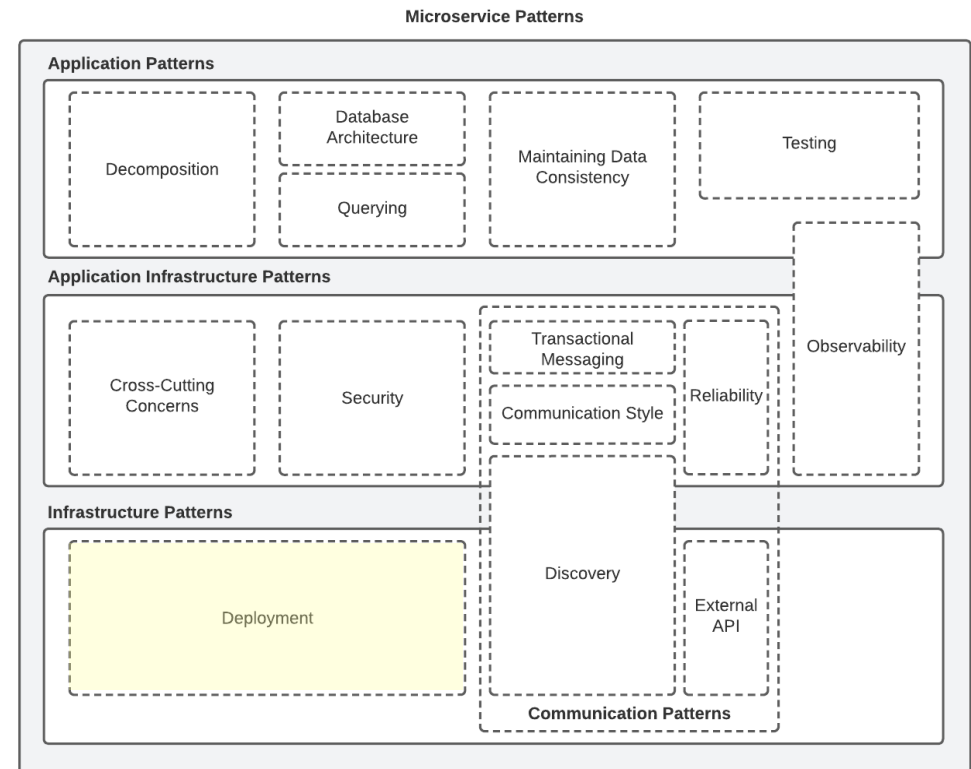
Pattern Language – Deployment Patterns

Microservice Architectures suggest a broad set of patterns as useful means for their realization – **Deployment Patterns**.

- **Infrastructure patterns** solve problems that are mostly infrastructure issues outside of software development.
- **Application infrastructure patterns** are addressing infrastructure issues that also impact software development.
- **Application patterns** cope with problems faced by software developers, directly.

A **Microservice Architecture Pattern Language** consists of a variety of **groups of patterns**.

Next, I provide an overview of a Microservice-related **Deployment Patterns**.

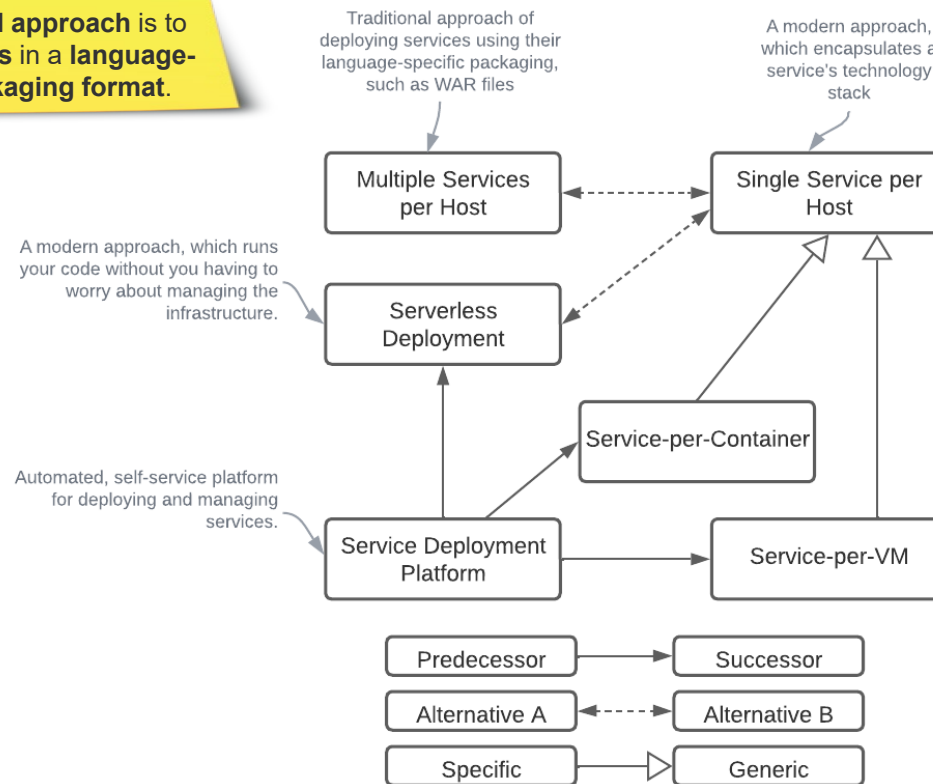


Architecture Patterns

Pattern Language – Deployment Patterns

Microservice Architectures suggest a broad set of patterns as useful means for their realization – **Deployment Patterns**.

The **traditional approach** is to **deploy services in a language-specific packaging format**.



A **broad variety of options** for **Microservice deployments** exist nowadays.

Deploying **Microservices-based applications** is much **more complex** than **deploying a monolithic application**.

More **modern approaches** deploy **Microservices** into **Virtual Machines (VM), Containers, Pods, or Serverless Technology** (e.g., AWS Lambda).

Architecture Patterns

Pattern Language – API Gateway Pattern

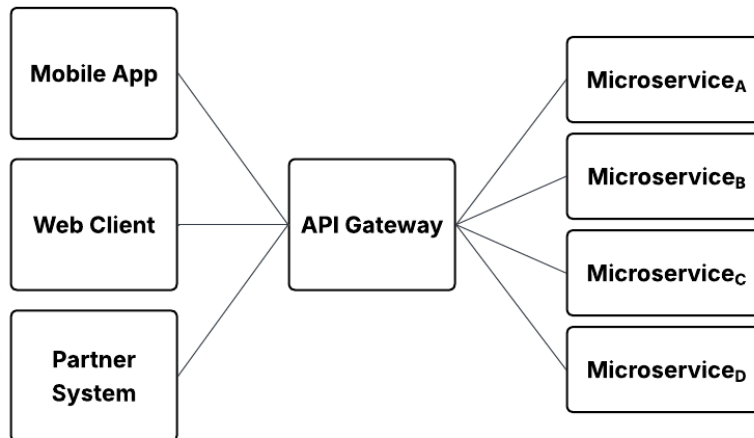
An **API Gateway** acts as a **single entry point** for **all client requests**. It **routes, transforms, aggregates, or filters requests** to **microservices** that it **façades**.

The **forces** that the **API Gateway Pattern** addresses are ...

Clients should be **decoupled** from the **internal service topology** and evolution.

Different clients (web, mobile, partner) may **require different aggregations** or **representations**.

Security concerns (authentication, authorization, throttling) **should** be **handled consistently**.



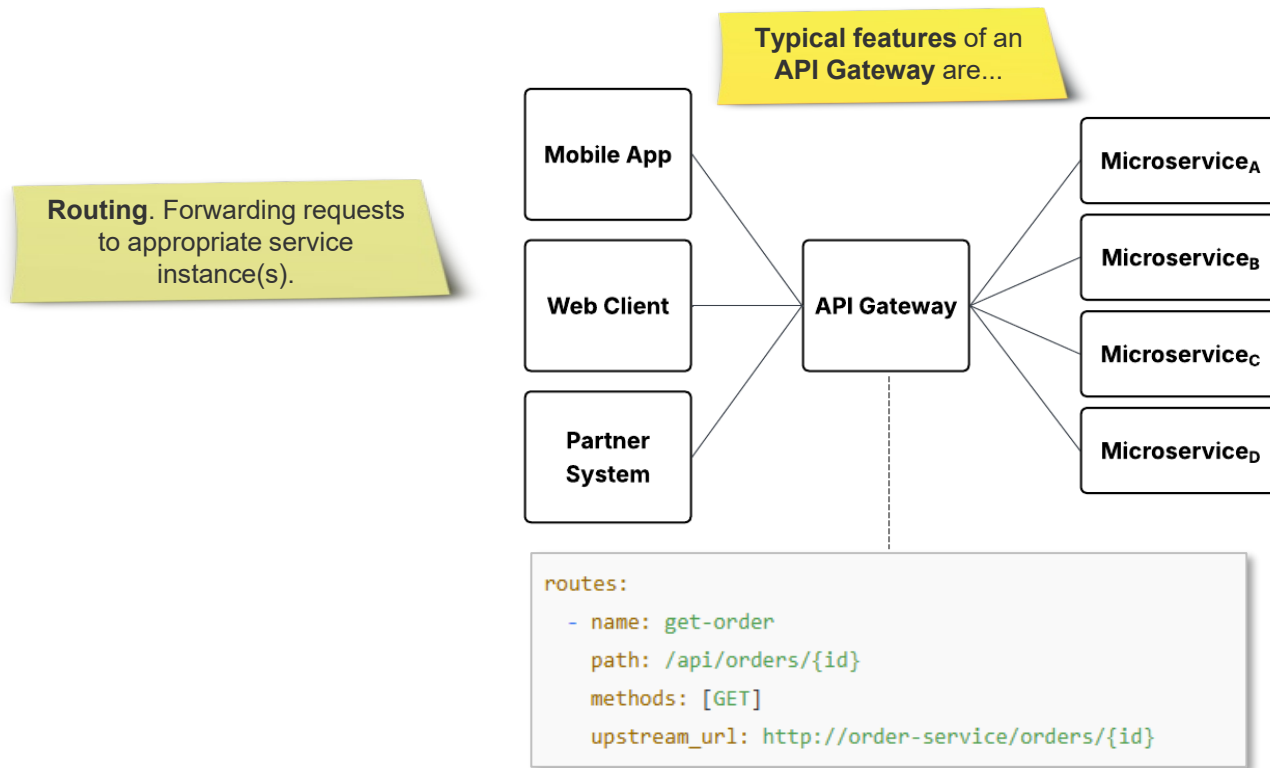
Changes to internal APIs should **not break client integrations**.

Centralized policies (logging, monitoring, metrics) **must be enforceable**.

Architecture Patterns

Pattern Language – API Gateway Pattern

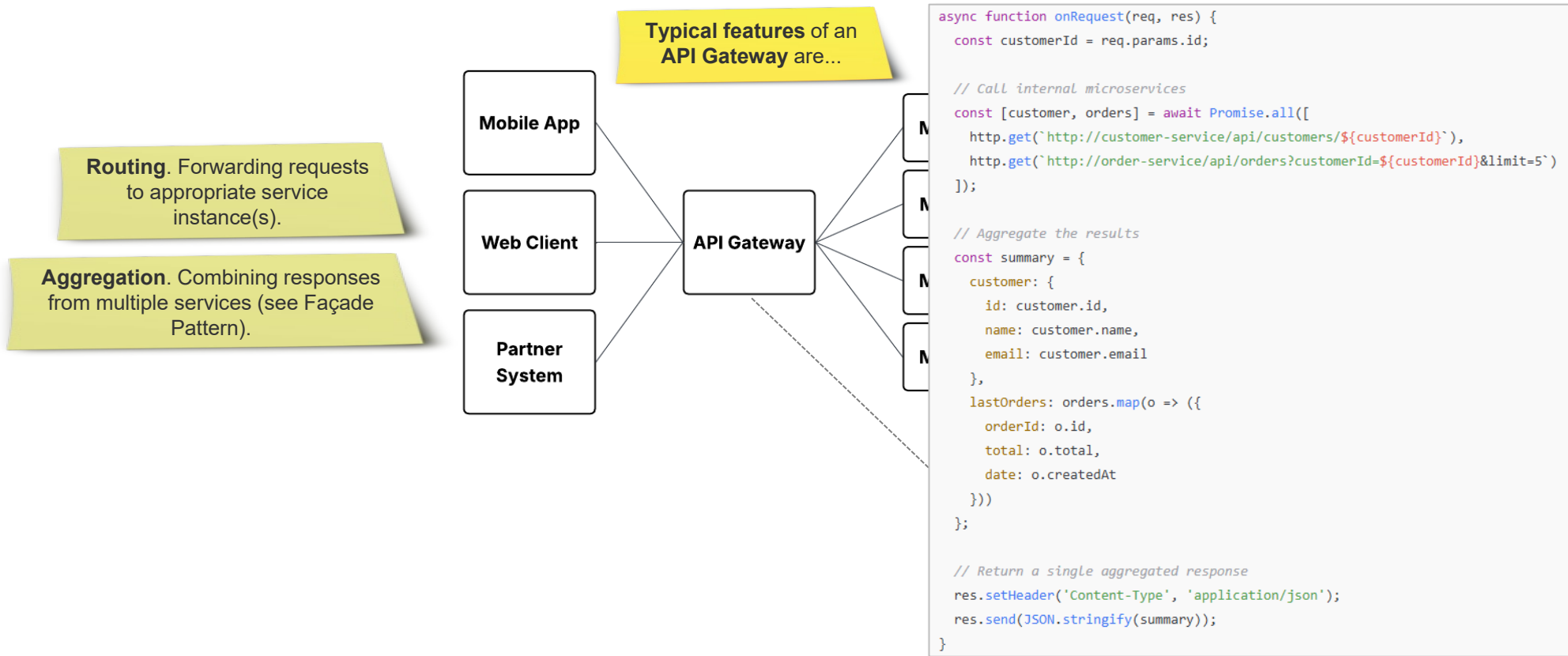
An **API Gateway** acts as a **single entry point** for **all client requests**. It **routes, transforms, aggregates, or filters requests** to **microservices** that it **façades**.



Architecture Patterns

Pattern Language – API Gateway Pattern

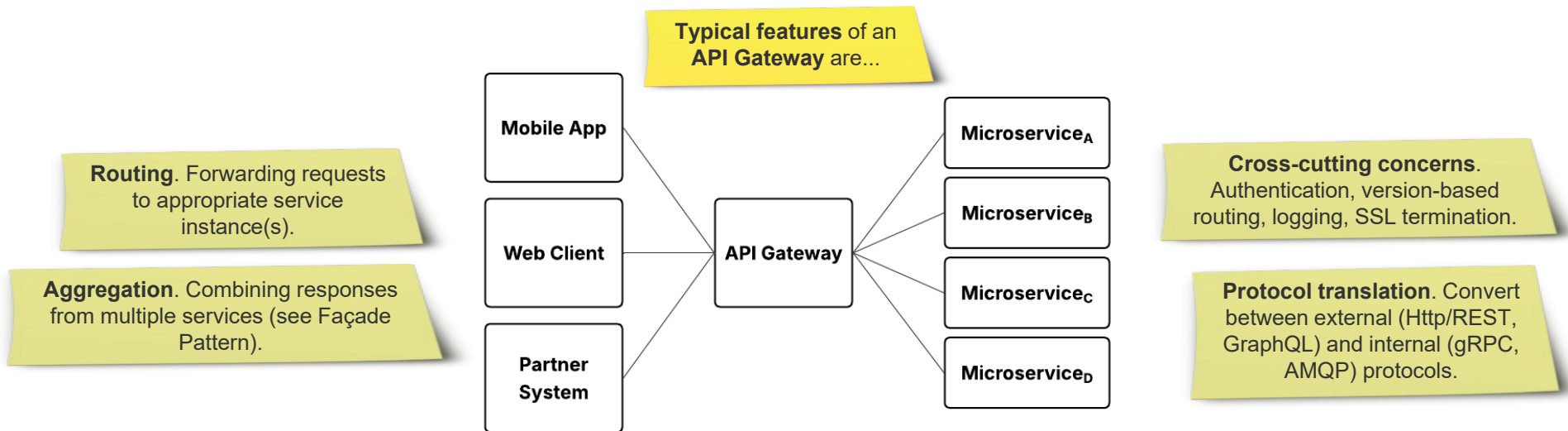
An **API Gateway** acts as a **single entry point** for **all client requests**. It **routes, transforms, aggregates, or filters requests** to **microservices** that it **façades**.



Architecture Patterns

Pattern Language – API Gateway Pattern

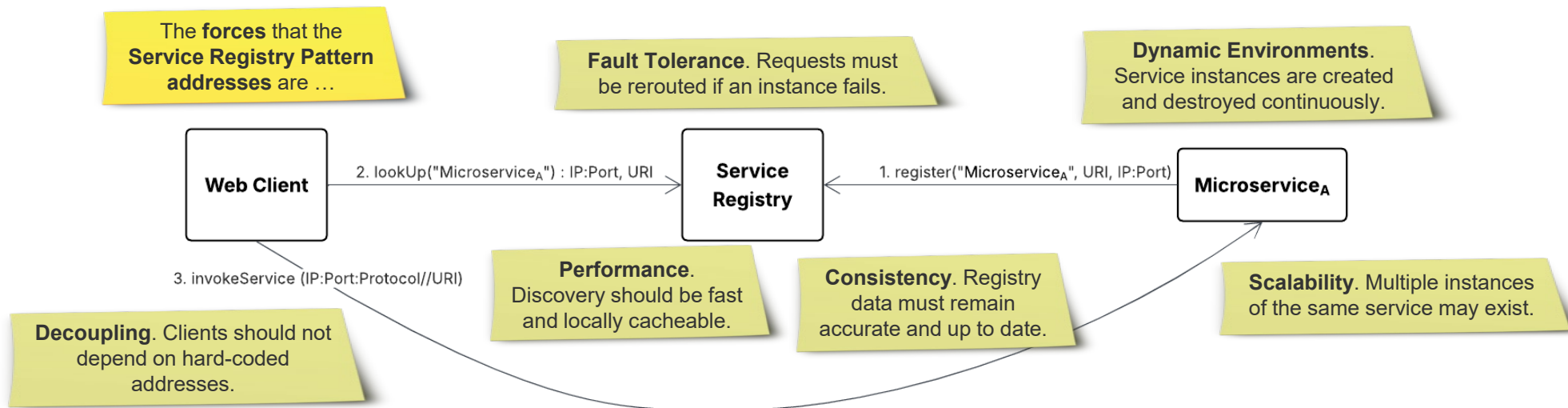
An **API Gateway** acts as a **single entry point** for **all client requests**. It **routes, transforms, aggregates, or filters requests** to **microservices** that it façades.



Architecture Patterns

Pattern Language – Service Registry Pattern

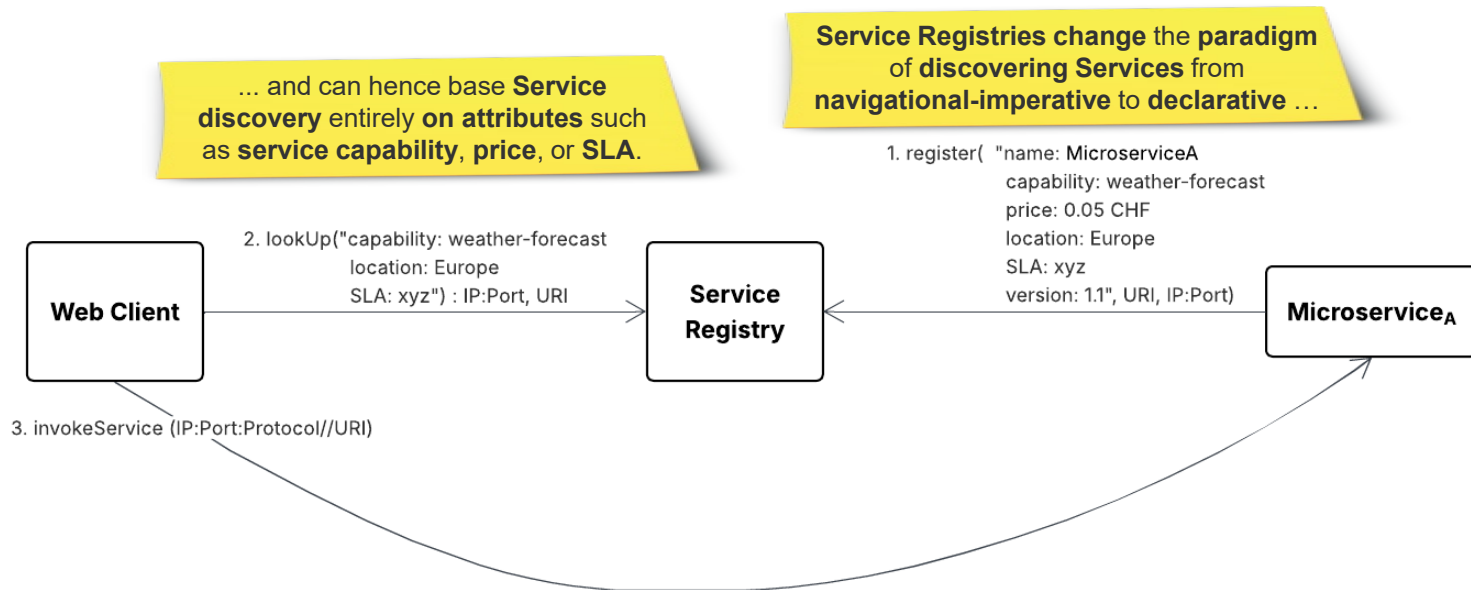
A **Service Registry** is a **central database** that maintains a **directory** of all **currently active service instances** and their **network locations**.



Architecture Patterns

Pattern Language – Service Registry Pattern

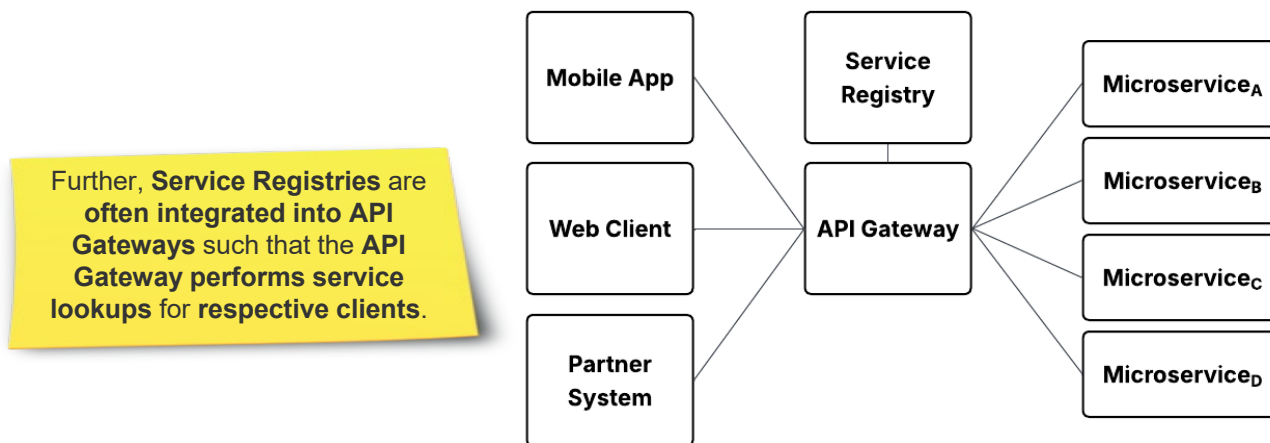
A **Service Registry** is a **central database** that maintains a **directory** of all **currently active service instances** and their **network locations**.



Architecture Patterns

Pattern Language – Service Registry Pattern

A **Service Registry** is a **central database** that maintains a **directory** of all **currently active service instances** and their **network locations**.



Architecture Patterns

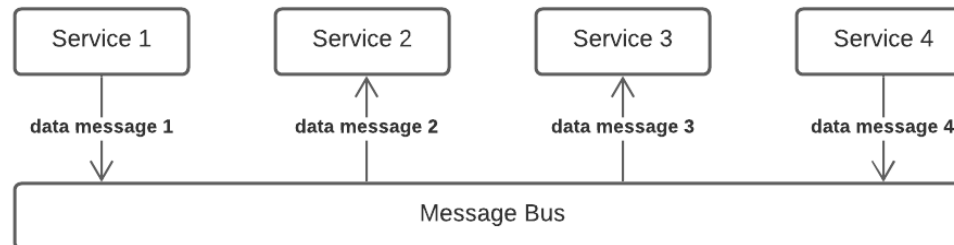
Pattern Language – Messaging Pattern

The **Messaging Pattern** replaces **synchronous service dependencies** with **asynchronous message exchange**.

The **Messaging Pattern** is a cornerstone of **scalable, resilient, and decoupled architectures** ...

... However, it requires careful design of **message semantics, delivery guarantees, and monitoring** to maintain **system integrity**.

The **Messaging Pattern** connects **services** via a **Message Bus (or Pub-Sub Topic)** that allows them to **transfer data messages asynchronously**.



Services that form the **distributed system** connect with the **message bus** to **exchange data messages** with **other services**.

Clients can initiate **collaborations** with **remote services** by **sending them data messages asynchronously**.

The **remote services** process the **received messages** and **return their responses** (if there are any), **asynchronously** to the **clients** via **messages** containing the **processing results**.

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Example of a self-describing (XML) message containing staff of a company.

```
<company>
  <staff id="1001">
    <firstname>ingo</firstname>
    <lastname>arnold</lastname>
    <nickname>ingo</nickname>
    <salary currency="CHF">0</salary>
  </staff>
  <staff id="2001">
    <firstname>jonas</firstname>
    <lastname>arnold</lastname>
    <nickname>joni</nickname>
    <salary currency="CHF">100000</salary>
  </staff>
  <staff id="2002">
    <firstname>luis</firstname>
    <lastname>arnold</lastname>
    <nickname>loisl</nickname>
    <salary currency="CHF">150000</salary>
  </staff>
</company>
```

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Two types of parsers exist for **parsing XML input**.

Both types **differ** in their **approach to parsing**, with each parser type having its strengths and weaknesses.

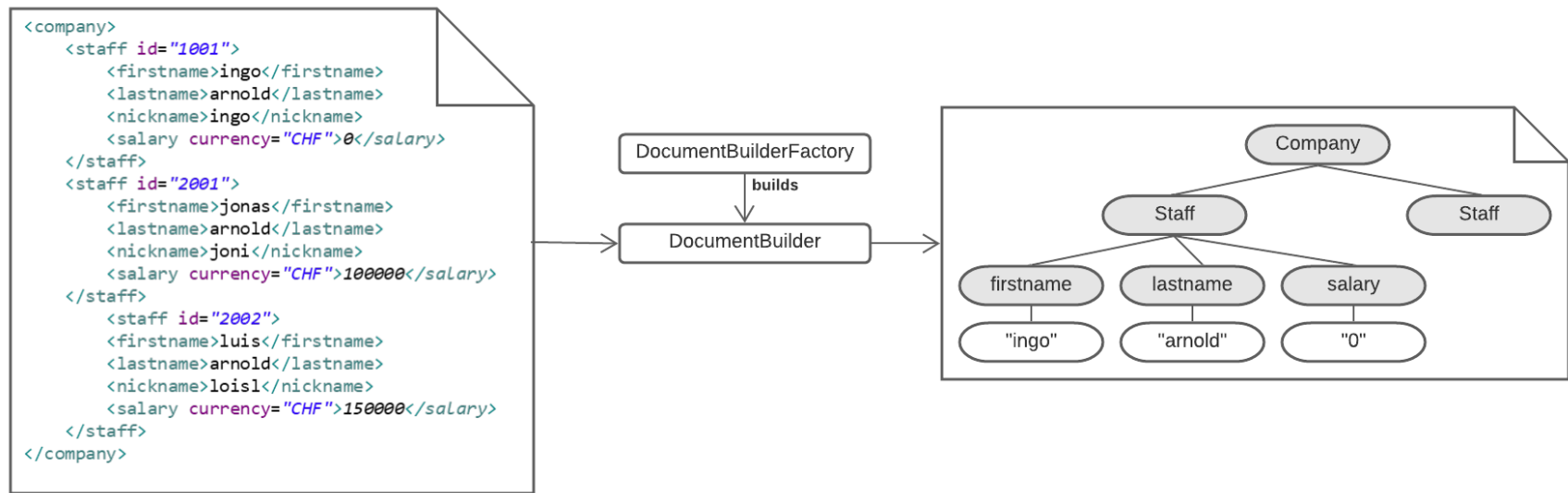
DOM (i.e., document object model) **Parsers ...**

- allow programs to **access** and **update** the **content** and **structure of XML documents dynamically**.
- **read** an **XML document into memory** and **represent** it as a **tree**; each node of the tree represents a particular piece of data from the original XML document.
- because the **tree structure** is a **standard programming** mechanism for **representing data**, **traversing** and **manipulating** the tree using Java is **relatively easy, fast, and efficient**.
- main **drawback** is that the **entire XML document** has to be **read into memory** to create the tree. This might **decrease** the **performance** of an application **as the XML documents get larger**.

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

DOM Parser example in Java.



Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a DOM¹ Parser to **parse** and **read** the **XML** file.

```
public class XmlDomParser {
    private static final String FILENAME = "./src/com/fhnw/patterns/xmlparser/company.xml";

    public static void main(String[] args) {
        // Instantiate the Factory
        DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

        try {
            DocumentBuilder db = dbf.newDocumentBuilder();

            Document doc = db.parse(new File(FILENAME));

            System.out.println("Root Element : " + doc.getDocumentElement().getNodeName());

            // get <staff>
            NodeList list = doc.getElementsByTagName("staff");

            for (int i = 0; i < list.getLength(); i++) {
                Node node = list.item(i);

                if (node.getNodeType() == Node.ELEMENT_NODE) {
                    Element element = (Element) node;

                    // get staff's attribute
                    String id = element.getAttribute("id");
                }
            }
        }
    }
}
```

¹) Document Object Model

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a DOM¹ Parser to **parse** and **read** the **XML** file.

```
// get text
String firstname = element.getElementsByTagName("firstname").item(0).getTextContent();
String lastname = element.getElementsByTagName("lastname").item(0).getTextContent();
String nickname = element.getElementsByTagName("nickname").item(0).getTextContent();

NodeList salaryNodeList = element.getElementsByTagName("salary");
String salary = salaryNodeList.item(0).getTextContent();

// get salary's attribute
String currency =
salaryNodeList.item(0).getAttributes().getNamedItem("currency").getTextContent();

System.out.println("Current Element : " + node.getNodeName());
System.out.println("Staff Id : " + id);
System.out.println("First Name : " + firstname);
System.out.println("Last Name : " + lastname);
System.out.println("Nick Name : " + nickname);
System.out.printf("Salary [Currency] : %,2f [%s]%n%n", Float.parseFloat(salary),
currency);
    }
}
} catch (ParserConfigurationException | SAXException | IOException e) {
    e.printStackTrace();
}
}
```

¹) Document Object Model

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a **DOM¹ Parser** to **parse** and **read** the **XML** file.

```
Root Element :company
Current Element :staff
Staff Id : 1001
First Name : ingo
Last Name : arnold
Nick Name : ingo
Salary [Currency] : 0.00 [CHF]

Current Element :staff
Staff Id : 2001
First Name : jonas
Last Name : arnold
Nick Name : joni
Salary [Currency] : 100'000.00 [CHF]

Current Element :staff
Staff Id : 2002
First Name : luis
Last Name : arnold
Nick Name : lois1
Salary [Currency] : 150'000.00 [CHF]
```

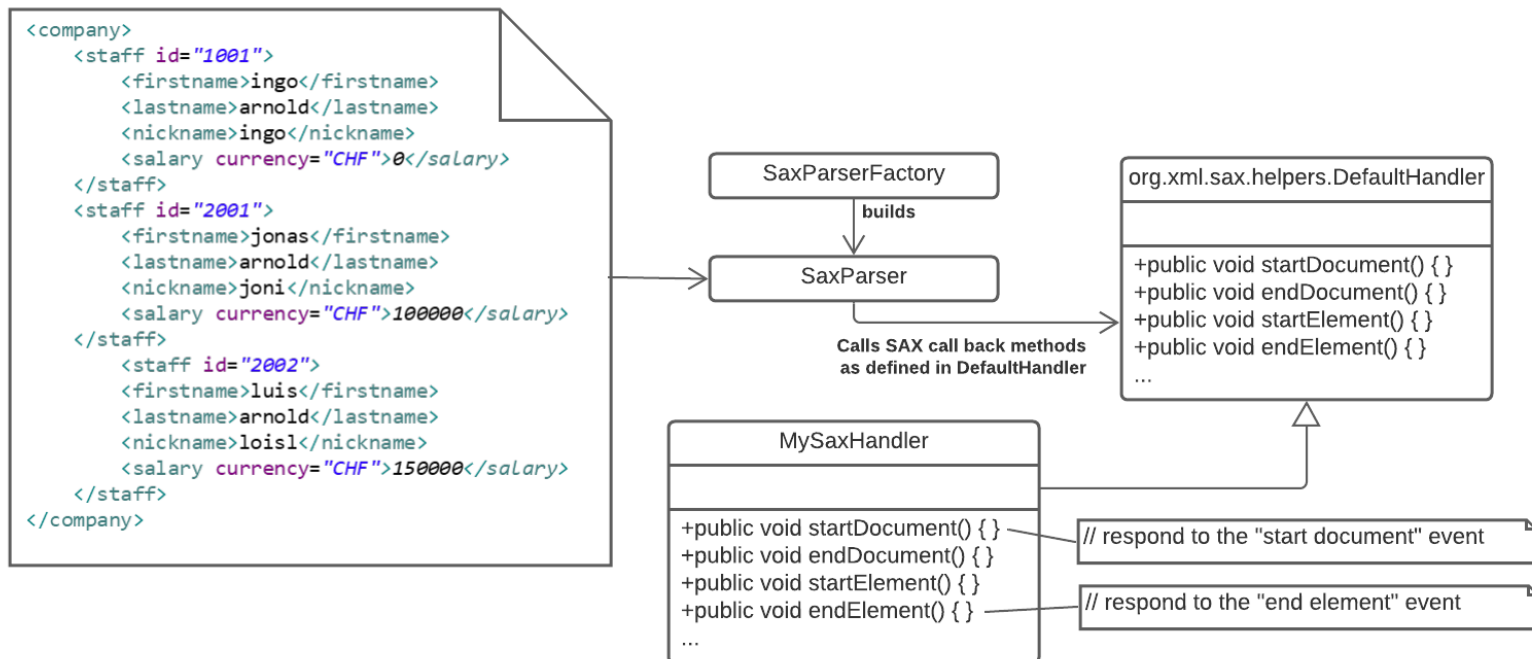
¹⁾ Document Object Model

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

SAX (i.e., simple API for XML) Parsers ...

- provide a **platform-independent** and neutral **standard interface** for **event-based XML parsing**.
- **SAX defines events** that **can occur** as a **parser** is **reading through an XML document**, such as the start or the end of an element.
- **Programmers provide** handlers to **deal** with **different events** as the **document is parsed**.



Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a **SAX¹** Parser to **parse** and **read** the **XML** file.

```
public class MyHandlerSax extends DefaultHandler {
    private StringBuilder currentValue = new StringBuilder();
    @Override
    public void startDocument() {
        System.out.println("Start Document");
    }
    @Override
    public void endDocument() {
        System.out.println("End Document");
    }
    @Override
    public void startElement(String uri, String localName, String qName, Attributes attributes) {
        // reset the tag value
        currentValue.setLength(0);
        System.out.printf("Start Element : %s%n", qName);

        if (qName.equalsIgnoreCase("staff")) {
            // get tag's attribute by name
            String id = attributes.getValue("id");
            System.out.printf("Staff id : %s%n", id);
        }
        if (qName.equalsIgnoreCase("salary")) {
            // get tag's attribute by index, 0 = first attribute
            String currency = attributes.getValue(0);
            System.out.printf("Currency :%s%n", currency);
        }
    }
}
```

¹) Simple API for XML

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a **SAX**¹ Parser to **parse** and **read** the **XML** file.

```
@Override
public void endElement(String uri, String localName, String qName) {

    System.out.printf("End Element : %s%n", qName);

    if (qName.equalsIgnoreCase("name")) {
        System.out.printf("Name : %s%n", currentValue.toString());
    }

    if (qName.equalsIgnoreCase("role")) {
        System.out.printf("Role : %s%n", currentValue.toString());
    }

    if (qName.equalsIgnoreCase("salary")) {
        System.out.printf("Salary : %s%n", currentValue.toString());
    }

    if (qName.equalsIgnoreCase("bio")) {
        System.out.printf("Bio : %s%n", currentValue.toString());
    }

}
}
```

¹) Simple API for XML

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a **SAX¹** Parser to **parse** and **read** the **XML** file.

```
public class XmlSaxParser {  
    private static final String FILENAME = "./src/com/fhnw/patterns/xmlparser/company.xml";  
  
    public static void main(String[] args) {  
        SAXParserFactory factory = SAXParserFactory.newInstance();  
  
        try {  
            SAXParser saxParser = factory.newSAXParser();  
  
            MyHandlerSax handler = new MyHandlerSax();  
            saxParser.parse(FILENAME, handler);  
  
        } catch (ParserConfigurationException | SAXException | IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

¹⁾ Simple API for XML

Architecture Patterns

Pattern Language – Messaging Pattern (XML Excursus)

Using a **SAX¹** Parser to **parse** and **read** the **XML** file.

```
Start Document
Start Element : company
Start Element : staff
Staff id : 1001
Start Element : firstname
End Element : firstname
Start Element : lastname
End Element : lastname
Start Element : nickname
End Element : nickname
Start Element : salary
Currency :CHF
End Element : salary
Salary :
End Element : staff
Start Element : staff
Staff id : 2001
Start Element : firstname
End Element : firstname
Start Element : lastname
End Element : lastname
Start Element : nickname
End Element : nickname
Start Element : salary
Currency :CHF
End Element : salary
Salary : 100000
End Element : staff
```

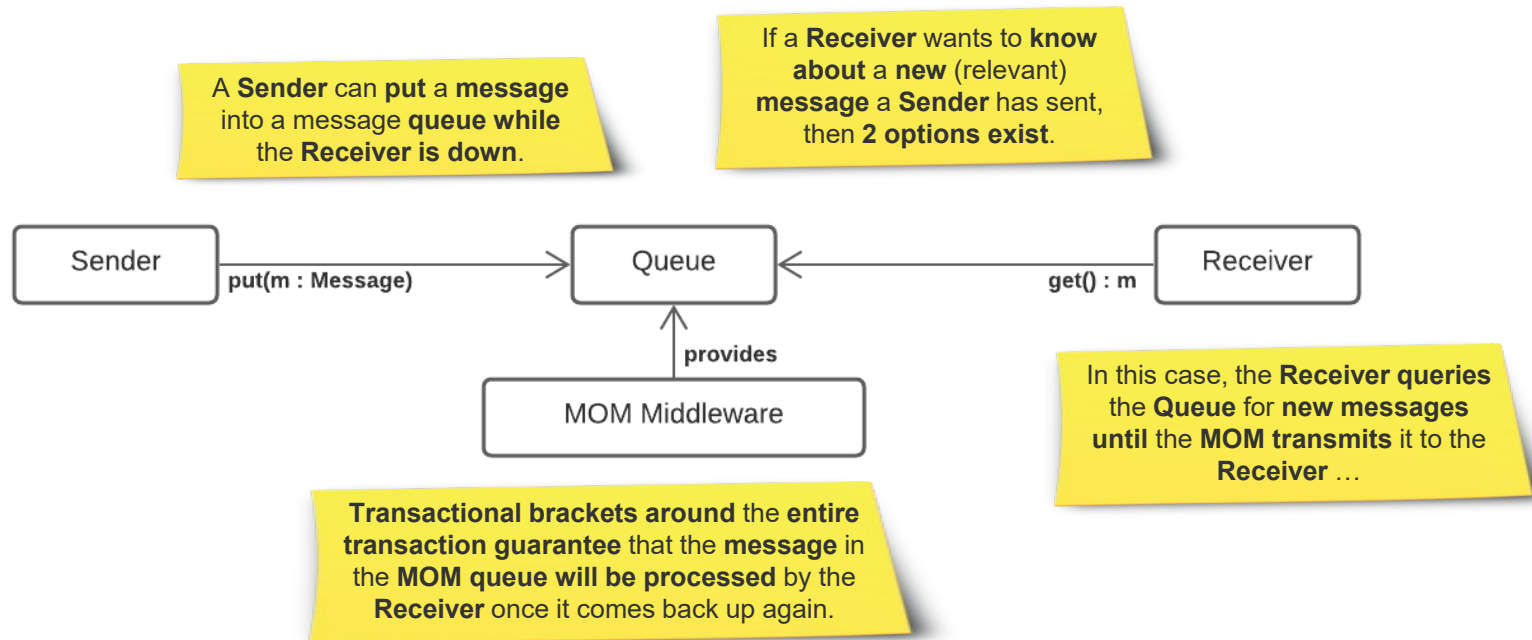
```
Start Element : staff
Staff id : 2002
Start Element : firstname
End Element : firstname
Start Element : lastname
End Element : lastname
Start Element : nickname
End Element : nickname
Start Element : salary
Currency :CHF
End Element : salary
Salary : 150000
End Element : staff
End Element : company
End Document
```

¹⁾ Simple API for XML

Architecture Patterns

Pattern Language – Messaging Pattern

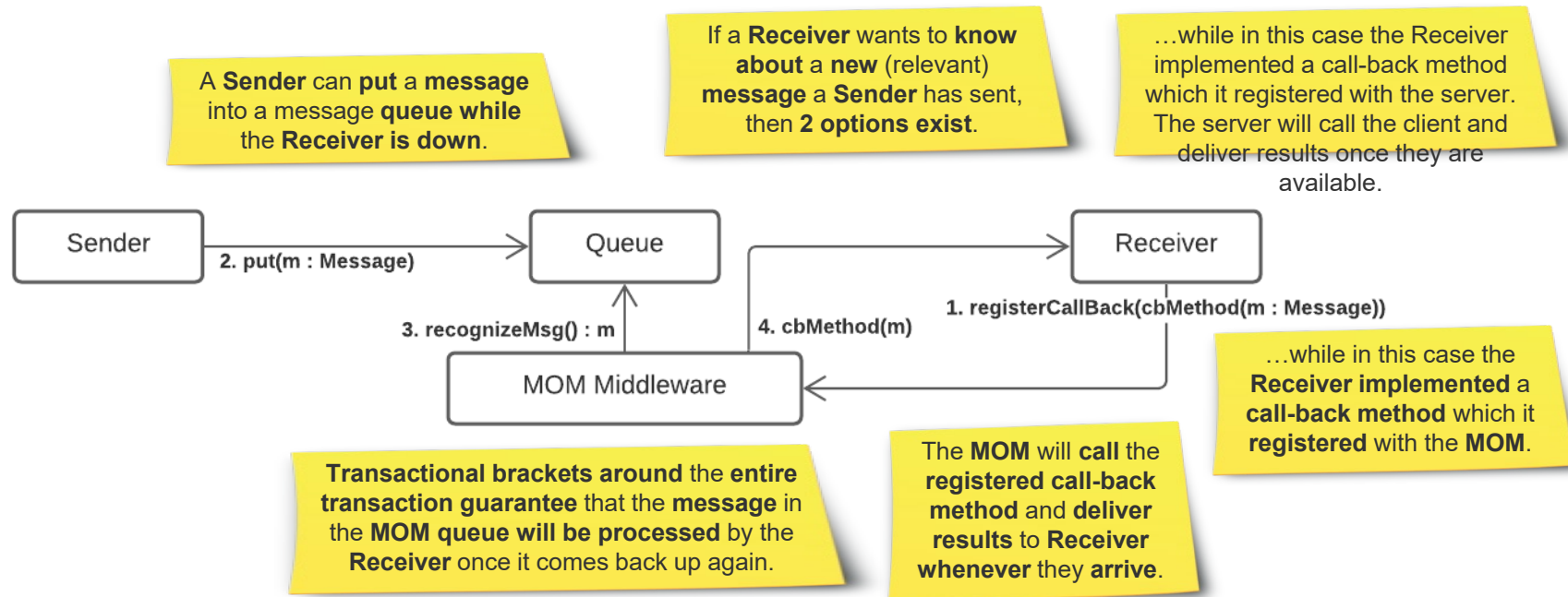
Message Oriented Middleware is abbreviated **MOM**. A **MOM** decouples clients and **servers**. If the **MOM** supports **ACID-transactions**, clients and **servers** do not have to be **available** at the **same time**.



Architecture Patterns

Pattern Language – Messaging Pattern

Message Oriented Middleware is abbreviated **MOM**. A **MOM decouples clients** and **servers**. If the **MOM supports ACID-transactions**, **clients** and **servers** do **not** have to be **available at the same time**.



Architecture Patterns

Pattern Language – Messaging Pattern

A **variety** of **Message Delivery characteristics** can be **distinguished** and usually **delegated** to the **MOM**:

- **Delivery cardinality** (e.g., a message is delivered once and only once).
- **Delivery guarantees** (e.g., delivery is guaranteed even if a participating system is down).
- **Message priority**.
- **Message correlation** (i.e., correlation identifiers to re-correlate coarse messages).
- **Message synchronicity** (e.g., asynchronous versus synchronous messaging).
- **Error handling** (e.g., dedicated error queues, monitoring agents).
- **Messaging model** (e.g., point-2-point versus publish-subscribe model).

Architecture Patterns

Pattern Language – Messaging Pattern

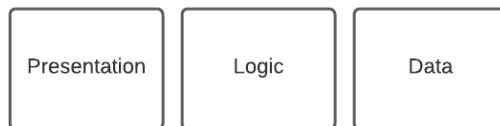
Contrasting the remoting models Messaging and Broker (RPC-Style).

Feature	Messaging-Style	Broker (RPC-Style)
Metaphor	Post office-like	Telephone-like
Client / Server synchronization	Asynchronous – clients and servers may operate at different times and speeds	Synchronous – clients and servers must run parallelly. Servers must keep up with request loads
Client / Server sequencing	No fixed sequence	Servers must be available for service before clients can talk to them
Style	Queued	Call return
Partner needs to be available	No	Yes
Load balancing	Single queue can be used to implement FIFO or priority-based policy	Requires a separate TP-Monitor
Support for transactions	Yes (some products). Message queue can participate in the commit synchronization	No (not common). Requires transactional RPC
Message filtering	Yes	No
Performance	Slow. An intermediate hop is required	Fast
Asynchronous processing	Yes. Queues and triggers are required	Limited. Requires threads and tricky code for managing threads

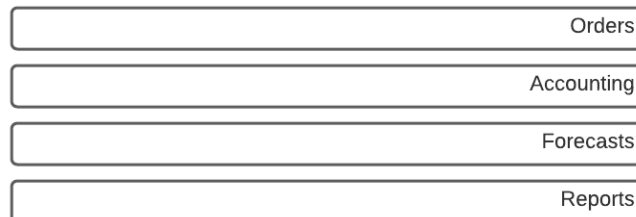
Architecture Patterns

Pattern Language – Layered System Pattern

Unlike the classic horizontal layering of systems (i.e., layers pattern), ...



... in the **context of microservices tasks** are divided into domain objects ...



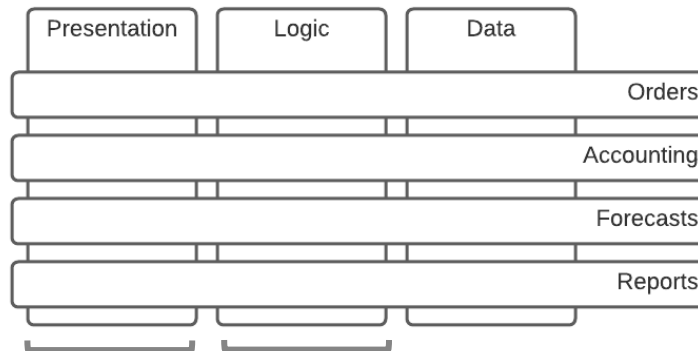
Each domain object (i.e., microservice) has its **business logic**, **data storage logic** and adapters and, if necessary, **presentation logic** and adapters.



Architecture Patterns

Pattern Language – Layered System Pattern

This results in a matrix-like organization of solutions adapting the microservices pattern.



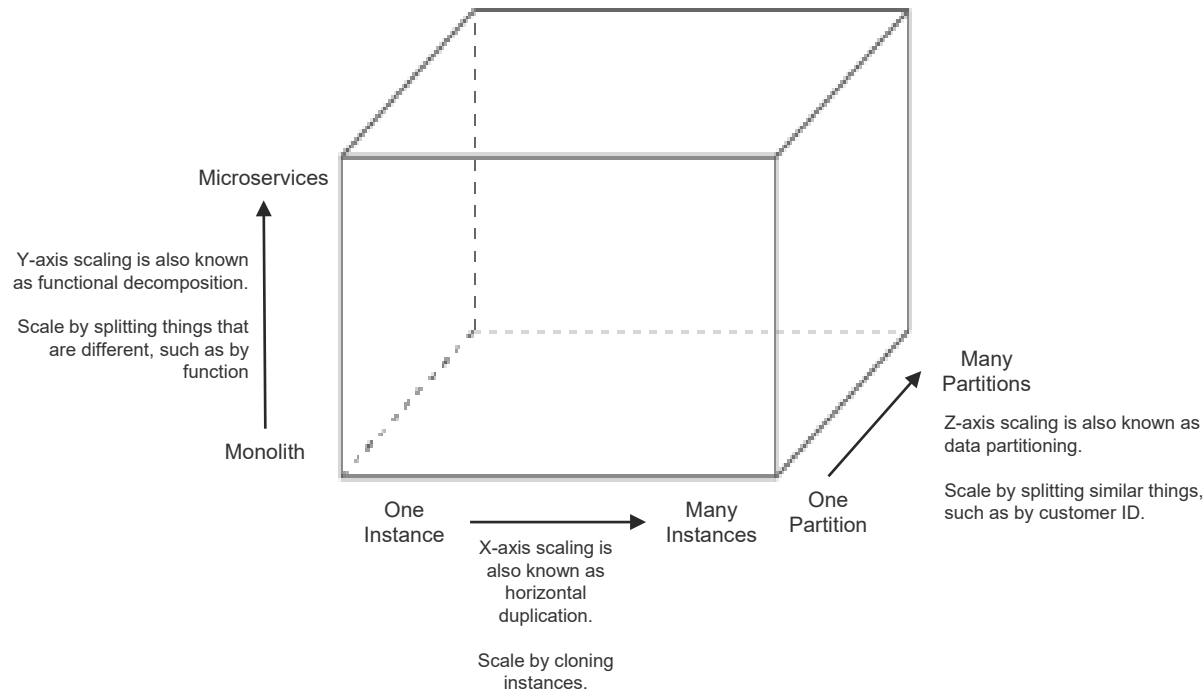
If presentation aspects of microservices are well decoupled from their business logic (e.g., via standardized data inputs (DTO)), presentation components can be used flexibly and combined across different microservices.

Business Logic building blocks of a microservice should be stateless if possible to enable easy on/off and easy replication.

Architecture Patterns

Pattern Language – Scalability Patterns

Microservices offer scalability that goes **beyond** what **monolithic applications** typically **offer**. This is made **visible** via the **Scalability Cube**.



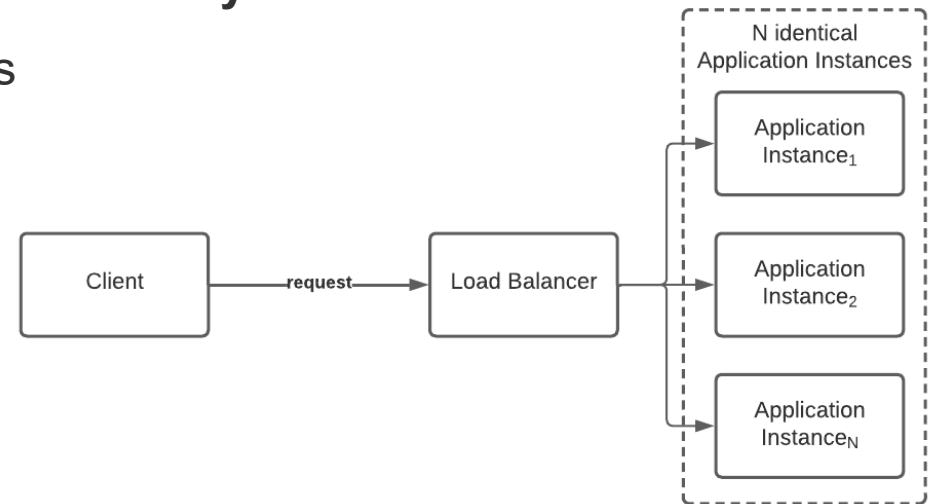
Architecture Patterns

Pattern Language – Scalability Patterns

Microservices offer scalability that goes **beyond** what **monolithic applications** typically **offer**. This is made **visible** via the **Scalability Cube**.

X-Axis scaling load balances requests across multiple instances

- **Multiple instances** of an **application** are run **behind** a **load balancer**.
- The **load balancer distributes requests** among the **N identical instances** of the application.
- This is a **great way** of **improving** the **capacity** and **availability** of an application.



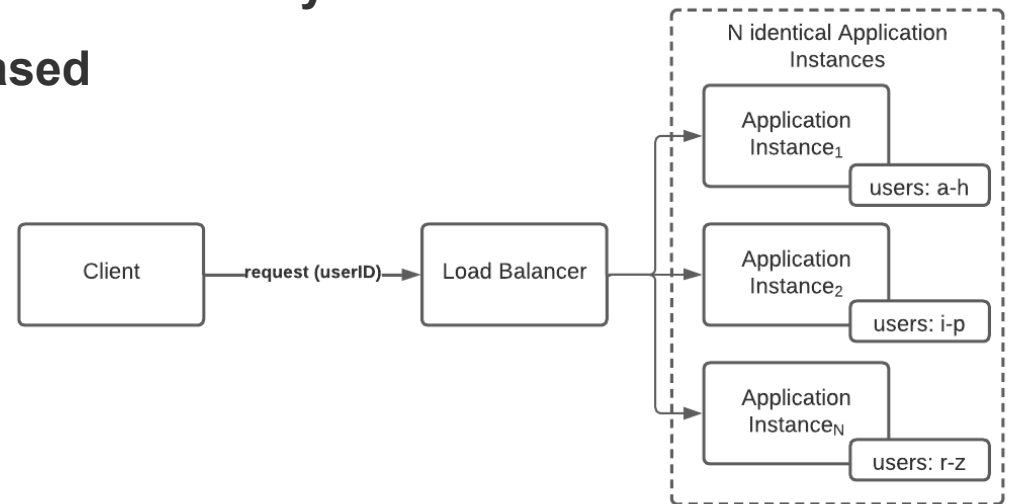
Architecture Patterns

Pattern Language – Scalability Patterns

Microservices offer scalability that goes **beyond** what **monolithic applications** typically **offer**. This is made **visible** via the **Scalability Cube**.

Z-Axis scaling routes requests based on attribute(s) of a request

- **Z-axis scaling also runs multiple instances** of an application **behind a load balancer**.
- **Unlike X-axis scaling, each instance is responsible for only a subset of the data.**
- The **load balancer** in front of the instances **uses a request attribute to route** it to the **appropriate instance**.
- An **application might**, for example, **route requests using an userID attribute**.
- **Z-axis scaling** is a **great way** to scale an application to handle **increasing transaction** and **data volumes**.



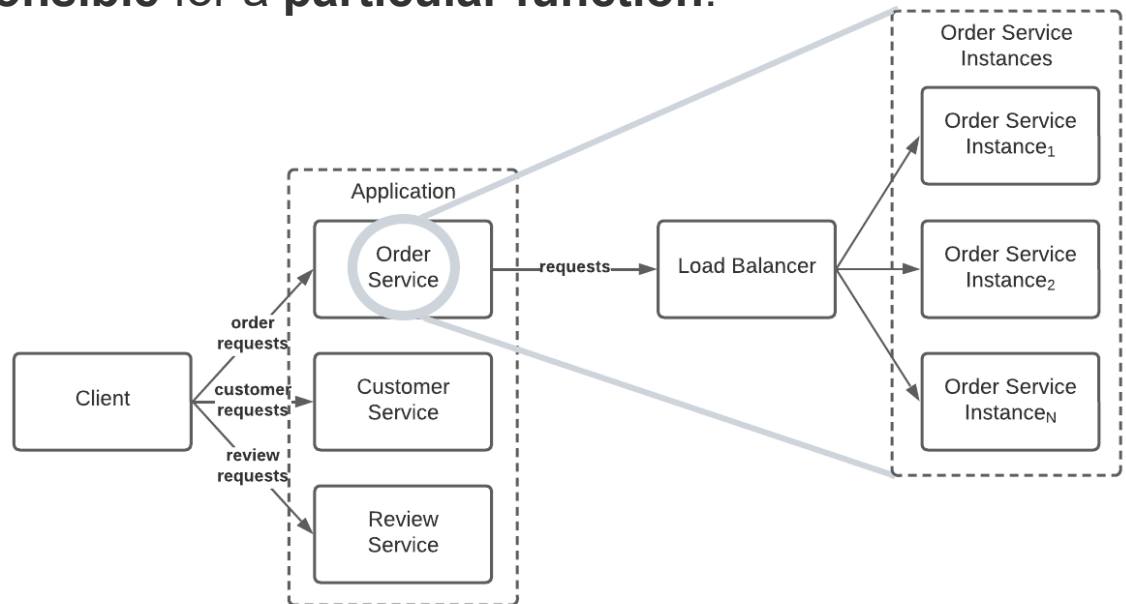
Architecture Patterns

Pattern Language – Scalability Patterns

Microservices offer scalability that goes **beyond** what **monolithic applications** typically **offer**. This is made **visible** via the **Scalability Cube**.

Y-Axis scaling splits an **application** into a **set of services**, with **each service** being **responsible** for a **particular function**.

- **X-axis** and **Z-axis** scaling **improve** the application's **capacity** and **availability**.
- However, **neither approach solves** the **problem** of **increasing development** and **application complexity**.
- To **solve those**, one needs to **apply Y-axis scaling**, or **functional decomposition**.
- **Y-axis scaling** is applied by **splitting a monolithic application** into a **set of services**

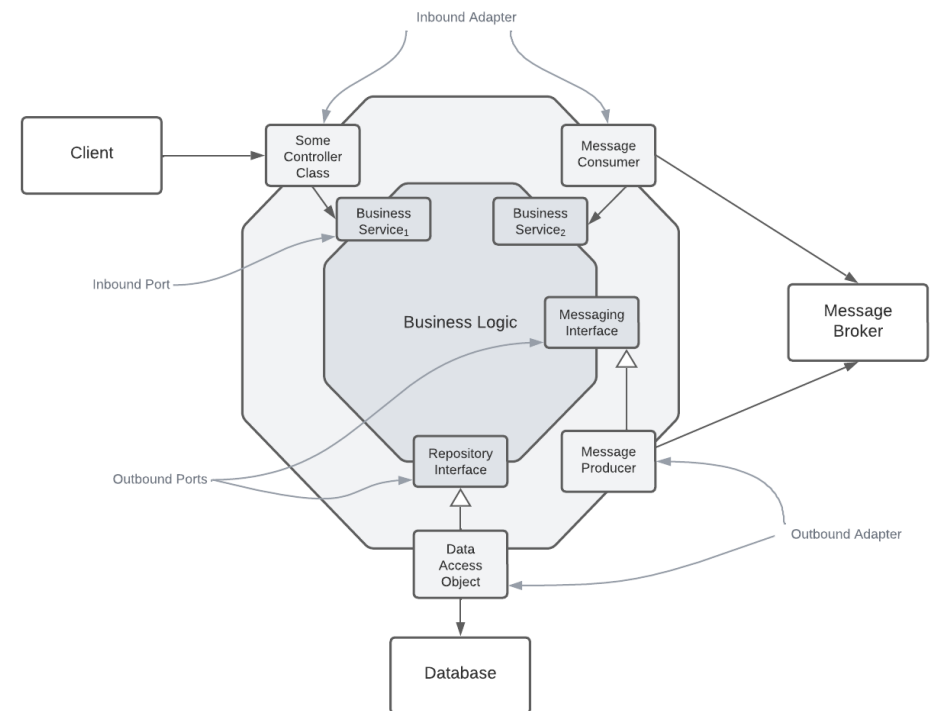
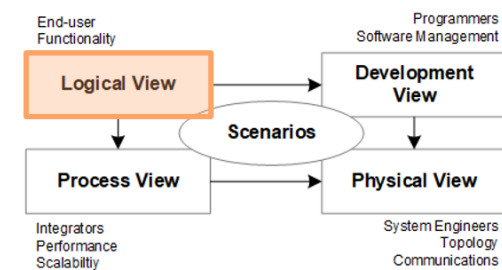


Architecture Patterns

Hexagonal Architecture

The **Hexagonal Architecture style** organizes the **logical view** in a way that places the **business logic** at the center.

- Instead of the presentation layer the application has one or more **inbound adapters** that handle requests from the outside by invoking the **business logic**.
- Similarly, **instead of a data persistence tier**, the application has one or more **outbound adapters** that are invoked by the business logic and invoke external applications.
- A **key characteristic** and benefit of this architecture is that the **business logic does not depend on the adapters**.
- **Instead the adapters depend on the business logic**.
- The **business logic has** one or more **ports**. A **port defines** a set of **operations** and may be a Java interface.

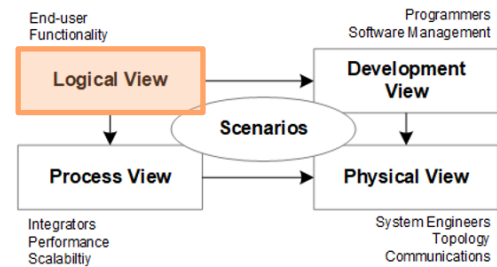
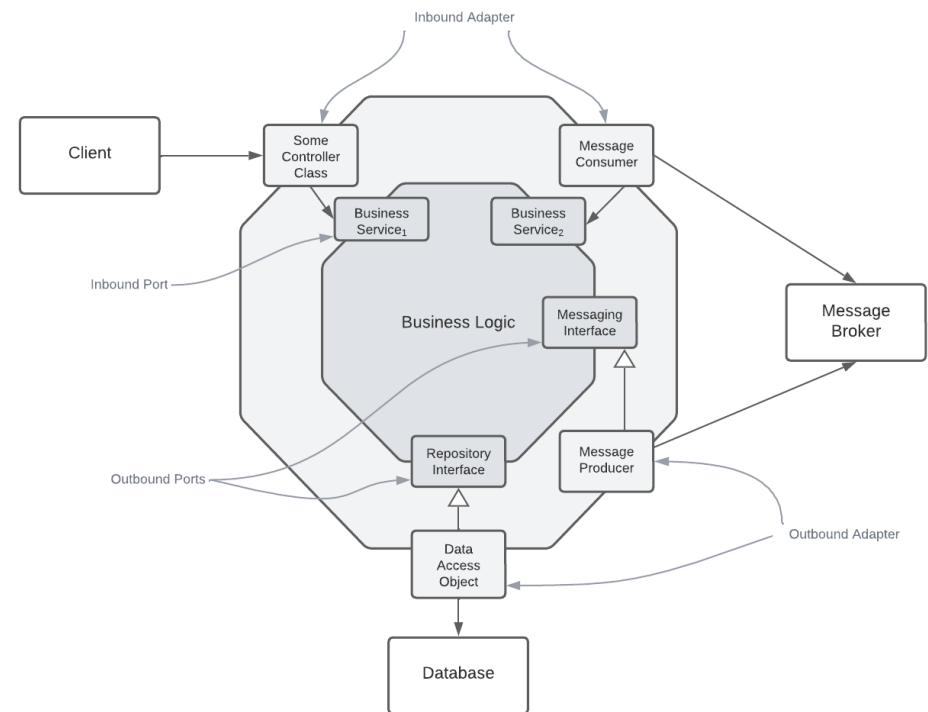


Architecture Patterns

Hexagonal Architecture

The **Hexagonal Architecture style** organizes the **logical view** in a way that **places the business logic at the center**.

- An **inbound port** is an **API exposed** by the business logic, which enables it to be **invoked** by **external applications**. An example of an inbound port is a service interface, which defines a service's public methods.
- An **outbound port** is **how the business logic invokes external systems**. An example of an outbound port is a repository interface, which defines a collection of data access operations.
- **Surrounding the business logic** are **adapters**. As with ports, there are two types of them: **inbound and outbound adapters**.

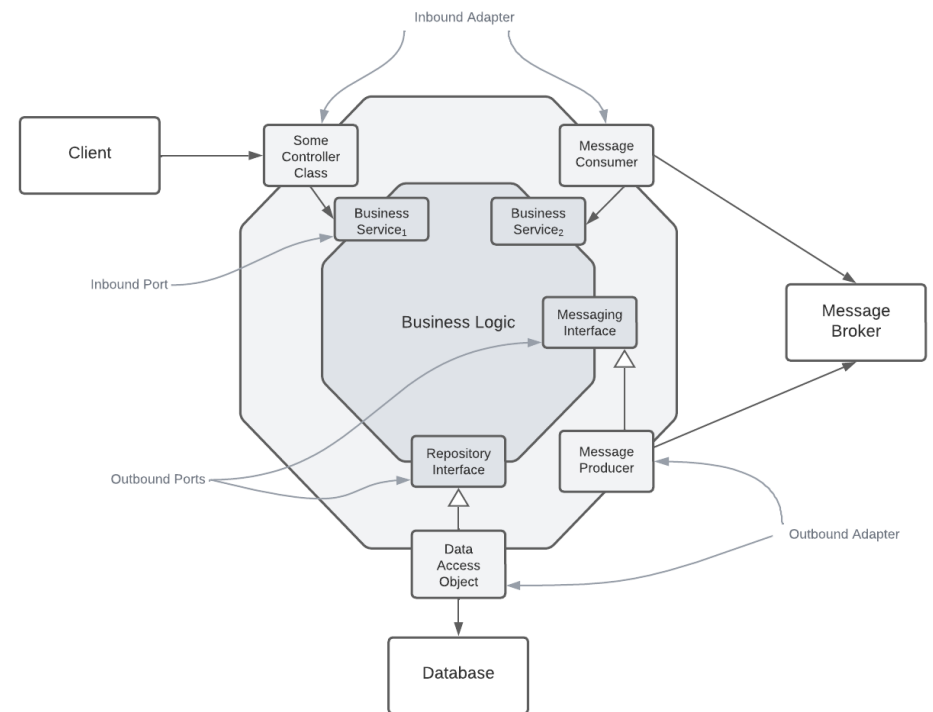
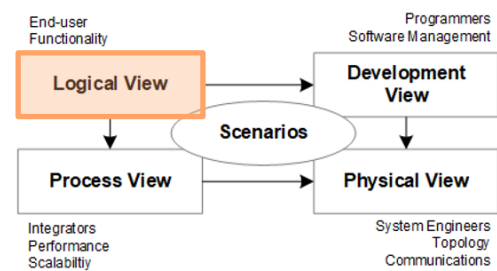


Architecture Patterns

Hexagonal Architecture

The **Hexagonal Architecture style** organizes the **logical view** in a way that **places the business logic at the center**.

- An **inbound adapter** handles requests **from the outside world** by **invoking an inbound port**. An example of an inbound adapter is a Spring MVC Controller that implements either a set of REST endpoints or a set of web pages. Another example is a message broker client that subscribes to messages. Multiple inbound adapters can invoke the same inbound port.
- An **outbound adapter** implements an **outbound port** (i.e., sub-typing) and **handles requests** from the **business logic** by **invoking an external application or service**. An example of an outbound adapter is a data access object (DAO) class that implements operations for accessing a database. Another example would be a proxy class that invokes a remote service.

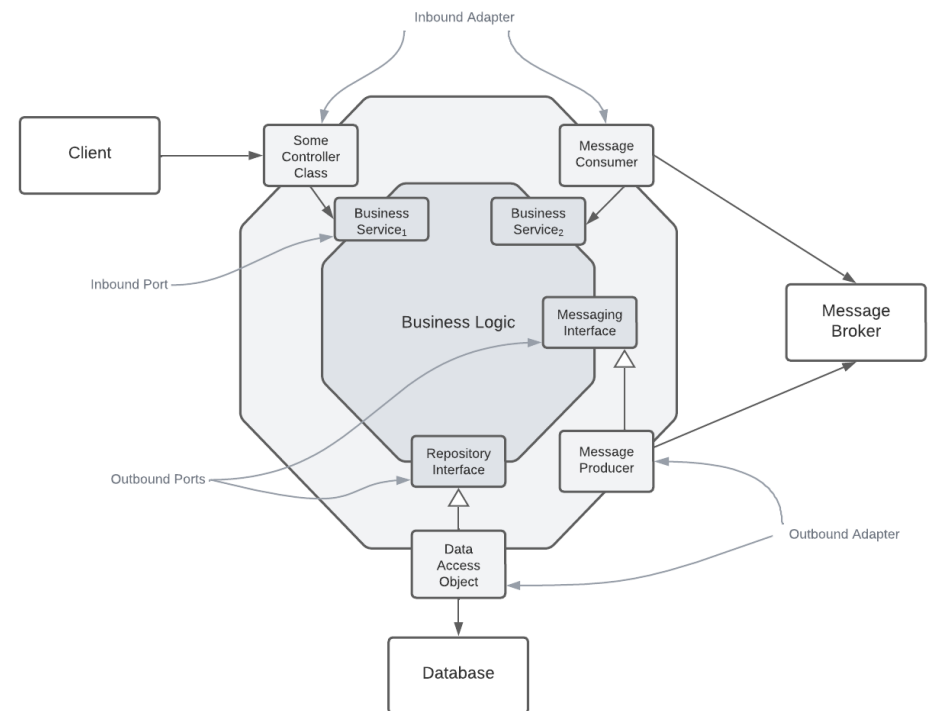
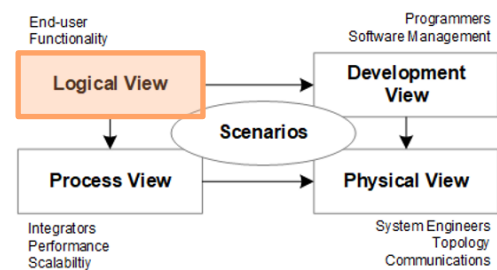


Architecture Patterns

Hexagonal Architecture

The **Hexagonal Architecture style** organizes the **logical view** in a way that **places the business logic at the center**.

- An **important benefit** of the **hexagonal architecture** style is that it **decouples the business logic from the presentation and data access logic** in adapters.
- The **business logic** does not depend on **either the presentation logic or the data access logic**.
- The **business logic can be invoked via multiple adapters** each of which implements a particular API or UI.
- The **business logic can also invoke multiple adapters**, each one of which invokes a different external system.
- Hexagonal architecture is a great way to **describe the architecture of each service in a microservice architecture in terms of Kruchten's logical view**.

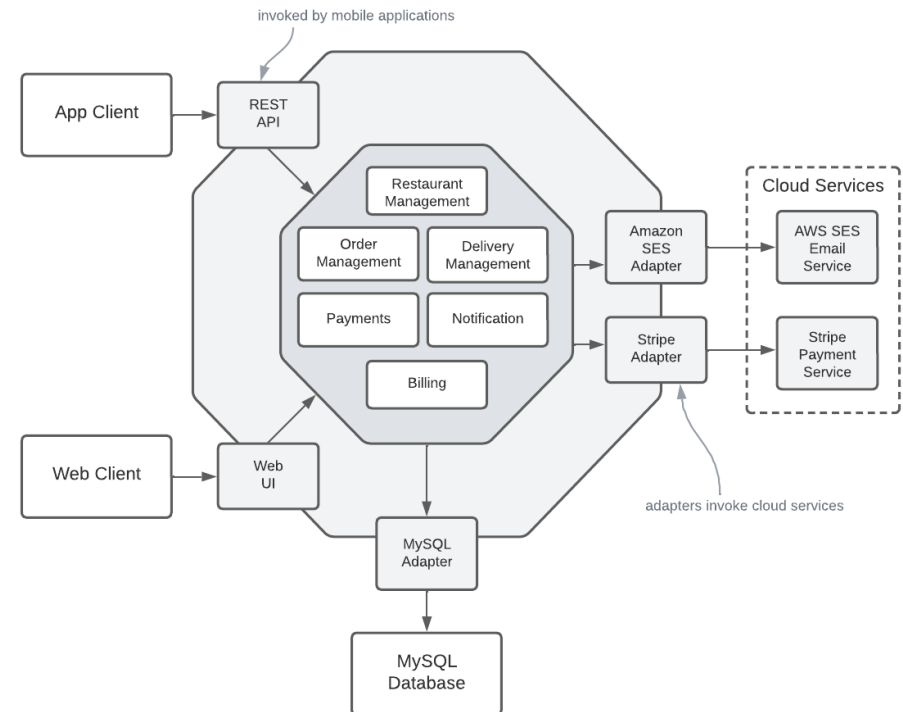
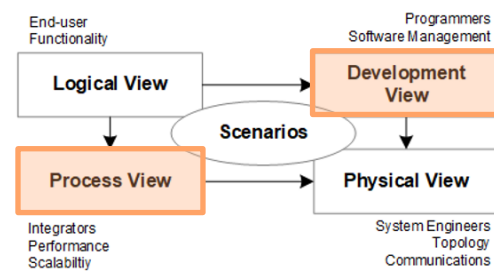


Architecture Patterns

Hexagonal Architecture (Monolith)

Also **Monoliths** frequently **adapt** a **hexagonal architecture style** above their **lower level modularization approaches**.

- **Monolithic Architecture** is an Architecture Style in which the **monolithic nature** of an overall **solution becomes** particularly **evident** in the **development view** (→ single deployment unit) and in the **process view** (→ single operating process and infrastructure).
- At the same time, **meaningful modularization of components** can certainly **still be found within singular deployment units** (i.e., modular monolith).



Lecture Agenda

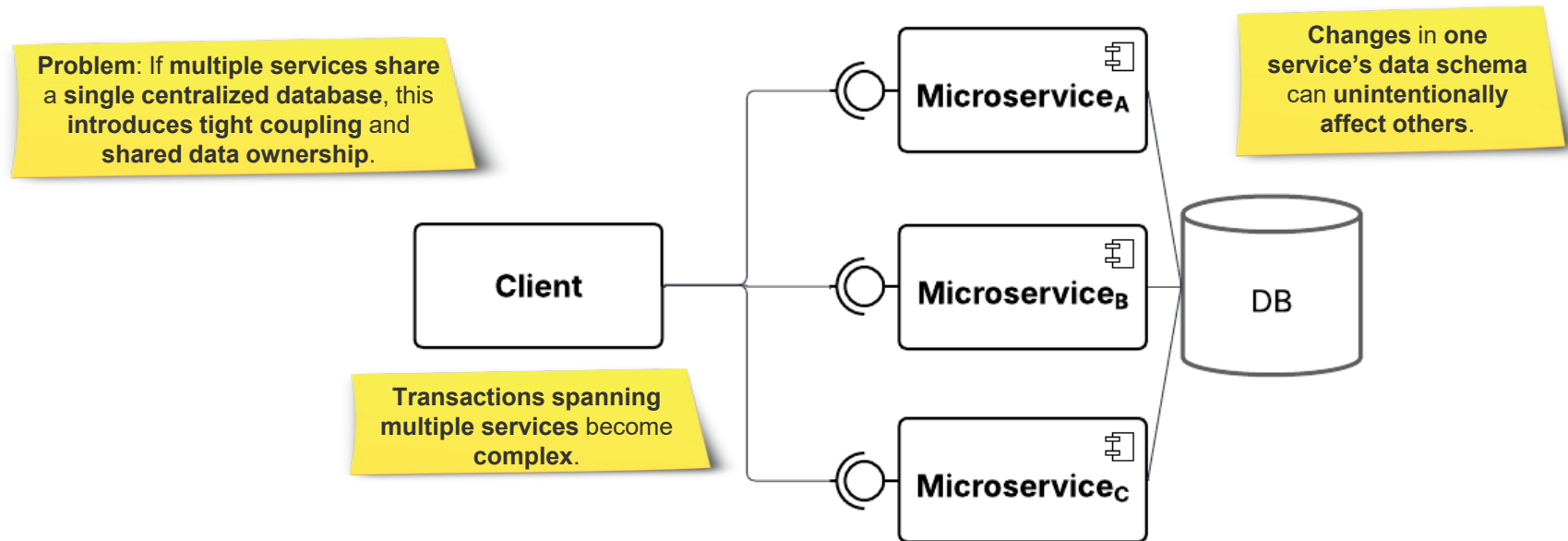


- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

Persistence

Database per Service Pattern

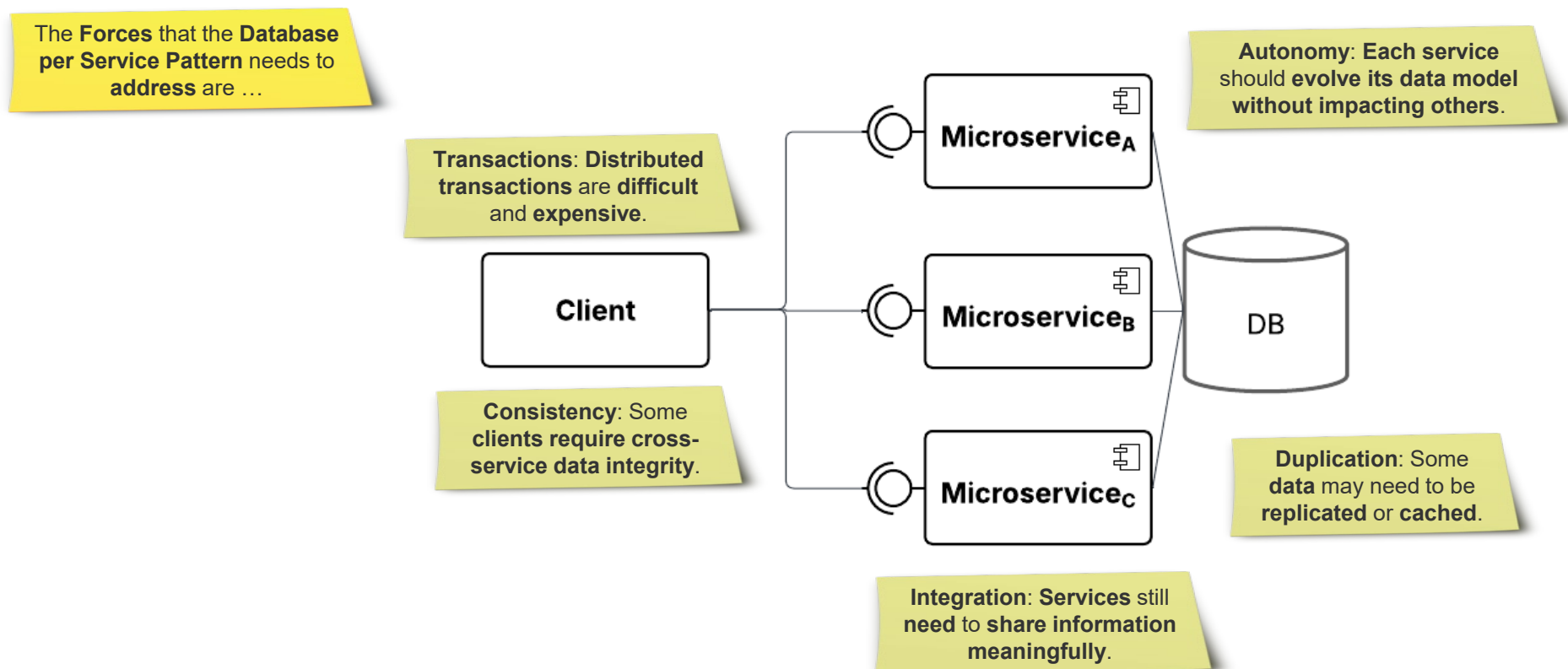
In a **Microservice architecture**, each service is supposed to manage its own **business functionality** and respective **data**.



Persistence

Database per Service Pattern

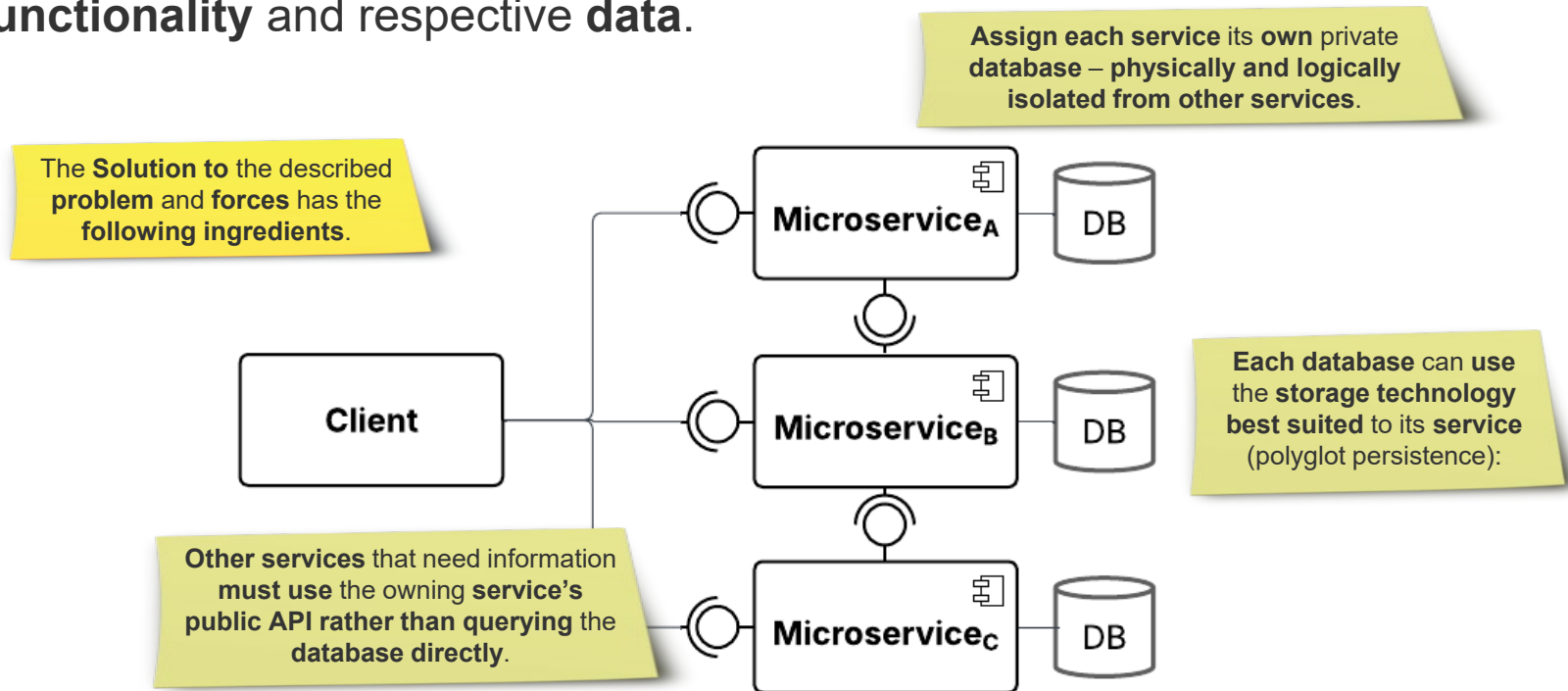
In a **Microservice architecture**, each **service** is supposed to manage its own **business functionality** and respective **data**.



Persistence

Database per Service Pattern

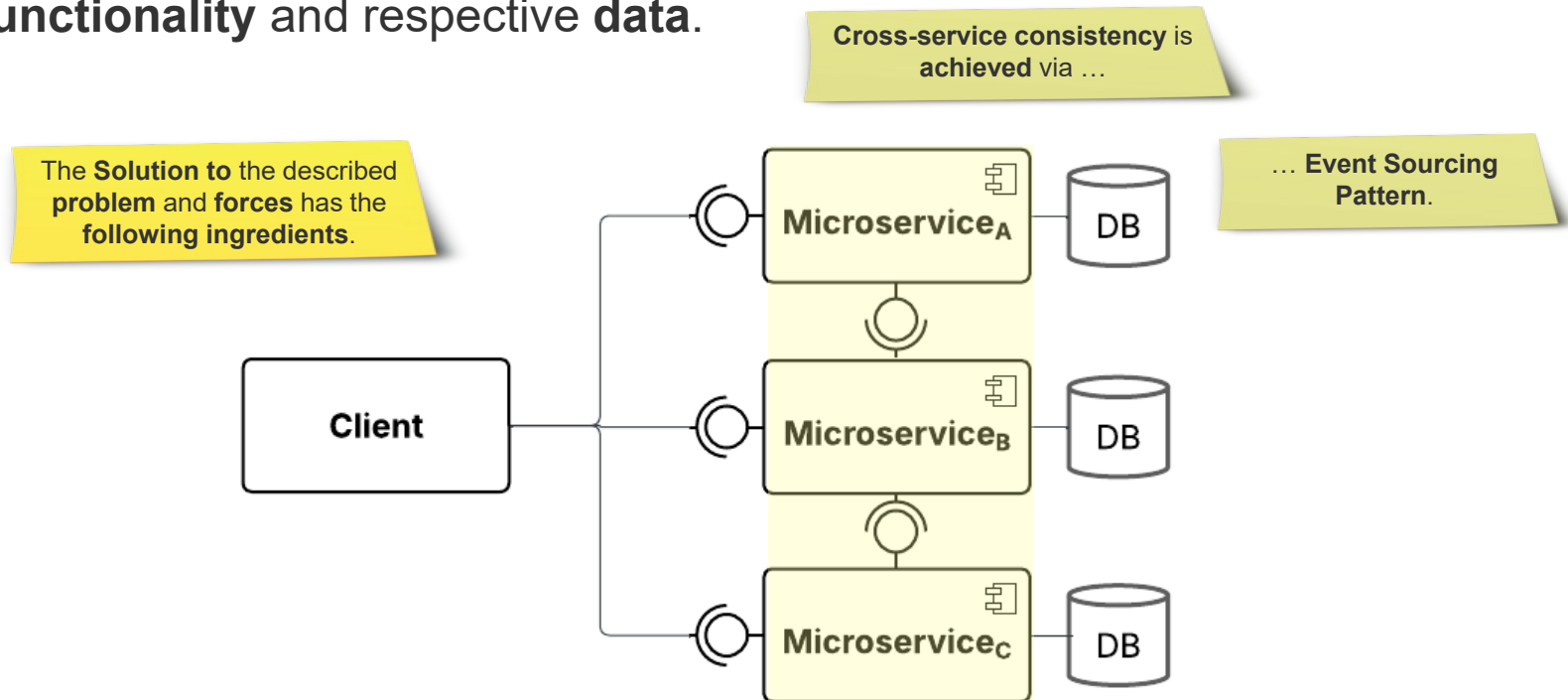
In a **Microservice architecture**, each service is supposed to manage its own **business functionality** and respective **data**.



Persistence

Database per Service Pattern

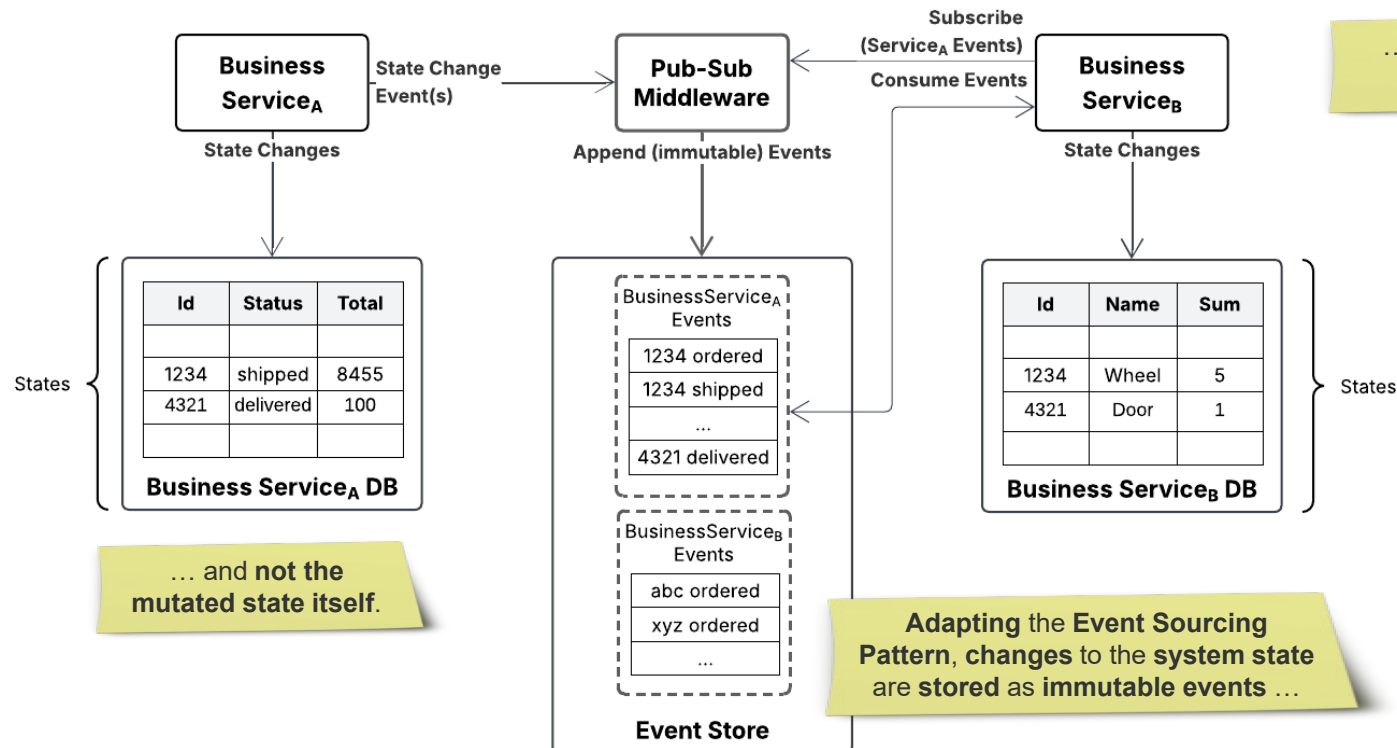
In a **Microservice architecture**, each **service** is supposed to manage its own **business functionality** and respective **data**.



Persistence

Database per Service Pattern

In a **Microservice architecture**, each service is supposed to manage its own **business functionality** and respective **data**.



Persistence

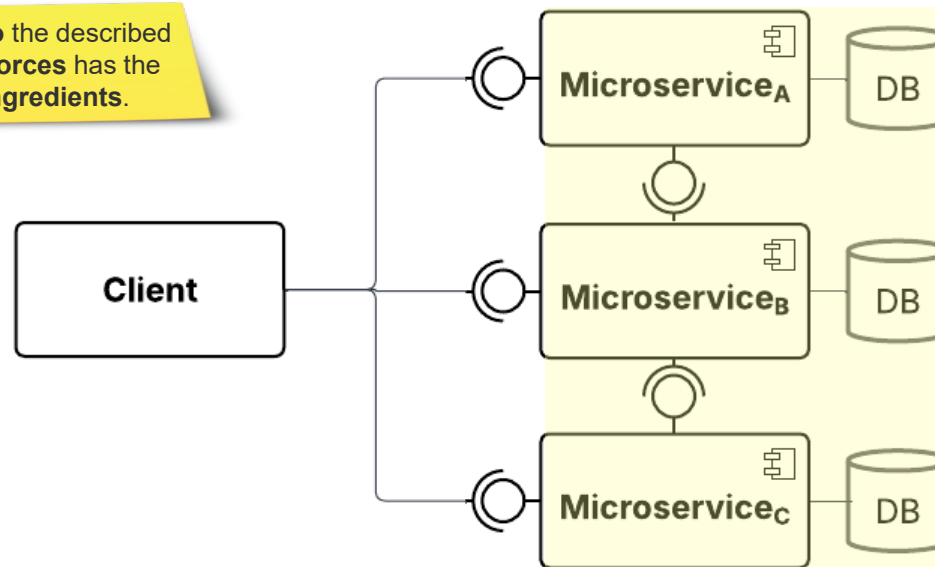
Database per Service Pattern

In a **Microservice architecture**, each **service** is **supposed to manage its own business functionality** and respective **data**.

Cross-service consistency is further achieved via ...

The **Solution** to the described problem and **forces** has the following ingredients.

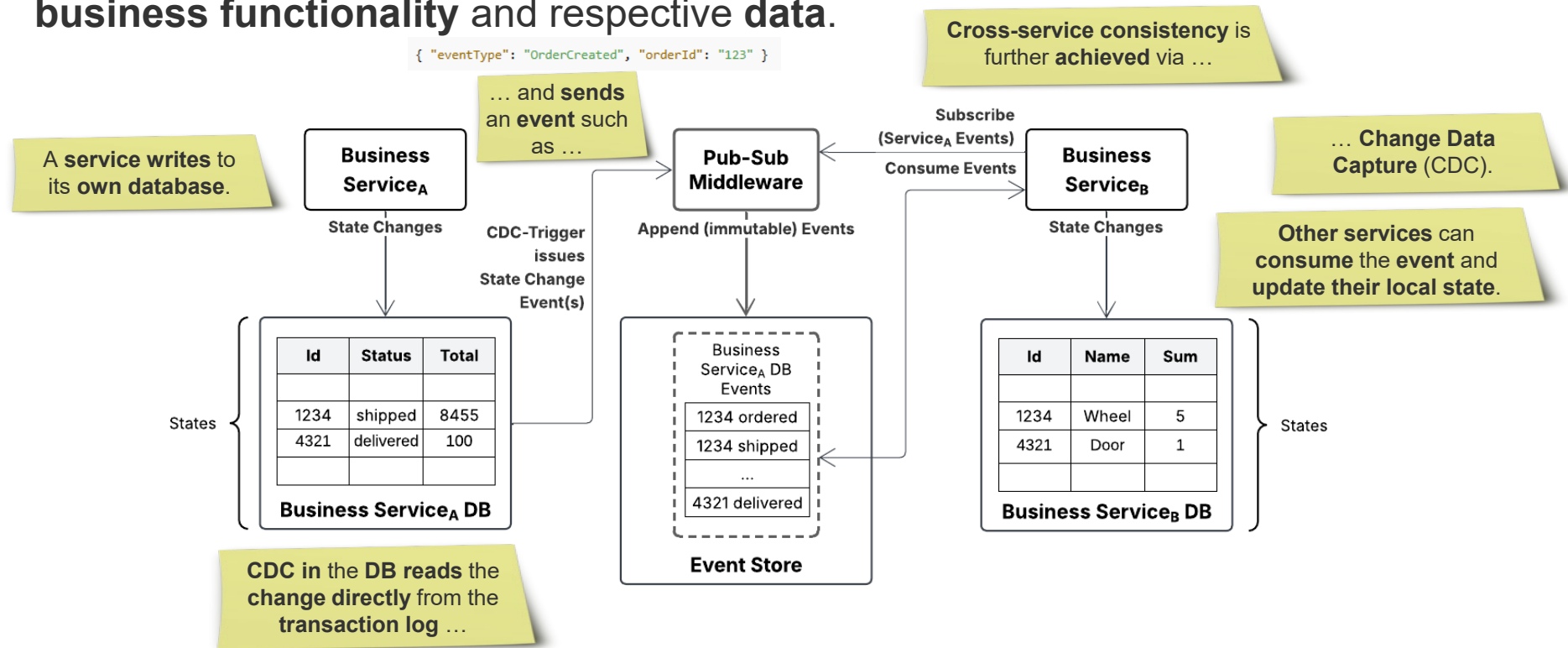
... **Change Data Capture (CDC)**.



Persistence

Database per Service Pattern

In a **Microservice architecture**, each service is supposed to manage its own **business functionality** and respective **data**.



Persistence

Database per Service Pattern

In a **Microservice architecture**, each **service** is supposed to manage its own **business functionality** and respective **data**.

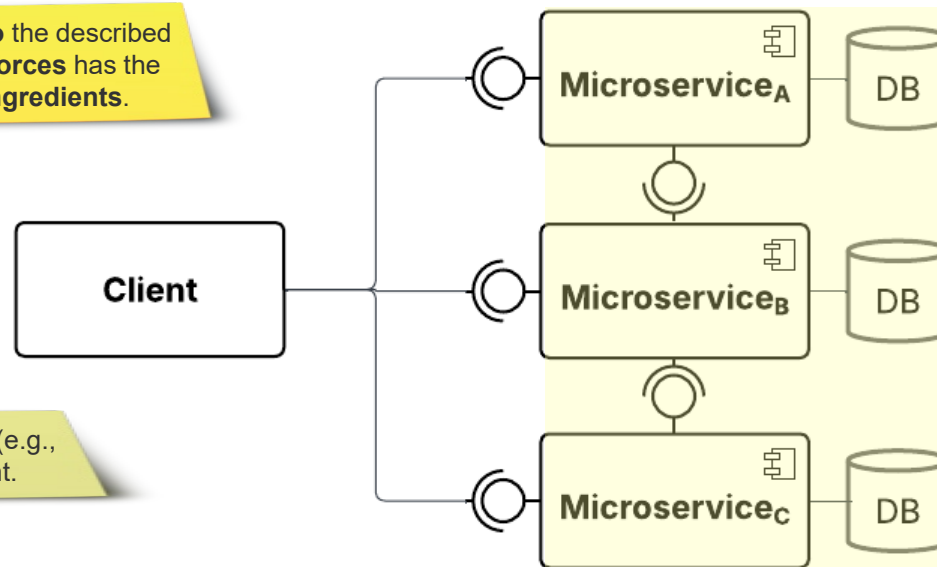
The **Solution** to the described problem and **forces** has the following ingredients.

Instead of a global two-phase commit, a **saga** uses **events** and **compensation actions**.

Step 1: Each service performs a **valid operation locally** (e.g., Order Service saves an order).

Step 2: **Services communicate via events** (e.g., Order Services sends OrderCreated Event).

Step 3: **Compensation actions** are **executed in case of errors** (e.g., Release order reservation in case the payment failed).



Cross-service consistency can also be achieved via ...

... the **Saga Pattern**

A **saga** coordinates a **distributed business transaction** that **spans multiple services** by dividing the **overall transaction** into **local ACID transactions** (i.e., local per service).

Lecture Agenda



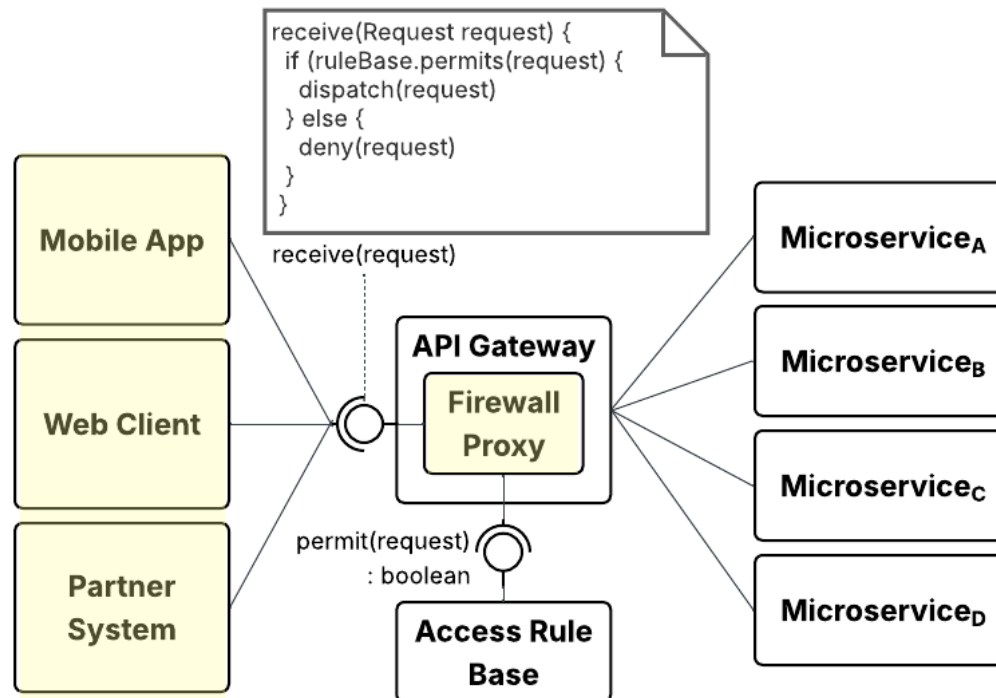
- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

Clients can only access microservices via the API Gateway with an integrated Firewall Proxy.

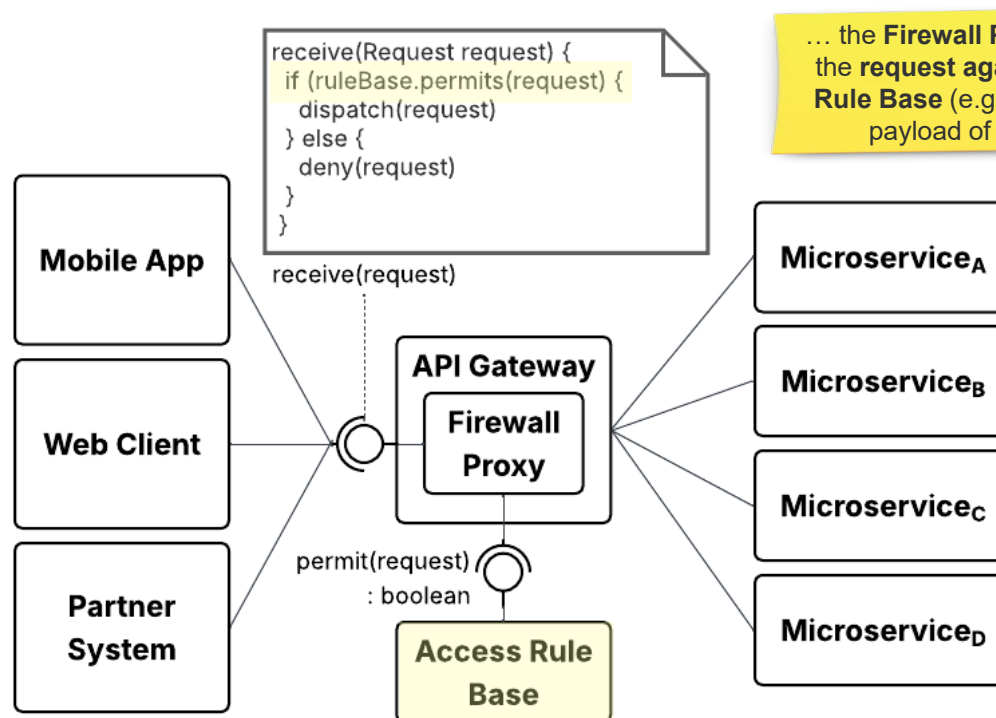


Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a security proxy that is often integrated into **API gateways** to control access to microservices.

When a client sends a request to a microservice and the client's authorization has not yet been checked (see **Stateful Firewall Proxy**) ...

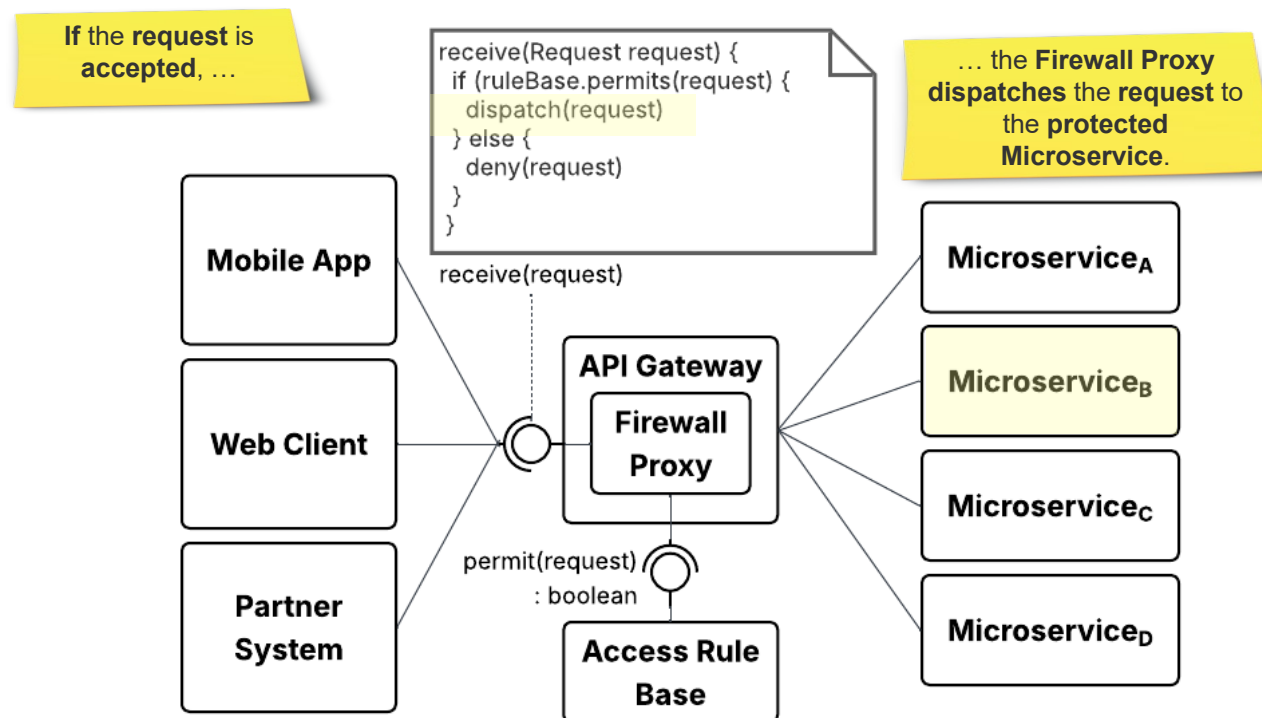


... the **Firewall Proxy** first checks the request against the **Access Rule Base** (e.g., by checking the payload of the request).

Security

Firewall Proxy

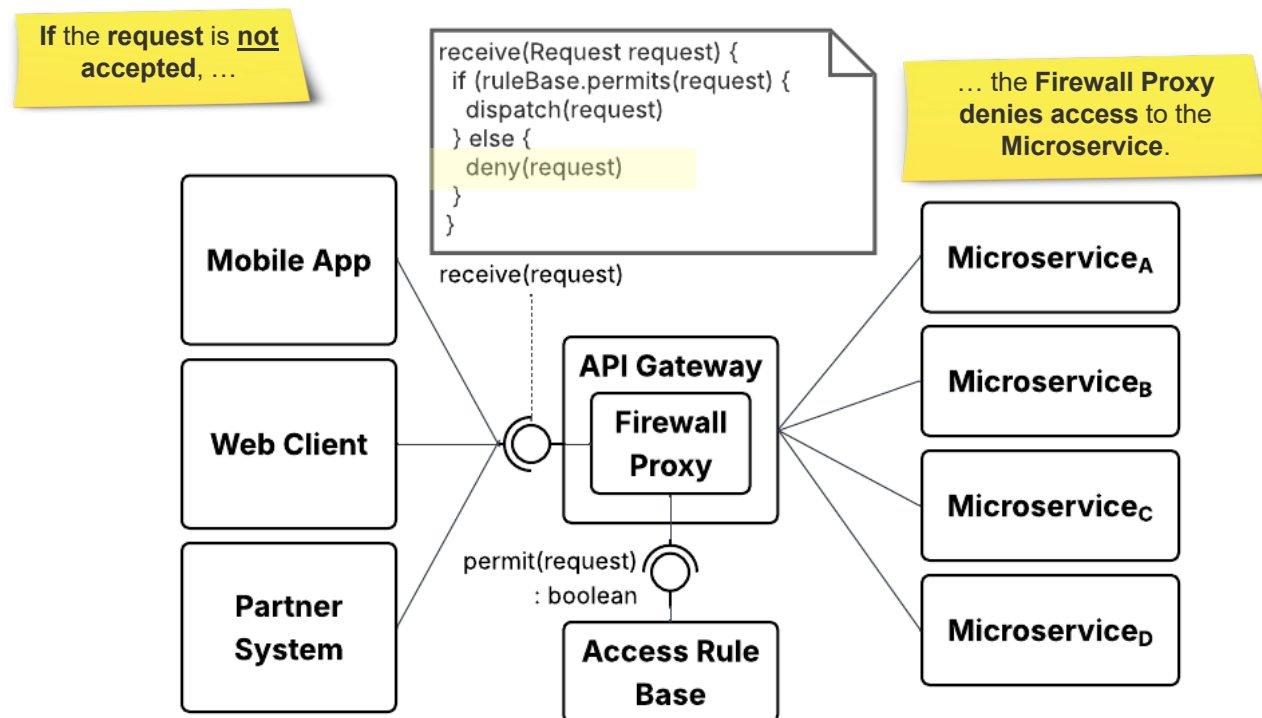
The **Firewall Proxy Pattern** introduces a security proxy that is often integrated into **API gateways** to control access to microservices.



Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a security proxy that is often integrated into **API gateways** to control access to microservices.



Security

Firewall Proxy

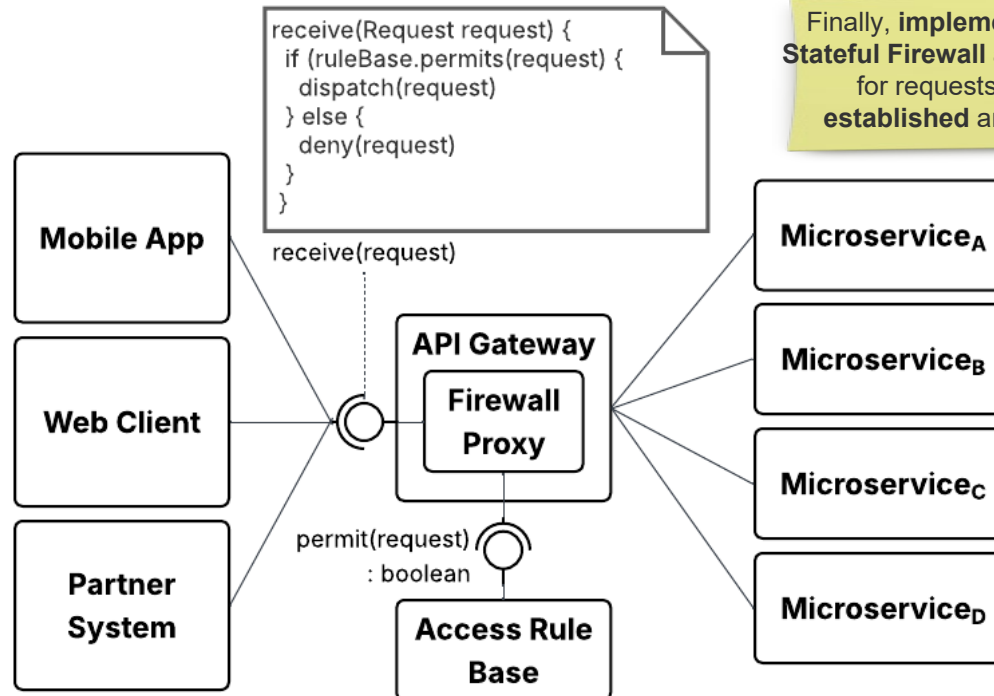
The **Firewall Proxy Pattern** introduces a security proxy that is often integrated into **API gateways** to control access to microservices.

Very **generally**
spoken ...

... **Firewall Proxy** supports the
detailed filtering of service
requests for **Microservices** on
a **packet** and **payload** level ...

... which helps to identify
cyber attacks above
networking protocols

Further, **security checks** are
separated from **Microservices**
which addresses the **Separation**
of **Concerns (SoC)** principle.



Finally, implementing **Firewall Proxy** as a
Stateful Firewall avoids payload inspections
for requests received from **already**
established and checked connections

The **security policy** applied by a **Firewall**
Proxy is often captured in a **rule base** that
is **separate** from the **proxy** itself ...

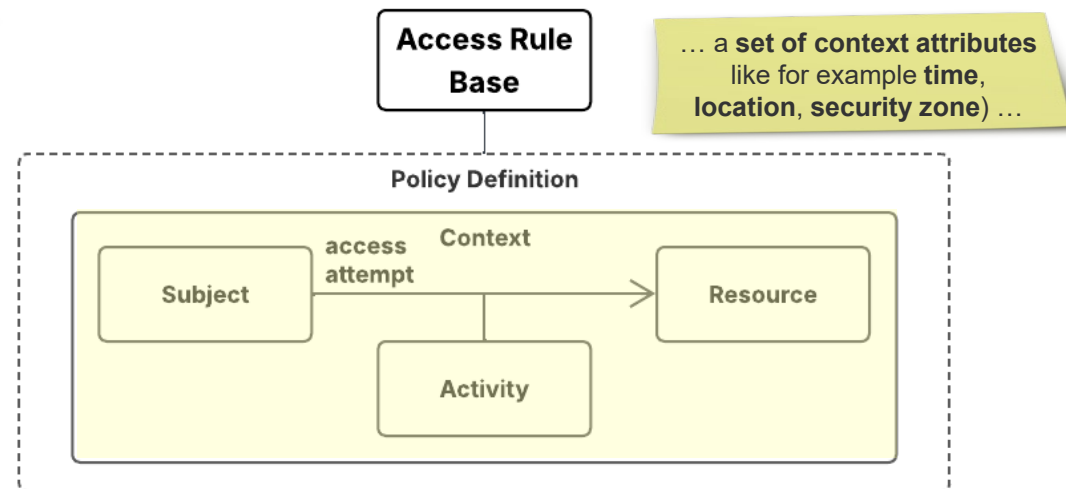
... which supports **declarative variation** in
security policies (i.e., without modifying
the proxy implementation).

Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

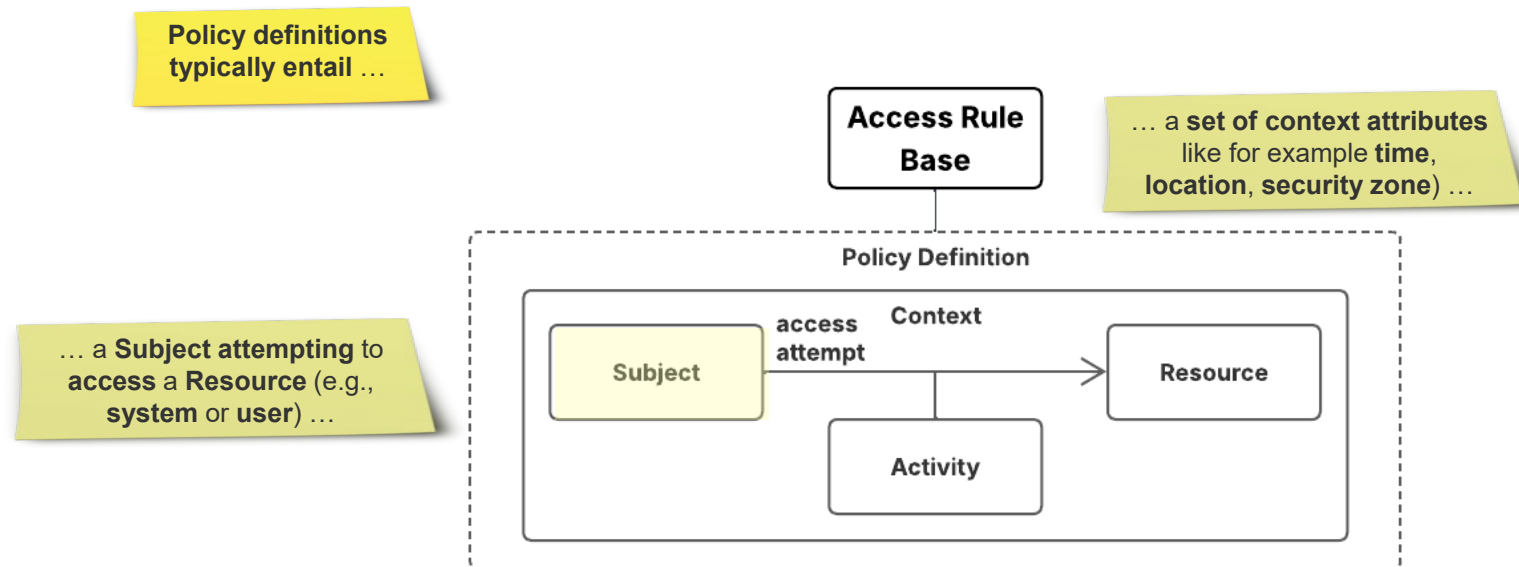
Policy definitions
typically entail ...



Security

Firewall Proxy

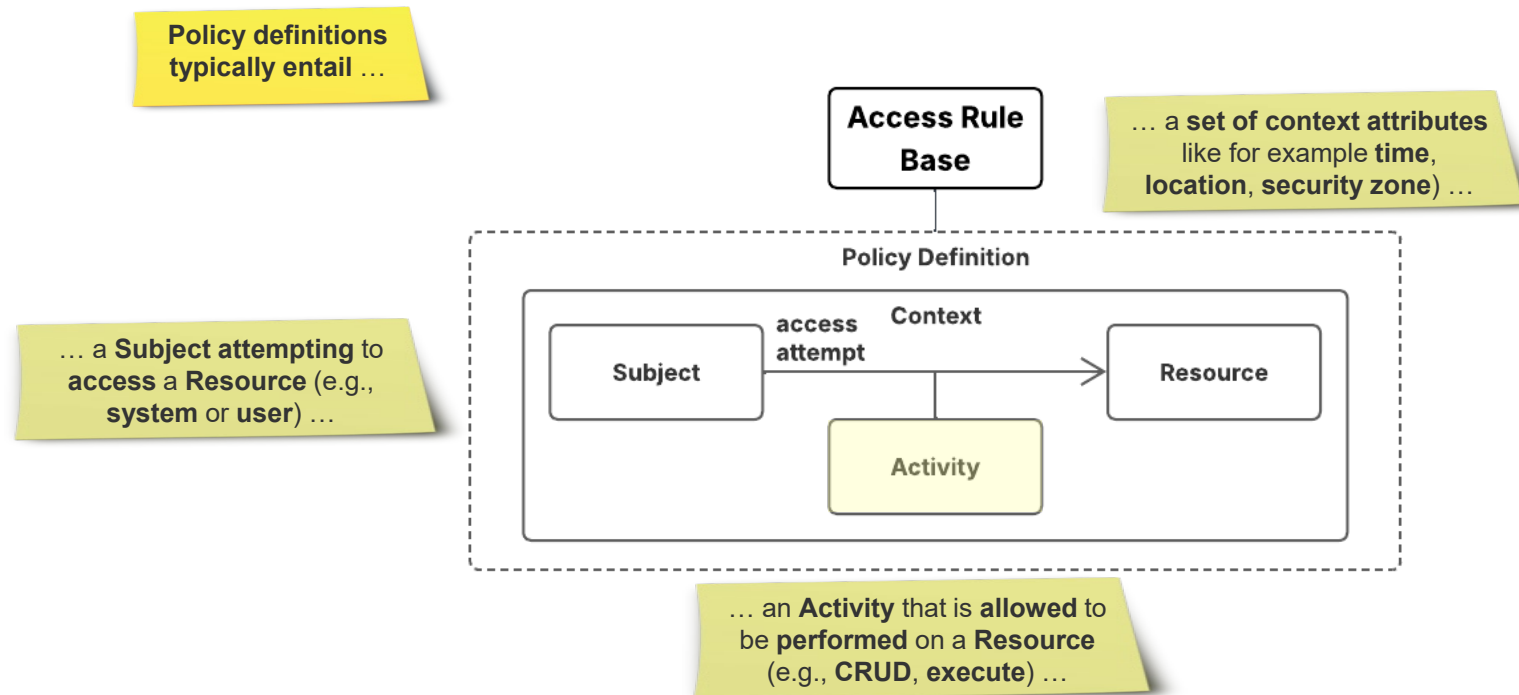
The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.



Security

Firewall Proxy

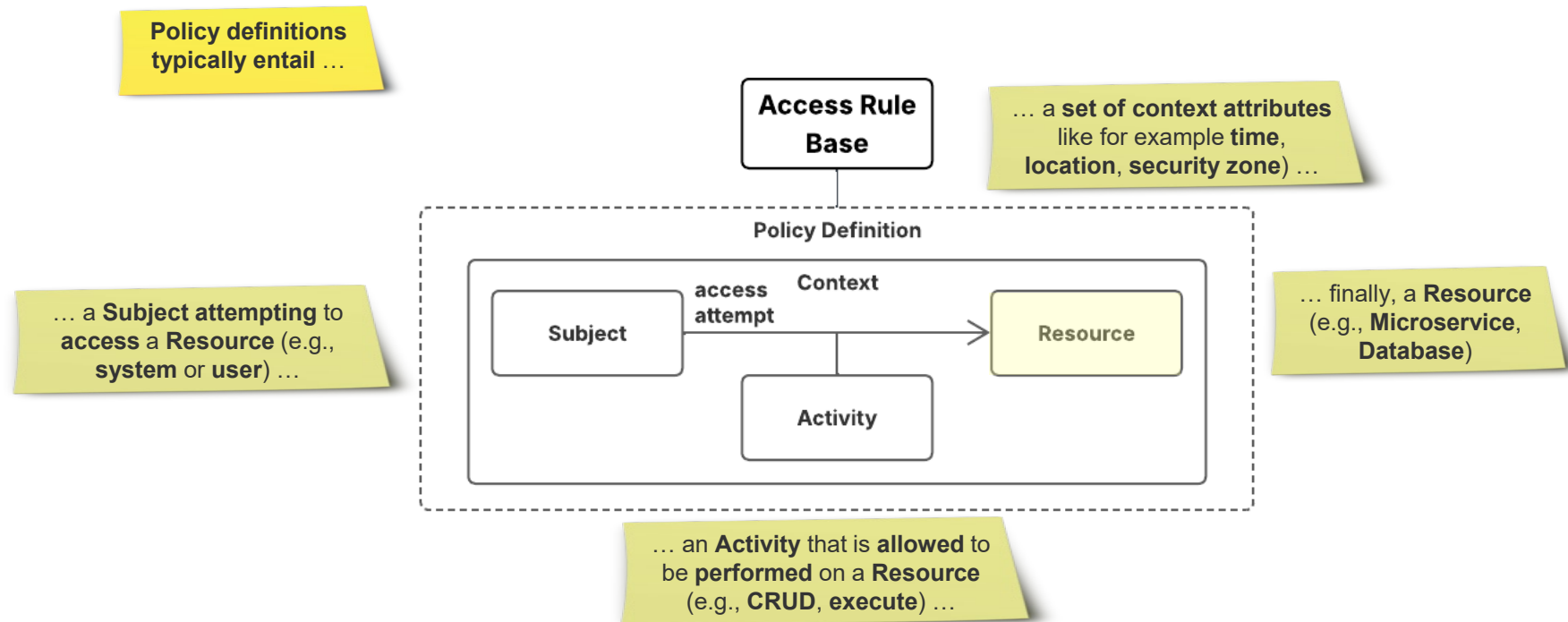
The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.



Security

Firewall Proxy

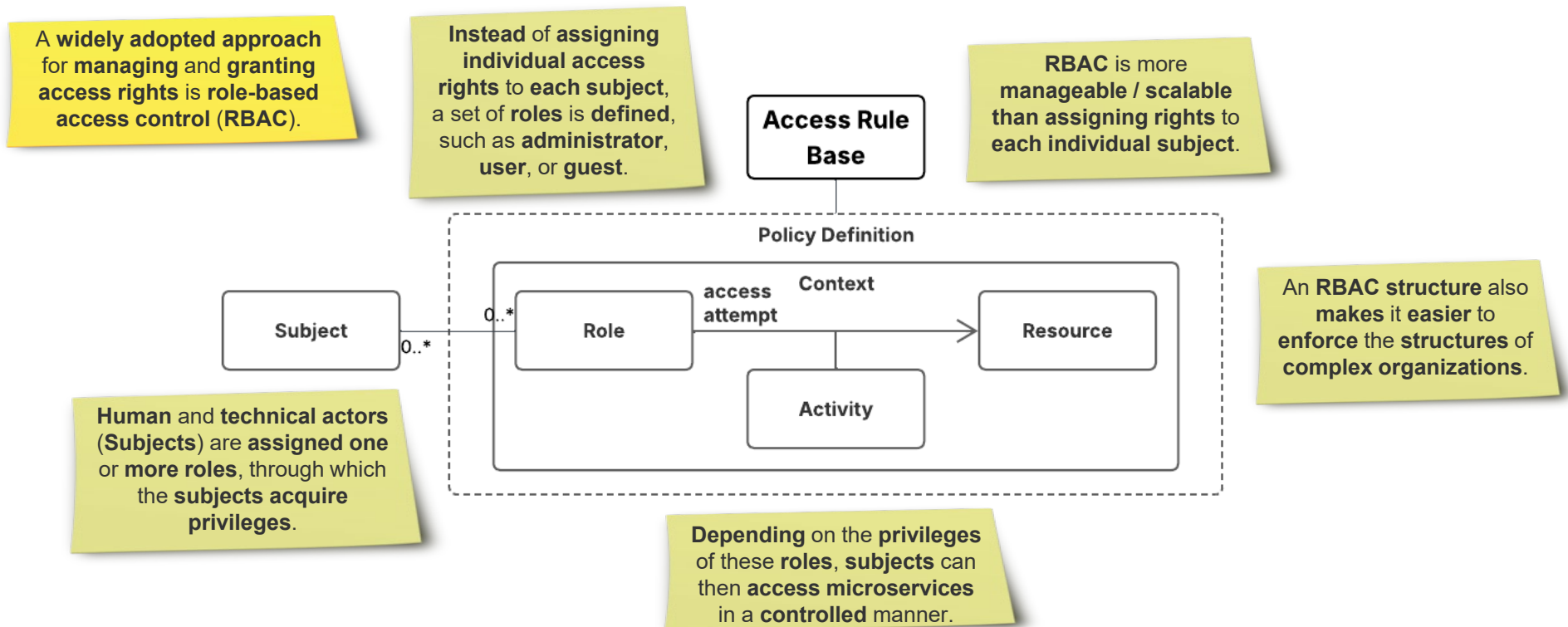
The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.



Security

Firewall Proxy

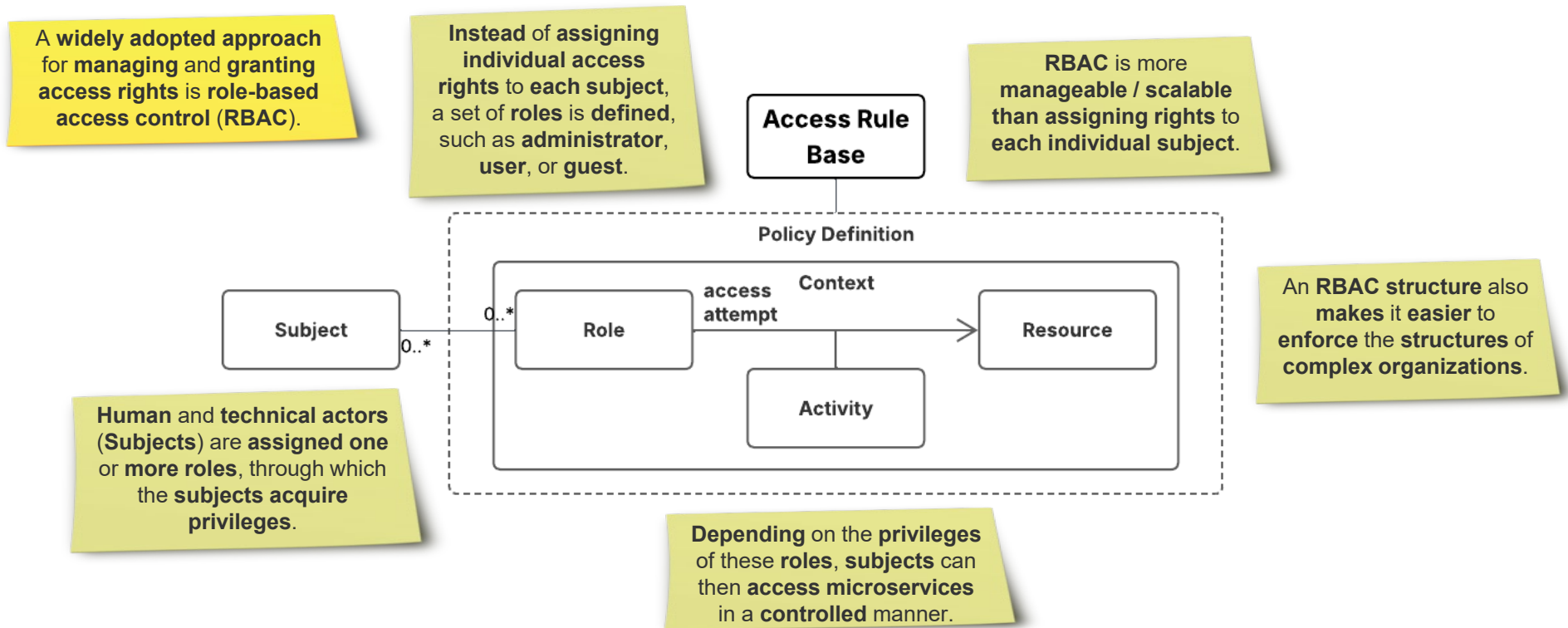
The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.



Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.



Security

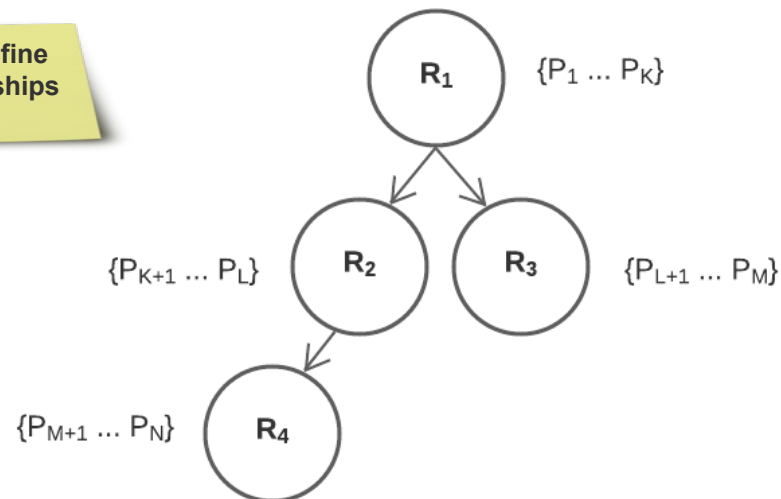
Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

RBAC also often supports role hierarchies.

Role hierarchies define inheritance relationships between roles.

In this example, permissions (P_i) are assigned based on roles.



A Subject assigned the role R_2 has the (composite) permissions of R_1 and R_2 .

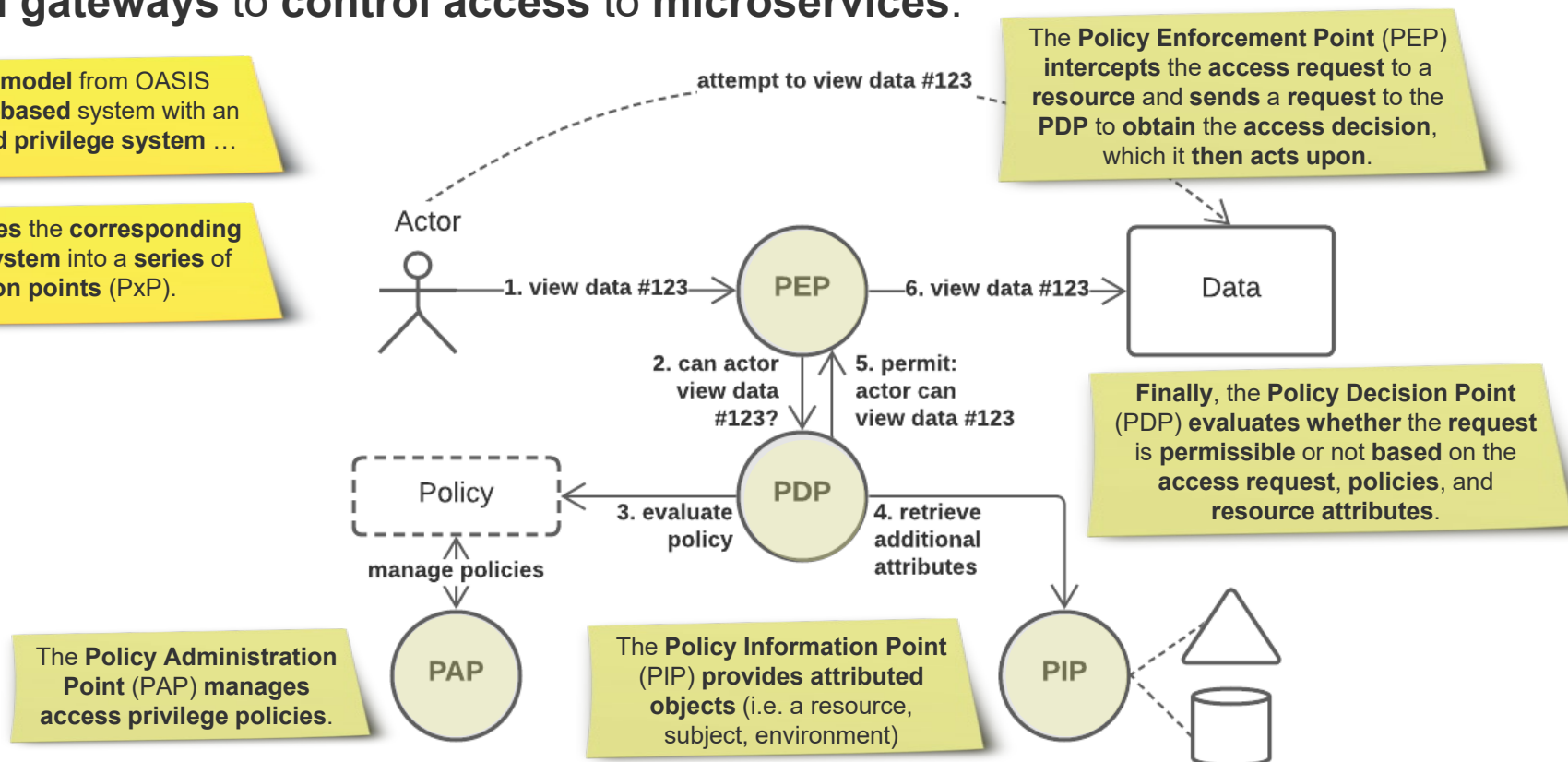
Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

The **XACML** model from OASIS extends a role-based system with an attribute-based privilege system ...

... and divides the corresponding privilege system into a series of function points (PxP).

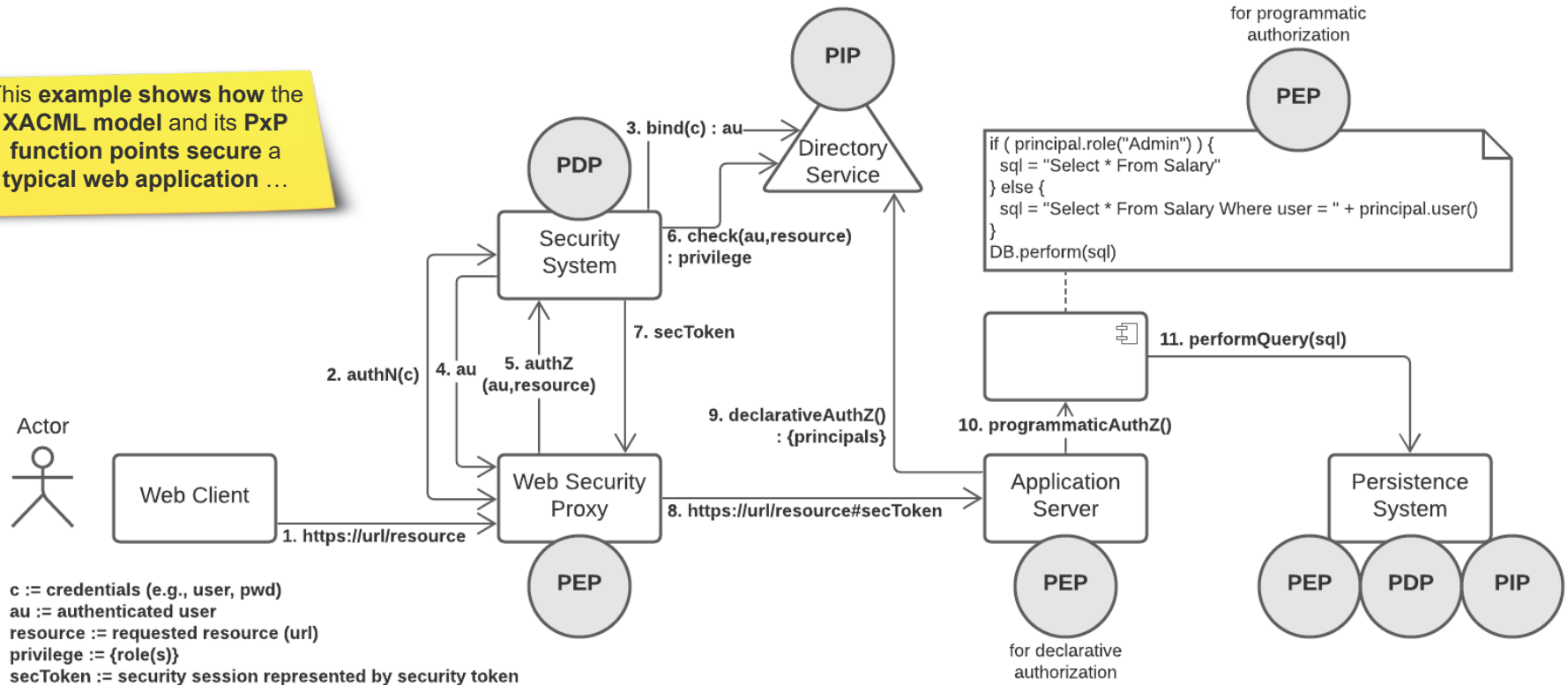


Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

This example shows how the **XACML model** and its **PxP** function points secure a typical web application ...

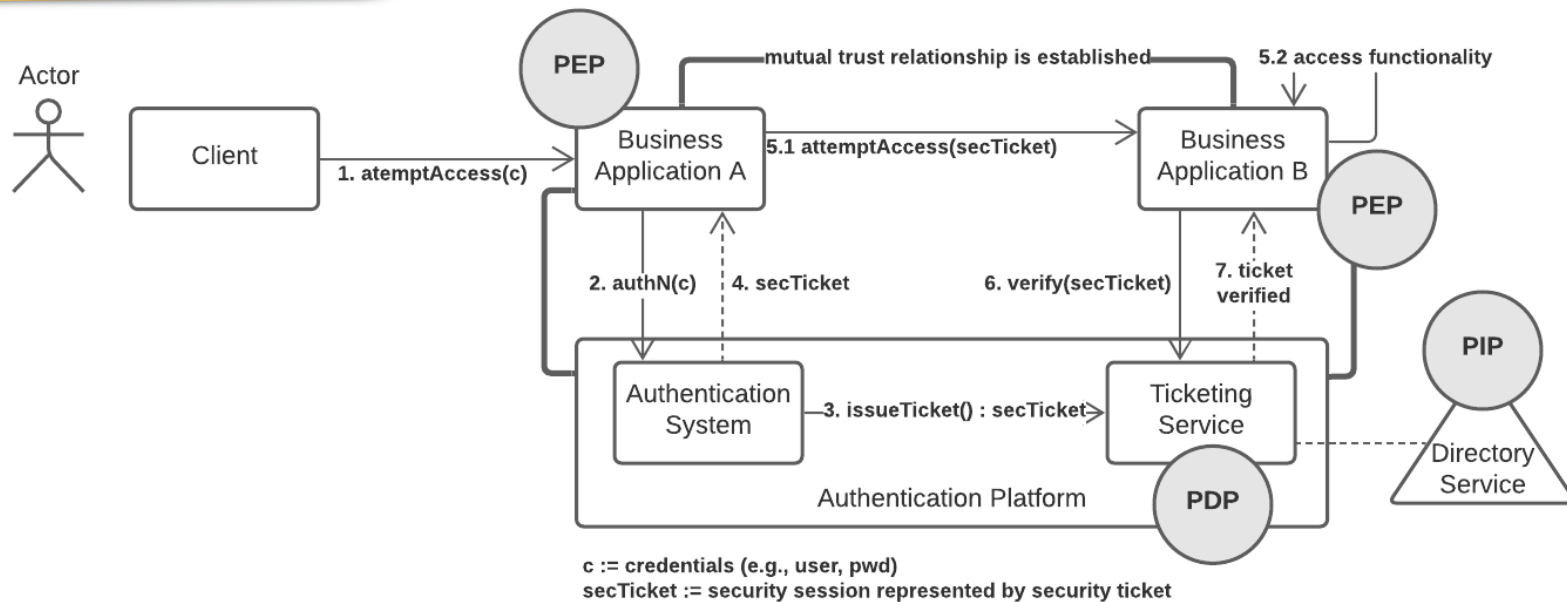


Security

Firewall Proxy

The **Firewall Proxy Pattern** introduces a **security proxy** that is often **integrated into API gateways** to **control access to microservices**.

... while this **example** illustrates a **typical application to application security setup**.



Lecture Agenda



- Motivation
- Definition
- 4+1 View Model Context
- Architecture Patterns
- Persistence
- Security
- Organizational Impact

Organizational Impact

Organizational Replication

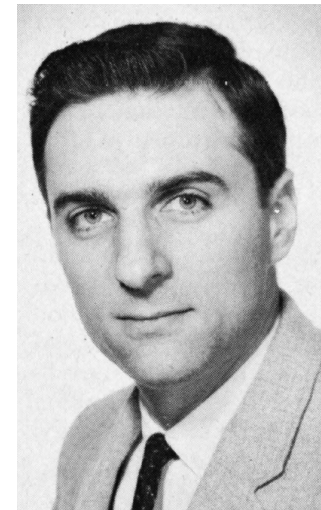
Melvin Conway researched and described the phenomenon of organizational replication (i.e., the impact organizational forces have on Systems) already in 1968.

From **Conway's** publication
"How do committees
invent?" from 1968!

"Any organization that designs a system will inevitably produce a design whose structure is a copy of the organization's communication structure.

The very act of **organizing** a design **team means** that **certain design decisions have** already **been made**, explicitly or otherwise.

Given any design team organization, there is a class of design alternatives which cannot be effectively **pursued** by such an organization **because** the necessary **communication paths do not exist.**"



Organizational Impact

Organizational Replication

Melvin Conway researched and described the **phenomenon of organizational replication** (i.e., the impact organizational forces have on Systems) already in 1968.

Christopher Alexander presents a compelling **analogy** in *"The Timeless Way of Building"*.

"If you **want to make a living flower**, you do **not build it physically**, with tweezers, cell by cell. You **grow it from the seed**." [Alexander 1979]

Alexander **compares** the **relationship** between the **architecture of a system** and the **architecture of the organization** responsible for **designing it** with the **relationship** between a **living flower** and the **seed from which it grew**.



Organizational Impact

Organizational Replication

Melvin Conway researched and described the phenomenon of organizational replication (i.e., the impact organizational forces have on Systems) already in 1968.

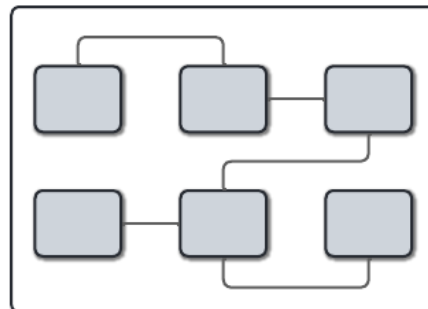
Organizations are like seeds,
and the **technical systems** they
use or produce are like flowers.



Organizations replicate their
inherent (communication) **designs**
in the **technical systems** they
use or produce.

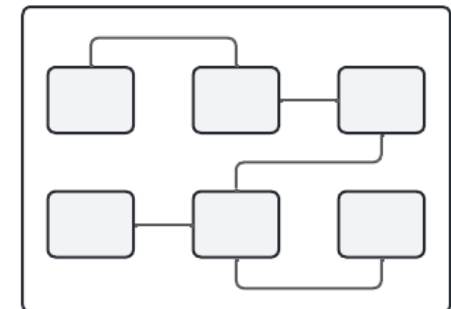
This means that an **organization's**
(communication) **design inevitably**
influences and significantly shapes the
design of their **technical systems.**

<<Social System>>
Organization



Communication Design of Social
System

<<Technical System>>
IT Solution



Component Design of Technical
System

Organizational
Replication

Organizational Impact

Organizational Replication

Melvin Conway researched and described the phenomenon of organizational replication (i.e., the impact organizational forces have on Systems) already in 1968.

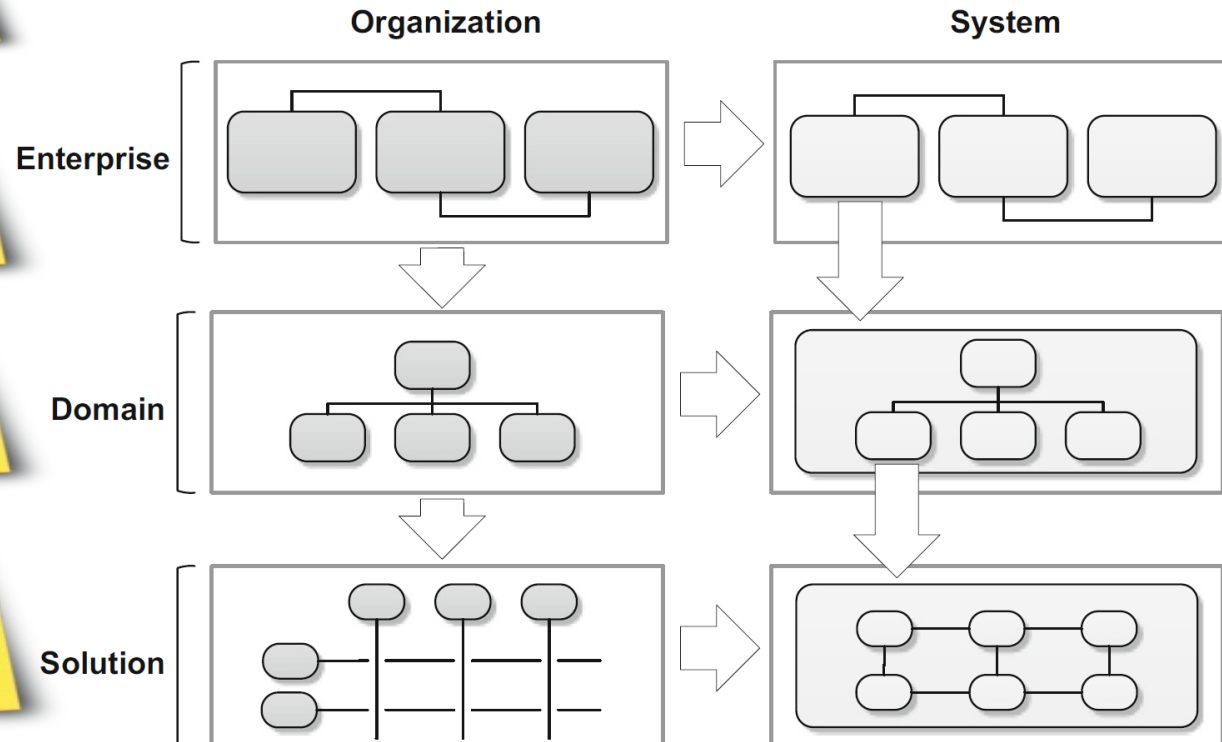
Organizational replication is a phenomenon you can observe at all levels of architecture.

The enterprise's organizational structure is replicated in your enterprise system architecture.

At the **enterprise level**, you can anticipate the architecture of enterprise systems by examining the design of the enterprise operating model.

At the **domain level**, you examine the structures of your domain organizations in order to anticipate the architecture of your **Big Systems** (aka: domains).

At the **solution level**, you explore the structures of projects or operationally responsible organizations to predict your landscape asset designs (e.g., the design of a new application).

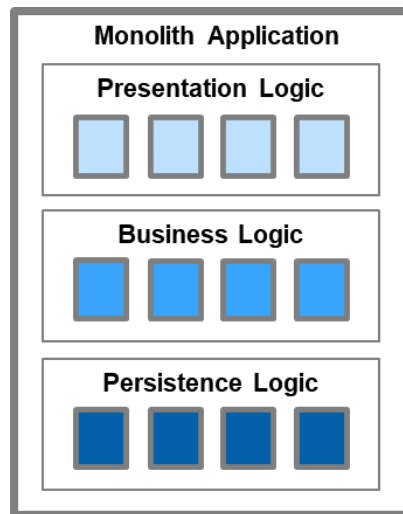


Organizational Impact

Organizational Replication and Microservices

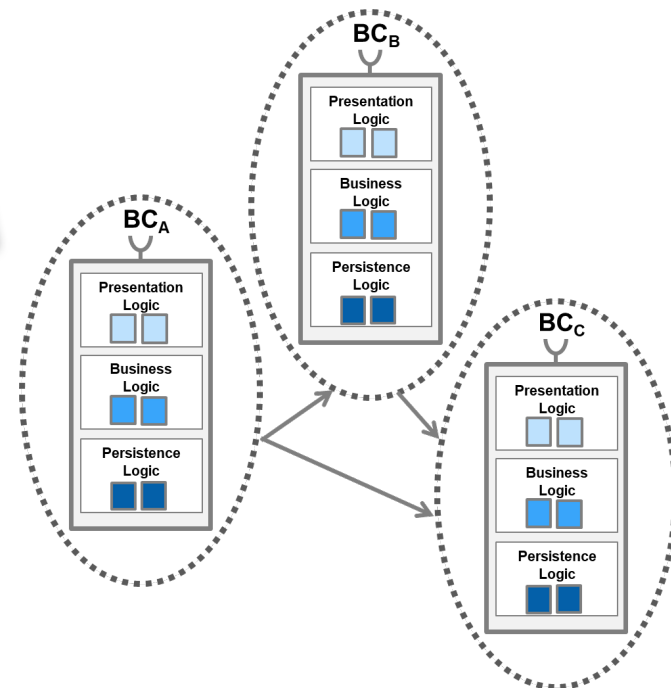
In the **context** the **organizational replication** phenomenon, a few **aspects** are to specifically be **considered**, regarding **microservices-oriented solution architecture**.

Technical organizations tend to orient themselves towards technical structures in software design (e.g., technically oriented layer architecture).



In the context of microservices, this approach must be avoided.

Rather technical organizations must adapt the organizational structures of the business organization to design microservices with a focus on business capabilities.



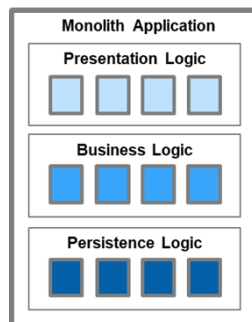
Organizational Impact

Organizational Replication and Microservices

In the **context** the **organizational replication** phenomenon, a few **aspects** are to specifically be **considered**, regarding **microservices-oriented solution architecture**.

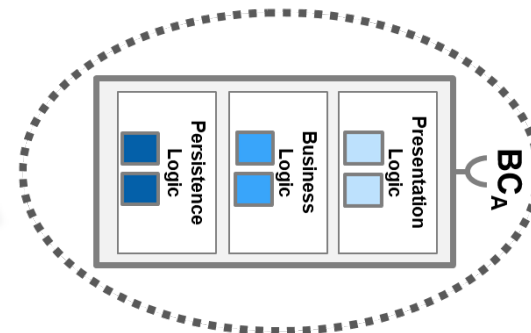
When **solutions** are **viewed** as larger **wholes**, their **development** is typically **carried out** within **corresponding projects**.

Project Orientation



However, **Microservices** suggest that **each service team** is **responsible** for its **services** throughout their **entire lifecycle** (i.e., based on Amazon's "**you build it, you run it**" principle).

Product Orientation



This **product mentality** **stems** from the **focus** on **business capabilities**.

Organizational Impact

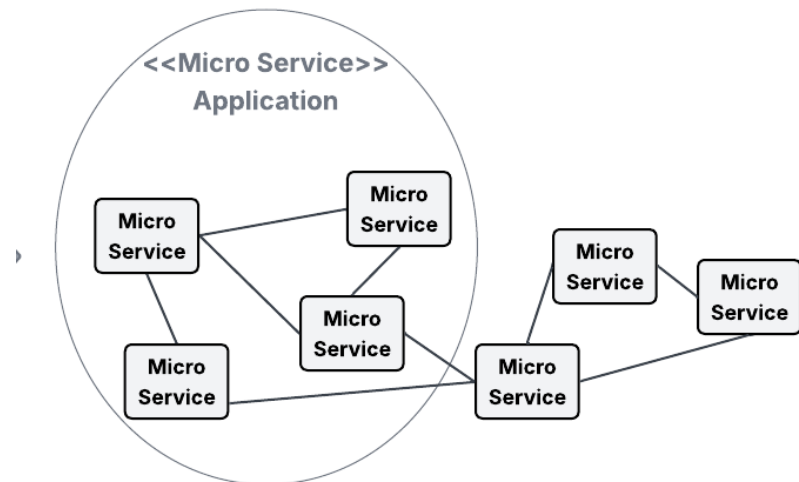
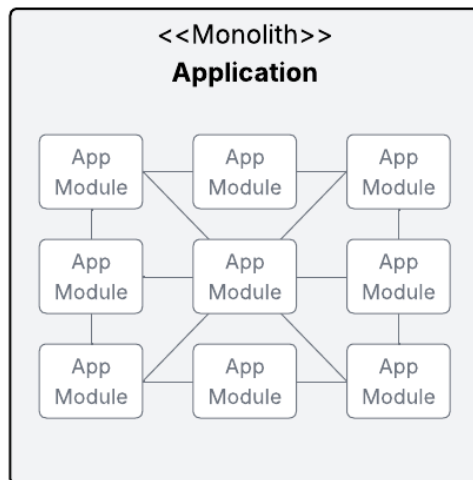
Organizational Replication and Microservices

In the **context** the **organizational replication** phenomenon, a few **aspects** are to specifically be **considered**, regarding **microservices-oriented solution architecture**.

The **advantage** of an **overarching application** that is **modularized only internally** is that it **offers** its **modules** a **comprehensive context**.

Contextual aspects that are **automatically shared** by **all modules** include, e.g., **value systems**, **privilege models**, or **domain ontologies**.

The **lack of an application-wide context** in **microservices** means that **services** must be **contextualized** much more **explicitly** in terms of **domain ontologies**, **privilege models**, **value systems**, etc., or that the **necessary contextual information** must be **organized** by the **services themselves**.



Organizational Impact

Organizational Replication and Microservices

In the **context** the **organizational replication** phenomenon, a few **aspects** are to specifically be **considered**, regarding **microservices-oriented solution architecture**.

As a **final point**, let's consider a **crucial prerequisite** for the **success** of a **microservice-oriented architecture** approach in an **organization**.

First, sharing **Microservices** with other parts of an **organization** requires a fundamental willingness to share.

Second, searching and reusing existing services requires a willingness to reuse something that an **organization** did not create itself.

Both of these **prerequisites** can be completely **counteracted** by an **organization's culture**, **incentive system**, etc.



Wrap-Up

■ Definition and Core Characteristics

- Microservice architectures consist of **loosely coupled, fine-grained services** that can be **developed, tested, deployed**, and **scaled independently**.
- **Communication** is **via lightweight protocols** (REST, events, messaging).
- **Advantages** are ...
 - Higher modularity
 - Scalability
 - Team autonomy
- **Disadvantages** are ...
 - Increased distributed system complexity
 - Observability challenges
 - Operational overhead

Wrap-Up

■ Architecture Patterns

• Communication Patterns ...

- **API Gateway**. Single entry point, routing, aggregation, security, protocol translation.
- **Service Registry**. Manages dynamic service endpoints for discovery.
- **Messaging**. Asynchronous decoupling, MOM guarantees, XML-based messaging
- **Layered System in Microservices**. Domain-oriented vertical slices instead of horizontal mega-layers.
- **Scalability Patterns** (X/Y/Z). X: cloning operational nodes, Y: functional decomposition, Z: partitioning by data-segment.

• Hexagonal Architecture

- Business logic is isolated by (a) inbound ports/adapters (controllers, message subscribers) and (b) outbound ports/adapters (DB adapters, service proxies).
- Perfectly aligns with microservice design.

Wrap-Up

■ Persistence Patterns

• Database per Service ...

- Each Microservice owns its private schema and database.
- No shared Database access.
- Enables autonomy but requires new consistency mechanisms.

• Cross-Service Consistency Mechanisms

- **Event Sourcing Pattern** stores changes as events, not state. On the other side downstream services reconstruct local state.
- **CDC** (Change Data Capture) extracts Database changes from transaction logs and publishes them as events.
- **Saga Pattern** organizes distributed transactions via local ACID steps as well as compensation actions.

■ Security Patterns

• Firewall Proxy ...

- Filters requests before reaching microservices.
- Declarative rule base for payload inspection and access control.
- Stateful versions skip repeated checks for established sessions.

• RBAC

- Access rights assigned to roles, not individuals. Further, RBAC supports role hierarchies.

• XACML Model

- Privilege system with PAP, PIP, PDP, PEP supports attribute-based access control (ABAC) for complex policies.

Wrap-Up

■ Organizational Impact

• Conway's Law ...

- Organizations replicate their communication structures in system design.
- Microservices require business-aligned, not technically aligned, team structures.

• Product Teams (“You build it, you run it”)

- Each microservice team is responsible for the full lifecycle.
- Opposes project-driven delivery structures.

• Loss of Application-Wide Context

- Without a monolith’s global context, microservices must explicitly maintain (a) domain ontologies, (b) privilege models as well as (c) value systems.

• Cultural Preconditions

- Success requires willingness to share and reuse services across the organization.

Questions

